



The Evolution of Attention in Large Language Models: Mechanisms, Trade-offs, and Emerging Trends

Zhentaο Tan, Jingyi Shen, Yanbo Li, Yao Liu, Yue Wu, Jieping Ye
Alibaba Token Hub, Alibaba Group

Abstract

Self-attention provides large language models with fine-grained, query-dependent access to contextual information, but dense token interactions incur quadratic prefill cost and require a key-value cache that grows with context length. Research has consequently evolved along several interacting directions, including explicit-memory compression, sparse access, recurrent state construction, structured state dynamics, and heterogeneous mechanism composition. This survey analyzes these developments through the shared notion of model-internal contextual memory. We introduce a five-dimensional analytical lens—Memory Representation, Memory Update, Access, Readout, and Integration—that distinguishes what information remains represented, how it changes, what is made eligible for a query, how eligible memory is read, and how one or more completed readouts are transformed or coordinated into the module output. This lens enables historically distinct and overlapping research lines to be compared without reducing them to a single computational model.

We use this perspective to reconstruct mechanism-level developments and examine their architectural adoption through a longitudinal inventory of 59 release-level records spanning 14 major model lineages, complemented by a frozen comparison of 11 high-performing open-weight model endpoints. A three-level synthesis emerges. At the mechanism level, explicit-memory and recurrent-state methods retain different memory interfaces, yet increasingly extend design control across an overlapping set of memory functions. At the architecture level, designs remain heterogeneous, but their coordination increasingly operates along network depth: layer-wise composition distributes complementary memory processing across representational stages, while emerging cross-layer reuse allows selected memory and routing artifacts to persist into later layers. Together, these developments make network depth an emerging dimension along which contextual memory is constructed and managed. Looking forward, they motivate a stateful multidimensional memory-routing hypothesis in which persistent memory is organized across temporal scope, network depth, substrate type, and representation granularity, with coordinated Sparse Write and Sparse Read governing what information is maintained and what stored information contributes to each query. Overall, efficient sequence architecture design is increasingly concerned with the joint organization, lifecycle, and selective use of contextual memory rather than the optimization of an isolated Attention operator.

Contents

1	Introduction	4
2	A Unified Memory-Centric View of Attention	6
2.1	Five Analytical Dimensions	6
2.2	Classification Principles and Family-Level Mapping	9
3	Softmax Attention	10
3.1	Memory-Representation Efficiency	11
3.2	Sequence-Representation Compression	12
3.3	Readout and Integration Modulation	14
3.4	Summary	14
4	Sparse Attention	15
4.1	Structure-Constrained Sparse Attention	16
4.2	Self-Routing Sparse Attention	17
4.3	Auxiliary-Proxy Sparse Routing	21
4.4	Temporal and Cross-Layer Reuse	24
4.5	Summary	26
5	Linear Attention	26
5.1	Memory Update: From Accumulation to Controlled State Editing	27
5.2	Capacity Expansion	32
5.3	Temporal Expansion	33
5.4	Auxiliary Coordination and Routing Enhancements	35
5.5	Summary	36
6	State Space Models	36
6.1	State Dynamics and Selective Control	36
6.2	State Organization and Read-Write Refinements	40
6.3	Summary	41
7	Hybrid Architecture: Composing Heterogeneous Memory Mechanisms	42
7.1	Layer-wise Hybrid	42
7.2	Head-wise Hybrid	44
7.3	Branch-wise Hybrid	45
7.4	Token-wise Hybrid	47
7.5	Summary	47
8	Attention Designs in Publicly Documented LLM Architectures: Evolution, Coordination, and Frontier Adoption	48
8.1	Diversification Rather Than Convergence in Attention Design	48

8.2	Increasing Hybridization and Emerging Cross-Layer Reuse	49
8.3	Attention Designs among High-Performing Open-Weight Models	50
8.4	Summary	50
9	Synthesis Across Mechanisms, Architectures, and Future Directions	51
9.1	Mechanism Level: Distinct Starting Points and Expanding Control Scopes	51
9.2	Architecture-Level Synthesis: From Complementary Placement to Memory-Artifact Lifecycles	52
9.3	Forward-Looking Hypothesis: Stateful Multidimensional Memory Routing	53
10	Conclusion	55
A	Supplementary Materials: Publicly Documented Architecture Inventory	57

1 Introduction

The Transformer replaced sequential recurrent propagation with self-attention, allowing each token to aggregate contextual information through direct, content-dependent interactions [1]. This mechanism enabled highly parallel training and established the computational foundation of modern large language models (LLMs) [2]. Its explicit representation of preceding tokens provides fine-grained, query-dependent access to context and supports in-context learning, long-range dependency modeling, and flexible reuse of information within a sequence. The same interaction pattern, however, creates the principal scaling limitations of dense attention. During prefill, the number of query-key comparisons grows quadratically with sequence length; during autoregressive decoding, the key-value (KV) cache grows linearly with the accumulated context. Moreover, a longer nominal context window does not by itself ensure reliable use of distant evidence: an expanding candidate set can increase retrieval interference and weaken long-range recall [3, 4]. These limitations have driven the evolution of attention along several interacting architectural directions, including approaches that store token memories more compactly, consult only selected past positions, carry context in recurrent states, structure how those states evolve, and combine different mechanisms within one model.

We organize this literature into four mechanism-centered research lines and one composition-centered line¹:

- **Softmax Attention** stores context in an enumerable collection of memory units, such as token KVs, latent entries, or summaries, and retrieves from them through normalized query-key weights [1, 5, 6]. Recent variants share or compress these units across heads, layers, or time [7, 8, 9]. This reduces KV storage and data movement while preserving content-dependent retrieval, although cost still grows with the number and size of retained units.
- **Sparse Attention** keeps explicit content memory but lets each query consult only a subset of the available tokens or blocks [10, 11, 12]. Selection ranges from fixed patterns to learned or compressed indexes [13, 14], with some decisions reused across layers [15, 16]. This reduces score computation and memory traffic, but its effectiveness depends on retaining the evidence relevant to each query.
- **Linear Attention** folds the preceding context into one or more recurrent associative states rather than keeping a growing list of token memories [17]. Later work improves how these states retain and revise information [18, 19, 20] or expands their capacity and temporal coverage [21, 22, 23, 24]. This avoids a token-growing KV cache during decoding, but individual past tokens are no longer directly retrievable.
- **State Space Models** also carry context in recurrent states, but derive their updates from structured dynamical systems rather than an associative reformulation of attention. Their structured and input-dependent transitions seek to combine parallelizable training with bounded-state recurrent decoding. As with other compressed-state designs, individual past tokens are not preserved as separate retrievable entries [25, 26, 27].
- **Hybrid Architecture** retains context through combinations of explicit, sparse, linear, and state-space paths placed across layers, heads, branches, or tokens. They distribute fine-grained token retrieval, local modeling, compressed long-term memory, and computation across different parts of a model. This provides complementary capabilities within one architecture, while introducing additional placement, routing, and coordination decisions [28, 29, 30, 31].

These categories are intentionally *not* a mutually exclusive partition. Most sparse mechanisms, for example, still apply Softmax after selecting a subset of keys; linear or state-space modules may also use selective operations; and Hybrid Architectures may contain members of every other line. We therefore place each method in the chapter that best matches its principal technical contribution and lineage.

Positioning relative to existing surveys. Prior surveys offer several complementary ways to navigate this rapidly expanding literature. General Transformer surveys organize architectural variants and applications, while efficient-Transformer surveys emphasize computational patterns, approximation strategies, and complexity [32, 33]. Long-context surveys examine context extension, length extrapolation, training strategies, retrieval, and application settings [34, 35, 36]. More specialized reviews provide detailed treatments of efficient sparse and linear attention [37], state space models [38], and KV-cache management and compression [39]. More recently, an architecture-centric memory survey organizes LLM memory along representation form, update dynamics, and persistence, covering a broader range of implicit and explicit memory mechanisms beyond attention [40]. Table 1 summarizes the primary organizing principles of these complementary survey perspectives.

¹In this survey, *Softmax Attention*, *Sparse Attention*, *Linear Attention*, and *State Space Models* denote the four mechanism-centered research lines, while *Hybrid Architecture* denotes the composition-centered line. Other lowercase attention terms describe particular operators, access patterns, layers, heads, or paths rather than survey-level categories. Established method names retain their original capitalization.

Table 1: Representative survey perspectives relevant to this work and their primary organizing principles. The table summarizes the dominant perspective of each survey category rather than its complete coverage.

Survey perspective	Primary organizing principle
General / efficient Transformers [32, 33]	Architecture families, computational complexity, and approximation patterns.
Long-context LLMs [34, 35, 36]	Context extension, length extrapolation, training, retrieval, and application strategies.
Efficient attention [37]	Sparse and linear attention algorithms, efficiency, and deployment.
State space models [38]	SSM lineage, state organization and dynamics, applications, and benchmarks.
KV-cache management [39]	Cache selection, compression, eviction, quantization, and execution efficiency.
Memory-centric LLM architectures [40]	Representation form, update dynamics, and persistence across a broad range of implicit and explicit memory mechanisms.

Taken together, these surveys provide complementary architecture-, efficiency-, context-, state-, cache-, and memory-centered views of the field. In comparison with broader memory-centered surveys, this survey focuses specifically on attention and adjacent sequence mixers. We adopt contextual memory as a shared unit of analysis and examine these mechanisms through five *analytical dimensions*: **Memory Representation**, **Memory Update**, **Access**, **Readout**, and **Integration**. In brief, they ask what historical information remains represented, how the represented memory changes, what is made eligible for the current query, how eligible memory is read, and how one or more completed readouts are transformed or coordinated into the module output. The five dimensions and the chapter-level research lines serve different purposes: the chapter labels provide a readable historical and technical organization, whereas the analytical dimensions provide a multi-label description of the functions modified by an individual method. This lens thereby complements classifications based on architectural lineage, computational complexity, or application setting.

Scope and literature coverage. This survey focuses on model-level mechanisms used by autoregressive LLMs and closely related causal sequence mixers, with model-internal contextual memory as the shared unit of analysis. We cover representative work publicly available through September 22, 2026, identified through targeted literature searches, relevant prior surveys, backward and forward citation tracing, and primary technical documentation. The coverage is structured but not intended as an exhaustive systematic review: we emphasize foundational methods, major architectural transitions, and representative extensions that clarify recurring design choices across Memory Representation, Memory Update, Access, Readout, and Integration. Test-time learning, retrieval from external databases, multimodal-specific memory designs, and implementation-only optimizations remain outside the core taxonomy unless they directly alter the model’s contextual-memory semantics.

At the architecture level, the mechanism review is complemented by a curated longitudinal inventory of publicly documented model releases and a frozen comparison of high-performing open-weight model endpoints, both closed on September 22, 2026. The inventory is based primarily on official papers, technical reports, model cards, and released configurations. Models released together are grouped when they share the same language-model attention backbone, whereas separately released versions or documented changes to that backbone form separate records. Records with insufficient architectural disclosure are marked as undisclosed rather than inferred. Natively multimodal models are included only when the relevant autoregressive language backbone is documented; visual encoders, modality interfaces, and other modality-specific components are excluded from the classification. The inventory is purposively curated rather than exhaustive or market-share weighted, and the inventory and frontier comparison are used to characterize documented adoption and coexistence rather than to attribute model quality causally to an attention mechanism.

The principal contributions of this survey are fourfold:

1. We introduce a five-dimensional analytical lens—Memory Representation, Memory Update, Access, Readout, and Integration—for comparing how sequence mechanisms retain and use contextual memory. It provides a common vocabulary for Softmax Attention, Sparse Attention, Linear Attention, State Space Models, and Hybrid Architectures without reducing them to one computational model.
2. We reconstruct the development of Softmax Attention, Sparse Attention, Linear Attention, and State Space Models by examining how their representative methods intervene in Memory Representation, Memory Update, Access, Readout, and Integration, while preserving overlaps among these historically defined research lines.
3. We connect mechanism-level developments with two complementary analyses of publicly documented LLM architectures. A longitudinal inventory of 59 release-level records spanning 14 major model lineages traces the diversification of attention design, the growing use of layer-wise hybrid composition, and the emergence of cross-layer artifact reuse. A frozen comparison of 11 high-performing open-weight model endpoints pro-

vides a cross-sectional view of the attention structures represented near the performance frontier. Together, these analyses show continued architectural heterogeneity and the persistent role of explicit token retrieval in contemporary high-performing models.

4. We develop a three-level memory-centric synthesis of the surveyed literature. At the mechanism level, explicit-memory and state-based methods retain different memory interfaces while expanding design control across an increasingly overlapping set of memory functions. At the architecture level, layer-wise composition distributes complementary memory processing across representational stages, while cross-layer reuse extends the lifetime of selected memory and routing artifacts across those stages. Together, these developments make network depth an emerging dimension along which contextual memory is constructed and managed. At the forward-looking level, this depth-wise perspective combines with temporal scope, substrate type, and representation granularity to motivate a stateful multidimensional memory-routing hypothesis governed by coordinated Sparse Write and Sparse Read.

The remainder of this survey is organized as follows. Section 2 introduces the memory-centric analytical lens and its classification principles. Sections 3-6 examine Softmax Attention, Sparse Attention, Linear Attention, and State Space Models, respectively. Section 7 analyzes the composition of heterogeneous memory mechanisms across layers, heads, branches, and tokens. Section 8 examines architectural evolution and coordination through a longitudinal model inventory and a frozen comparison of high-performing open-weight models. Section 9 develops the mechanism-level and architecture-level syntheses and introduces the forward-looking multidimensional memory-routing hypothesis. Finally, Section 10 concludes the survey.

2 A Unified Memory-Centric View of Attention

When a model processes a token, it must make information from the preceding context available to the current computation. Different sequence architectures do this in visibly different ways. Softmax Attention retains separately addressable memory units; Sparse Attention limits which of those units are examined; and recurrent mechanisms continually compress the preceding context into one or more states. Looking only at these surface-level operators makes the families appear difficult to compare. At a functional level, however, each can be viewed as a system that maintains and uses internal memory while processing a sequence.

This memory-oriented interpretation is already present in several lines of research. Prior work has described linear attention as associative or fast-weight memory, related attention and state-space recurrences through their memory structure, and studied compressive, bounded, or growing recurrent memories [17, 27, 30, 41, 42, 43]. Building on this shared perspective, we use *contextual memory* as a common term for the model-internal, input-dependent information that remains available while a sequence is being processed. Depending on the architecture, that information may take the form of token KV representations, compressed tokens or chunks, memory slots, associative matrices, structured recurrent states, or heterogeneous combinations of these forms. The term does not refer to the model’s pretrained parameters, an external retrieval corpus, or biological memory.

Once these mechanisms are viewed in terms of contextual memory, their design differences can be organized around five questions:

1. **Memory Representation:** What information from the past is still represented, and in what form?
2. **Memory Update:** How does the current input add to, modify, compress, or overwrite that memory?
3. **Access:** Which represented information is eligible to be used for the current query?
4. **Readout:** How is the eligible information weighted, decoded, or aggregated?
5. **Integration:** How are one or more completed readouts transformed or combined into the module output?

These questions form the analytical lens used throughout this survey. They describe functional roles rather than five components that every architecture must implement separately. A single operation may perform several roles at once, and the roles need not appear as a fixed sequence of implementation stages. Figure 1 provides an architectural primer for this view: it illustrates the five dimensions through representative memory operations and maps them onto example Linear Attention and Sparse Attention blocks within a schematic layer-wise hybrid model. The concrete operations and block arrangement are illustrative rather than universal; the roles are defined formally below.

2.1 Five Analytical Dimensions

We now formalize the five questions one at a time. As a running setting, suppose that a model has processed the first $t - 1$ tokens of a document and is processing token t . It may retain every preceding token as a separate KV entry,

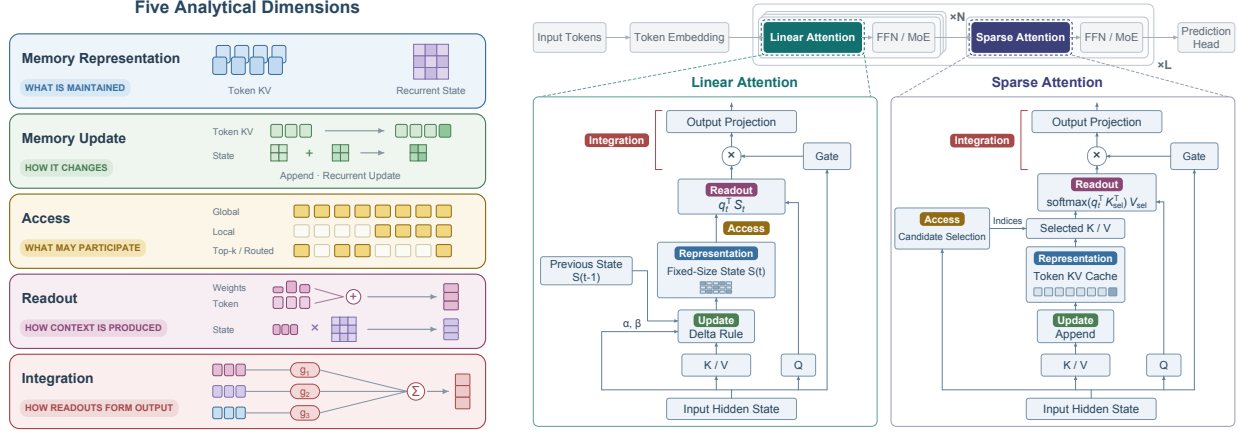


Figure 1: An architectural primer for the five-dimensional memory-centric view. The left panel illustrates Memory Representation, Memory Update, Access, Readout, and Integration through typical explicit-memory and recurrent-state operations. The center and right panels map these dimensions onto representative Linear Attention and Sparse Attention blocks, respectively, while the top row places the two mechanisms in a schematic layer-wise hybrid model. In the Linear Attention example, a delta-style update maintains a fixed-size recurrent state that is read through state contraction; in the Sparse Attention example, candidate selection restricts access to a token KV cache before the core attention readout. Gates and output projections illustrate Integration. The displayed update rule, gates, and block ratio are representative design choices rather than universal properties of these families, and the colored labels denote analytical roles rather than a mandatory execution sequence.

retain only selected or compressed entries, or carry the preceding context in a fixed-size recurrent state. Let x_t denote the current input to the memory mechanism, and let q_t denote the representation used to request information relevant to the current position.

Memory Representation. The first question is what the model has retained before it attempts to retrieve anything. Memory Representation specifies the form and organization of the maintained information, the granularity at which historical content remains distinguishable, and the amount of information the representation can carry. Softmax Attention retains separately addressable memory units. Multi-query attention (MQA) [7], grouped-query attention (GQA) [8], and multi-head latent attention (MLA) [5] reduce redundancy across heads or channels while preserving token-level memory units. Linear Attention instead compresses history into an associative recurrent state [17], whereas SSMs maintain structured recurrent states [25, 26].

These examples expose two properties that recur throughout the survey. *History coverage* describes how much of the preceding sequence may influence the maintained memory, whereas *addressability* describes whether a particular historical unit remains separately selectable. A fixed-size state may cover the entire preceding sequence but no longer preserve every token as an individually addressable item. Conversely, an explicit KV cache preserves token-level addressability but grows as more tokens are retained.

To express these alternatives uniformly, let ρ denote a memory schema that specifies the type, organization, granularity, capacity, and persistence of the representation. The maintained memory $\mathcal{M}_t$ belongs to the state space permitted by that schema:

$$\mathcal{M}_t \in \mathfrak{M}_\rho. \quad (1)$$

For example, ρ may describe a growing list of token KVs, a bounded collection of summary slots, one associative matrix, a structured recurrent state, or several heterogeneous memory paths. This notation describes the information made available by the mechanism rather than requiring one physical realization: the memory may be materialized as a decoding cache, constructed in parallel during training, or carried recurrently as a state. The subscript ρ on each operator below indicates that the concrete Update, Access, Readout, and Integration rules depend on the schema; a heterogeneous schema may therefore apply different rules to different memory units

Memory Update. Once the form of memory has been identified, the next question is how it changes when the model receives x_t . Memory Update covers the operations that write new information, preserve existing information, or remove and revise what was previously stored. In Softmax Attention, the standard update appends a key and value derived from the current input. A bounded or compressed memory may instead merge the new information with an

existing summary. Recurrent mechanisms may use additive writes, multiplicative decay, input-dependent retention, delta correction, or explicit erase–write operations [17, 20, 26].

Let $\mathcal{M}_t^-$ and $\mathcal{M}_t^+$ denote the memory immediately before and after the current update. The update is written abstractly as

$$\mathcal{M}_t^+ = \text{Update}_\rho(\mathcal{M}_t^-, x_t). \quad (2)$$

The same expression therefore covers append-only KV caches, recurrent state transitions, compression into fixed-capacity slots, and controlled erase–write rules. The concrete equations differ across families and are introduced in the corresponding chapters. At this stage, the important distinction is that Update determines how memory changes; it does not determine which parts of the updated memory a particular query will use.

Access. That latter decision belongs to Access. Given a represented memory, Access determines which memory units or state interfaces are eligible to participate in the current read. Let $\widehat{\mathcal{M}}_t$ denote the memory visible to the read path. Depending on the mechanism’s read–write convention, it may be the state before the current update, the state after the update, or an implementation-specific view constructed during the same computation. Access exposes an eligible memory view $\mathcal{C}_t$:

$$\mathcal{C}_t = \text{Access}_\rho(q_t, \widehat{\mathcal{M}}_t). \quad (3)$$

Softmax Attention exposes all causally available memory units. Local or block-sparse patterns expose only positions allowed by a prescribed structure [10, 11], while learned sparse mechanisms use routers, indexes, or Top- k selection to construct a query-dependent candidate set [12]. For a recurrent mechanism, Access instead exposes the current state interface: that state may carry information influenced by the complete causal history, but the individual historical tokens are no longer separately selectable. Accordingly, $\mathcal{C}_t$ may contain all represented token memories, a selected set of tokens or blocks, one or more recurrent states, or another mechanism-specific interface. It may also carry local metadata required by Readout, such as routing scores or group-level normalization quantities.

Readout. Once Access has established the eligible view $\mathcal{C}_t$, Readout determines how the query extracts contextual information from it. Softmax Attention computes normalized query–key similarities over the eligible memory units and aggregates their values [1]. Sparse Attention commonly changes the candidate set while retaining the same Softmax Readout over the selected memory units. Linear Attention reads an associative state through a kernelized contraction or a related state operation [17], whereas an SSM produces a contextual representation through a structured projection of its recurrent state [27]. Access therefore answers *what may be read*, while Readout answers *how the eligible information is read*.

The resulting contextual representation is written as

$$r_t = \text{Readout}_\rho(q_t, \mathcal{C}_t). \quad (4)$$

This separation allows two methods to expose different candidate sets while using the same readout rule, or to expose similar state interfaces while decoding them differently.

Integration. Integration determines how completed readouts are combined into the representation returned by the memory mechanism. With a single readout, this operation may reduce to an identity mapping or a fixed projection. In multi-head Softmax Attention, the contextual representations produced by the individual heads are concatenated and then mixed by the output projection, allowing information recovered by different heads to contribute jointly to the module output. Parallel branches and heterogeneous memory paths follow the same general principle, but may combine their readouts through summation, gating, routing, or other learned fusion rules.

Specific mechanisms illustrate how this stage can be modified. Gated Attention applies an input-dependent sigmoid gate to each head output before the heads are combined, allowing the contribution of retrieved information to vary across tokens [44]. At the branch level, Infini-attention uses a learned gate to combine the readout from local causal Softmax Attention with the readout from its compressive memory [30]. These mechanisms alter how completed readouts contribute to the output without changing which historical memory units were eligible for the preceding read.

When multiple paths participate, r_t may therefore denote a collection such as $\{r_t^{(p)}\}_{p \in \mathcal{P}_t}$ rather than a single vector. The general Integration rule is written as

$$o_t = \text{Integration}_\rho(r_t; x_t), \quad (5)$$

where o_t is the output of the memory mechanism. Concatenation followed by the output projection is the inherited Integration rule of standard multi-head Softmax Attention. A method directly modifies Integration when it changes

how completed readouts contribute to the output, for example through learned gating, conditional routing, or fusion across heterogeneous memory paths.

The five dimensions are functionally distinguishable, but they are not necessarily orthogonal or separately implemented. Changing Representation can alter what remains accessible; an input-dependent state transition can jointly affect Representation and Update; and a hierarchical sparse index can change both the address representation and the candidate set. Equations (1)–(5) should therefore be read as a shared analytical vocabulary rather than a universal computational graph.

Table 2 collects the terminology, abbreviations, and framework-level symbols that recur across the survey. In this survey, *Softmax Attention* denotes the broader explicit-memory lineage in which contextual information is retained as separately addressable representations and read through normalized query–key interactions. Sparse Attention is discussed separately because candidate selection and access-budget allocation form its central design problem. Later chapters also introduce local notation for mechanism-specific equations. Such local symbols retain the definitions given in their own sections and should not be interpreted as additions to the global framework notation below.

Table 2: Core terminology, abbreviations, and framework-level notation used throughout the survey. Chapter-specific symbols are defined locally in the corresponding sections.

Term or symbol	Meaning in this survey
<i>Core terminology and abbreviations</i>	
Contextual memory	Model-internal, input-dependent information retained or made available while processing the current sequence; it excludes pretrained parameters and external retrieval corpora.
KV	Key–value representation associated with a token, compressed entry, or other explicit memory unit.
Softmax Attention	The broader explicit-memory lineage in which contextual information is retained as separately addressable representations and read through normalized query–key interactions.
Sparse Attention	A research line centered on restricting the eligible token-, block-, or entry-level candidate set; most methods retain a Softmax Readout over that set.
Linear Attention	A family that represents history through one or more recurrent associative states rather than retaining the complete token-wise KV history.
SSM	State Space Model; a sequence model that propagates contextual information through a structured recurrent state and state transition.
Hybrid Architecture	An architecture that composes heterogeneous memory mechanisms or access regimes across layers, heads, branches, or tokens.
<i>Framework-level notation</i>	
$t; x_t; q_t$	Current sequence position; current input to the memory mechanism; and the representation used to query contextual memory.
$\rho; \mathfrak{M}_\rho$	Memory schema and the state space admitted by that schema. The schema records properties such as memory type, organization, granularity, capacity, and persistence.
$\mathcal{M}_t$	Contextual memory represented at position t .
$\mathcal{M}_t^-; \mathcal{M}_t^+$	Memory immediately before and after the current update.
$\widehat{\mathcal{M}}_t$	Memory view visible to the read path under the mechanism’s read–write convention.
$\mathcal{C}_t$	Eligible memory view produced by Access; it may contain memory units, state interfaces, and local metadata required by Readout.
r_t	Contextual representation produced by Readout before any explicitly modeled Integration step.
o_t	Output of the memory mechanism after Integration. If a chapter omits a separate Integration operation, its local operator output may coincide with this quantity.
$\mathcal{P}_t; r_t^{(p)}$	Set of participating memory paths and the completed readout from path p .

A small notational qualification is useful when applying this table to later chapters. Mechanism-specific papers often use o for the immediate output of an attention head or recurrent operator. Under the present framework, such a quantity functions as a readout r_t when a subsequent head-, branch-, or path-level Integration step is modeled explicitly; it can coincide with the module output o_t when Integration is the identity or is left implicit.

2.2 Classification Principles and Family-Level Mapping

With the five dimensions and their notation in place, we now use the framework to compare the research lines surveyed in the following chapters. These lines reflect their principal technical questions and historical development rather than being derived mechanically from the five dimensions. The chapter labels therefore indicate the primary context in

which a method is discussed, whereas the dimensions provide a multi-label description of the memory functions it modifies.

Table 3 summarizes the dimensions most frequently emphasized in each research line. **P** marks a dimension commonly treated as a direct design target, whereas **S** marks one usually inherited or modified only in particular subfamilies. These labels are interpretive rather than exhaustive, and all five dimensions remain applicable to every research line.

Table 3: Qualitative emphasis of the surveyed research lines across the five analytical dimensions. **P** denotes a dimension commonly treated as a direct design target in the selected works, while **S** denotes a dimension inherited from the underlying mechanism or modified in particular subfamilies and extensions. The labels are intended as an interpretive guide rather than an exhaustive quantitative coding of the literature.

Research line	Memory substrate	Rep.	Upd.	Acc.	Read.	Int.
Softmax Attention	Explicit memory units	P	P	S	S	S
Sparse Attention	Explicit token/block KV	S	S	P	S	S
Linear Attention	Associative recurrent state	P	P	S	S	S
State Space Models	Structured recurrent state	P	P	S	S	S
Hybrid Architectures	Heterogeneous memory substrates	P	S	S	S	S

Across the four mechanism-centered research lines, design effort is concentrated at different stages of memory processing. Softmax Attention primarily develops Memory Representation and Memory Update by reducing redundancy in explicit memory and changing how bounded or compressed memories are maintained [7, 8, 45, 5]. Sparse Attention instead concentrates on Access: most methods preserve explicit token- or block-level content memory and apply a Softmax Readout over a restricted candidate set [10, 12]. Routing keys, hierarchical indexes, compressed addresses, and cross-layer index reuse make Representation and Update secondary innovation axes, but these structures primarily support candidate construction rather than replace the underlying content memory [14, 15]. Linear Attention and State Space Models concentrate on Representation and Update because their principal developments concern the organization and evolution of recurrent states [17, 20, 26, 27]. Access, Readout, and Integration become more explicit mainly in multi-state, routed, gated, or otherwise specialized variants.

Hybrid Architecture requires a different interpretation because it composes multiple memory mechanisms or access regimes rather than defining a single internal memory operator. Memory Representation is therefore its primary family-level axis, while whether a particular design also modifies Memory Update, Access, Readout, or Integration depends on its composition granularity, as analyzed in Section 7.

The resulting pattern highlights complementary design priorities. Softmax Attention reduces redundancy and reorganizes explicit memory while retaining a Softmax-compatible Readout; Sparse Attention allocates a limited Access budget over that memory; and Linear Attention and State Space Models develop the representation and update of recurrent states. Hybrid Architectures instead primarily diversify and allocate heterogeneous memory representations across layers, heads, branches, or tokens. Their effects on the remaining dimensions depend on the composition granularity and are examined in Section 7. The research lines intersect when an individual method modifies several dimensions, shifting the design problem from optimizing an isolated operator toward coordinating Memory Representation, Memory Update, Access, Readout, and Integration under practical compute and storage constraints.

3 Softmax Attention

Softmax Attention is the explicit-memory lineage in which a query compares itself with an enumerable set of memory units, normalizes the resulting scores, and aggregates the corresponding values. For query position t and attention head h , let $q_{t,h}$, $k_{i,h}$, and $v_{i,h}$ be the query, key, and value vectors obtained by linearly projecting the layer input. The standard formulation is then

$$a_{t,i,h} = \frac{\exp(q_{t,h} k_{i,h}^\top / \sqrt{d_h})}{\sum_{j \in \mathcal{R}_t} \exp(q_{t,h} k_{j,h}^\top / \sqrt{d_h})}, \quad o_{t,h} = \sum_{i \in \mathcal{R}_t} a_{t,i,h} v_{i,h}, \quad (6)$$

where $\mathcal{R}_t$ is the set of memory positions eligible for query t . In the baseline Transformer, **Memory Representation** consists of token-wise key-value pairs, **Memory Update** appends a new pair at each decoding step, **Access** exposes all positions allowed by the causal mask, **Readout** is the normalized query-key weighting in Eq. (6), and **Integration** concatenates the head outputs and applies the output projection [1]. This five-dimensional description is more precise

than treating every efficiency modification as a change to “attention” in general: two methods can use the same Softmax Readout while changing entirely different memory functions.

Because each retained token adds an independently addressable key–value entry, the cache size and per-step readout cost both grow with context length. Consequently, efforts to reduce this cost while preserving the benefits of explicit memory can be organized around three optimization directions. First, *memory-representation efficiency* preserves token-level addressability but reduces the amount of information stored per token or the number of redundant copies across heads and layers. Second, *sequence-representation compression* changes both Representation and Update by consolidating a growing history into bounded states or lower-resolution remote memories. Third, *readout and integration modulation* leaves the underlying memory largely intact while changing either how eligible values are weighted or how completed head outputs are transformed or combined into the attention-module output. These directions are composable: a model may use grouped-query or latent KV storage, compress remote history, and gate attention outputs at the same time.

The boundary with neighboring families follows the primary intervention. Methods that retain fine-grained KV memory but restrict the query-specific candidate set are treated as Sparse Attention, whereas methods that replace enumerable content-bearing units with a recurrent associative statistic are treated as Linear Attention. This section covers mechanisms that maintain an enumerable set of token, latent, slot, or summary units and use Softmax normalization either directly or within the readout; token-level addressability is typical but not required.

3.1 Memory-Representation Efficiency

A baseline autoregressive Transformer caches a key and value for each position, KV head, and KV-producing layer. For batch size B , cached length T , L_{KV} distinct KV layers, H_{KV} stored heads per layer, head dimension d_h , and s bytes per scalar, the cache size is

$$M_{KV} = 2BTL_{KV}H_{KV}d_h s, \quad (7)$$

where the factor of two accounts for keys and values. These factors motivate three representation-efficiency directions: sharing across heads reduces H_{KV} by allowing multiple query heads to reuse KV states; compression along channels replaces full-width per-token KV states with a narrower latent payload; and sharing across layers reduces L_{KV} by allowing multiple consumer layers to reuse source-layer KV states. All three preserve token-level addressability and reduce only the coefficient of cache growth, not its linear dependence on T .

3.1.1 Sharing across Heads

Head sharing asks whether every query head requires an independently stored key and value for the same token. Let H_q be the number of query heads and let $g : \{1, \dots, H_q\} \rightarrow \{1, \dots, H_{KV}\}$ assign each query head to a stored KV head. Query head h then evaluates Eq. (6) with $(k_{i,h}, v_{i,h})$ replaced by $(k_{i,g(h)}, v_{i,g(h)})$. The access set $\mathcal{R}_t$ and the Softmax Readout remain unchanged; only the multiplicity of token representations differs.

Multi-Head Attention. Standard multi-head attention (MHA) uses $H_{KV} = H_q$ and assigns every query head its own key and value projections. The arrangement offers maximal head-specific representational freedom but requires the cache to store and the decoder to load a distinct KV pair for every head and token [1].

Multi-Query Attention. Multi-query attention (MQA) sets $H_{KV} = 1$: all query heads retain distinct query projections but read from one shared key head and one shared value head. It therefore removes most head-wise duplication in the cache and reduces memory bandwidth during incremental decoding, at the cost of sharing the same key and value representations across query heads [7].

Grouped-Query Attention. Grouped-query attention (GQA) partitions query heads into groups, with one KV head shared inside each group. Varying the number of groups creates a continuum between MHA and MQA, allowing model designers to trade head-specific memory representations for cache and bandwidth efficiency. GQA can also be obtained by adapting an MHA checkpoint, which makes it a practical architectural compromise rather than only a design for training from scratch [8].

The resulting continuum reduces the number of KV representations stored per token without changing the token candidate set, as summarized in Figure 2.

3.1.2 Compression along Channels

Channel compression reduces the width of each token memory rather than the number of its copies. Multi-head latent attention (MLA) maps each hidden state to a compact KV latent and derives the head-specific content keys and values

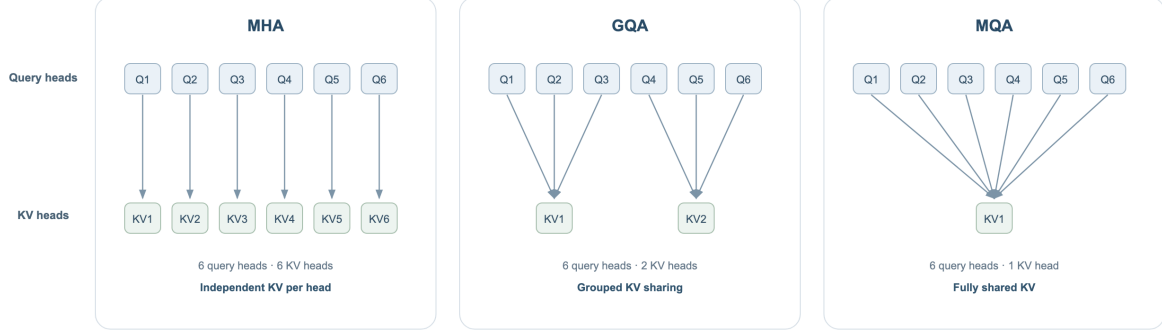


Figure 2: Head-sharing continuum from multi-head attention (MHA) to grouped-query attention (GQA) and multi-query attention (MQA). Each box represents one head; the number of query heads is fixed while the number of independently stored KV heads decreases.

from that shared latent. During decoding, the latent can be cached instead of the expanded multi-head tensors, and suitable projection matrices can be algebraically absorbed into the query and output paths so that the full keys and values need not be materialized at every step [5].

Rotary positional encoding complicates this factorization because applying RoPE to reconstructed content keys prevents the key up-projection from being absorbed into the query path. MLA therefore separates content and positional components and caches the compact KV latent together with a shared positional component, preserving token-level addressability through a low-rank bottleneck [5].

TransMLA studies the complementary deployment problem of converting an existing GQA checkpoint into an MLA-compatible parameterization. It rewrites the shared GQA projections into an equivalent multi-head form, applies low-rank factorization to obtain a common down-projection and head-specific up-projections, and uses continued training to adapt the model to the new latent representation [46]. This distinction is important for evaluation: latent KV is an architectural representation change, not merely a post-hoc compression of already generated cache tensors. Its realized benefit depends on latent width, approximation error, positional treatment, projection fusion, cache layout, and kernel support.

3.1.3 Sharing across Layers

Layer sharing targets redundancy along network depth. A conventional Transformer produces and stores new keys and values in every attention layer, so the cache grows approximately linearly with the number of such layers even after MQA or GQA has reduced the number of stored KV heads. Cross-Layer Attention (CLA) computes KV activations only in selected source layers and allows one or more subsequent consumer layers to reuse them [9]. Consumer layers retain their own queries, Softmax weights, output projections, residual paths, and feed-forward transformations; what is shared is the memory content, not the entire attention computation.

If one source layer serves c adjacent layers, the number of independently cached KV layers decreases by approximately a factor of c . Head and layer sharing are orthogonal: an MLA, GQA, or MQA source layer can itself be shared by multiple consumers. This also distinguishes cross-layer KV sharing from cross-layer index reuse in sparse attention: the former shares content-bearing KV activations, whereas the latter shares an Access decision while each layer retains different KV content.

3.2 Sequence-Representation Compression

The preceding methods reduce the cost of each token memory while retaining all token identities. Sequence-representation compression instead reorganizes history over time in two ways. Fixed-capacity recurrent memory repeatedly writes accumulated content into a bounded set of persistent states, while dynamic-resolution memory retains recent tokens at fine granularity and represents remote context with fewer, coarser units. The first bounds persistent state but may still use temporary local representations; the second may continue to grow with history, but more slowly than a token-wise cache.

Segment recurrence and compressed segment memory provide important precursors. Transformer-XL [47] reuses hidden states from previous segments as an extended context, whereas Compressive Transformer [48] adds a lower-

resolution memory for states displaced from the recent cache. Later methods make the update process more explicit, place stricter bounds on the recurrent state, or design learned summary units for very long contexts.

3.2.1 Fixed-Capacity Recurrent Memory

Fixed-capacity methods maintain K persistent memory units and repeatedly update their contents as new tokens or blocks arrive. Historical tokens therefore cease to exist as independently recoverable entries once their information has been consolidated. The central problem shifts from storing every token to controlling what a bounded memory retains, overwrites, and exposes to future queries.

Recurrent latent-state memory. Recurrent Memory Transformer (RMT) inserts special memory tokens into each segment. Read-memory tokens carry the state produced by the preceding segment, participate in attention with the current tokens, and are transformed into write-memory tokens that become the state for the next segment [45]. The update is therefore performed within the Transformer computation rather than by an external pooling step. TransformerFAM similarly uses a block-wise feedback interface: token queries read the previous Feedback Attention Memory, while memory queries selectively aggregate the current block and return updated latent states to the next block. The mechanism reuses existing attention and feed-forward parameters instead of introducing a separate memory network [49].

For both models, the persistent memory is a fixed set of hidden-state-like vectors updated at segment or block boundaries. Queries attend to these latent units rather than to the original tokens from earlier segments, so recurrence can propagate information across segment boundaries but does not preserve exact historical addressability: multiple past tokens may be entangled in the same state.

Recurrent KV-slot memory. Trellis and Lattice instead maintain explicit key and value slots. The keys define retrieval conditions and the values carry associated content, so current queries can attend directly to a bounded set of memory entries. Trellis uses a two-pass recurrent compression procedure and online-gradient-descent-inspired updates, together with forgetting that regulates the persistence of previous content [42]. Lattice formulates cache compression as online optimization, exploits low-rank structure in the KV matrices, and applies state- and input-dependent gates together with an orthogonal update intended to reduce redundant writes; chunkwise parallelization mitigates the sequential cost of recurrence [41].

The distinction between latent-state and KV-slot memory concerns the persistent interface rather than the update frequency. RMT and TransformerFAM are primarily block- or segment-recurrent, whereas Trellis and Lattice update at finer granularity in their proposed realizations. In both cases, bounded recurrent updates replace append-only token accumulation, and original token identities cease to be separately accessible once consolidated into latent states or slots.

3.2.2 Dynamic-Resolution Memory

Dynamic-resolution memory assigns different granularities to different portions of history. Recent tokens remain explicit to preserve exact local dependencies, whereas older content is consolidated into fewer summaries or compressed global entries. Unlike fixed-capacity recurrence, the remote memory may continue to grow, but at a slower rate determined by chunk size, summary ratio, hierarchy depth, or consolidation policy.

Kwai Summary Attention (KSA) combines a fine-grained recent path with summary memory for earlier chunks. Sliding Chunk Attention preserves local continuity, while summary attention gives queries access to compressed remote context; the memory state therefore contains both token-level recent units and coarser content-bearing summaries [6]. HCA Core constructs a smaller collection of highly compressed global entries and performs dense Softmax attention over those entries [50]. The complete DeepSeek-V4 architecture additionally combines local sliding-window and compressed sparse pathways, so the full system is hybrid; HCA Core falls under sequence-representation compression because it replaces remote KV entries with compressed units and attends densely over those units rather than selecting a query-specific subset of the original cache.

Dynamic-resolution memory therefore rewrites the units represented before readout, whereas Sparse Attention retains fine-grained memory and restricts the query-specific candidate set.

Both fixed-capacity and dynamic-resolution policies compress information before future queries are known. Bounded updaters trade retention against overwrite and interference, while dynamic-resolution systems must decide when to consolidate content and what their summaries should preserve. Once token states are overwritten or merged, omitted details are generally unrecoverable. Evaluation should therefore report persistent memory size, local-context budget, compression or update overhead, readout cost, and long-range recall; bounded persistent memory does not eliminate local or update computation, and dynamic-resolution state may still grow with history.

3.3 Readout and Integration Modulation

The preceding mechanisms change the memory substrate or its update rule. A separate line of work preserves the underlying memory units but modifies either the Readout in Eq. (6) or the subsequent Integration stage. Readout modulation transforms query–key scores or combines attention maps before they weight the values, whereas Integration modulation routes or gates completed head outputs before the output projection. These interventions can alter selectivity and information flow without reducing the stored memory or token candidate set.

3.3.1 Score and Map Modulation

Talking-Heads Attention applies learned linear projections across heads both before Softmax and after Softmax. It therefore allows different heads to cooperate in forming attention maps, rather than requiring each head to generate and use its weights independently; the same learned mixing matrices are applied to every input [51]. Dynamically Composable Multi-Head Attention (DCMHA) makes the cross-head composition input-dependent and modulates both pre-Softmax scores and post-Softmax weight matrices, increasing adaptive expressivity at the cost of additional composition and implementation overhead [52].

Differential Attention constructs two query–key paths, obtains two Softmax maps, and subtracts one from the other with a learned scale. Components that activate similarly in both paths can be suppressed, while their differences are retained. The effective weights may consequently be negative and no longer form a single probability distribution, although the component maps are Softmax-normalized [53]. Forgetting Transformer (FoX) instead introduces a data-dependent forget gate into the score path, allowing the contribution of historical positions to decay according to both content and distance before Softmax normalization [54].

These methods leave the underlying candidate memory and, in their basic forms, the append-only cache unchanged. They alter the weights used to read that memory, but more concentrated or smaller effective weights do not imply fewer stored KV entries or fewer executed score computations.

3.3.2 Head Routing and Gating

Mixture of Attention Heads (MoH) treats heads as experts and uses a token-dependent router to select and weight their contributions. Shared heads remain active for all tokens, while routed heads provide conditional specialization [55]. Because the controlled object is the contribution of head-level readouts to the module output, the principal change is **Integration**. A sufficiently specialized implementation may skip inactive heads and obtain conditional computation, but this head-level sparsity is distinct from selecting historical tokens in the Access stage.

Gated Attention introduces input-dependent sigmoid gates after the scaled dot-product attention outputs of individual heads and before their final combination [44]. The gates add a nonlinearity between value aggregation and output projection and allow the model to suppress a head’s retrieved information for a particular token. A near-zero post-readout gate, however, does not retroactively eliminate the score computation, Softmax normalization, or KV loads already performed for that head.

MoH and Gated Attention thus both modify Integration, but with different control semantics. MoH allocates contribution through routing and can support discrete conditional execution; Gated Attention continuously rescales head outputs and primarily regulates information flow. Whether either mechanism yields wall-clock savings depends on kernel and runtime support for the induced head-level sparsity.

Readout and Integration modifications should therefore be evaluated separately from memory compression. Numerical sparsity in scores, routing probabilities, or gates reduces computation only when the implementation can skip the corresponding score, load, or head operations before execution; evidence should therefore pair model quality with FLOPs, memory traffic, kernel efficiency, and end-to-end latency.

3.4 Summary

Softmax Attention research has pursued several complementary ways to reduce the cost of explicit-memory retrieval while retaining normalized content-based readout. For autoregressive decoding, MQA and GQA reduce head-wise replication, while MLA and cross-layer sharing compress token memory along channel and depth axes while preserving token-level addressability. At longer contexts, lowering the cost of each token is no longer sufficient to control growth with sequence length, motivating bounded recurrent memories and dynamic-resolution summaries that trade exact historical granularity for capacity. Score and map modulation, head routing, and output gating developed as an orthogonal direction for improving selectivity and information flow without directly shrinking the represented memory.

Head sharing is a common design for dense Softmax Attention in several mainstream model families: GQA is used across Llama 3/4 [56, 57] and Qwen2/3 [58, 59], and remains as the periodic gated full-attention path² in Qwen3.5 [60], Qwen3.6 [61, 62], and Qwen3.8-2.4T-A95B. Latent KV compression forms a second major lineage. MLA, introduced in DeepSeek-V2 and retained in DeepSeek-V3/V3.2 [5, 63, 64], is also used by Kimi K2 [65], by the Gated MLA layers of Kimi K3 [66], and as the content representation underlying DeepSeek Sparse Attention in the GLM-5 series [67, 68, 69]. More aggressive sequence compression appears in DeepSeek-V4’s Heavily Compressed Attention [50], which applies dense Softmax readout to consolidated KV entries, while output gating accompanies both GQA in recent Qwen hybrids [60, 44] and MLA in Kimi K3 [66].

Across these systems, the GQA, latent-KV, compressed-memory, and gating components belong to the developments surveyed here; where recurrent or sparse paths are also present, those paths are treated in later sections. The next section turns to Sparse Attention, where the central question is which represented token- or block-level units are made eligible for each query.

4 Sparse Attention

Dense causal Softmax Attention follows Eq. (6) and normalizes each query over all causally available positions, so $|\mathcal{R}_t| = \Theta(t)$. Consequently, training and prefill require $\Theta(L^2d)$ attention work, while each decoding step at context length L requires $\Theta(Ld)$ work and reads a KV cache whose size grows as $\Theta(Ld_{\text{KV}})$ per layer. Memory-efficient kernels can reduce intermediate storage but not the number of query–key interactions. Sparse Attention instead restricts Access to a budgeted subset $|\mathcal{I}_t| = b_t \ll t$, targeting computation and data movement while preserving the evidence needed by Readout [70, 10, 11, 33].

Let $\mathcal{U}_t^{(\ell)}$ denote the addressable memory units available before the read at position t . A unit may be a token, page, block, chunk, or compressed entry. Sparse Access first constructs a selection-evidence vector $e_{t,:}^{(\ell)}$ (one score per unit) and then chooses an index set under budget b_t :

$$\mathcal{I}_t^{(\ell)} = \text{Select}\left(e_{t,:}^{(\ell)}, \mathcal{U}_t^{(\ell)}; b_t\right). \quad (8)$$

The selected indices are then used to gather the corresponding entries from content memory. In the common case, the selected content remains ordinary token KVs and the readout is a candidate-local Softmax,

$$r_t^{(\ell)} = \text{softmax}\left(\frac{q_t^{(\ell)} (K_{\mathcal{I}_t}^{(\ell)})^\top}{\sqrt{d}}\right) V_{\mathcal{I}_t}^{(\ell)}. \quad (9)$$

These operators are written for a single layer; we suppress the layer index (ℓ) in the family-specific equations that follow and restore it only where cross-layer relationships are discussed.

Sparse Attention is primarily an **Access** intervention: methods differ in whether support is prescribed structurally, inferred from backbone representations, predicted by an auxiliary index, or reused across steps and layers. **Memory Representation** and **Memory Update** become relevant when selection requires address memory—such as pooled summaries, projected keys, or binary codes—or when content itself is compressed, refreshed, or shared. Most methods retain the candidate-local Softmax Readout in Eq. (9); Readout changes directly only when routing scores also allocate probability mass, while Integration matters mainly in multi-branch designs that combine sparse, local, or compressed paths. The chapter therefore centers on support construction and its execution cost, noting the other dimensions only when a method changes them directly. Figure 3 contrasts dense causal access with representative structure-constrained and routing-based sparse supports.

Sparse Attention methods are organized here by two questions: how candidate-support evidence is produced and how long a selection decision remains valid. Along the evidence-source axis, *Structure-Constrained* methods prescribe a topology or pattern family, *Self-Routing* methods derive evidence from backbone representations or execution state, and *Auxiliary-Proxy Routing* introduces a separate indexing space. Orthogonally, *Temporal and Cross-Layer Reuse* amortizes supports, indexes, or memory representations across time and depth and can be combined with any evidence source. These subsection labels therefore serve as editorial homes rather than mutually exclusive classes; routing granularity and training mode remain cross-cutting attributes.

²Here, *full attention* denotes full-context causal Softmax Attention, in which each query may attend to all causally eligible token positions represented at that layer. The term describes Access scope rather than head or KV parameterization; a full-attention path may use MHA, MQA, GQA, or MLA, and contrasts with local, streaming, or sparsely selected attention.

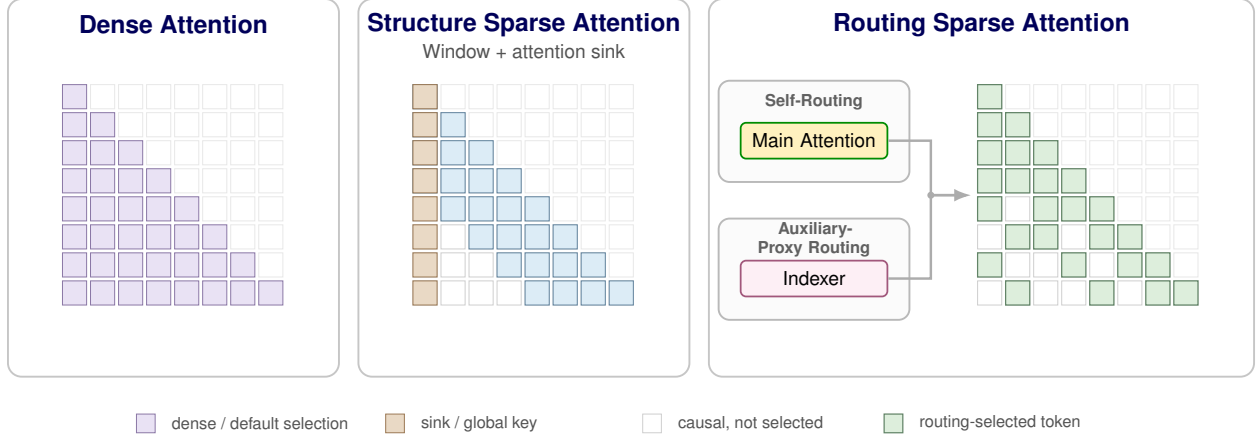


Figure 3: Attention-map comparison between dense and sparse access. Dense Attention retains every causal pair. Structure Sparse Attention combines a fixed four-token sliding window with a sink connection to the first token. Routing Sparse Attention retains the causal lower-triangular domain, selects the diagonal by default, and chooses up to three additional tokens per row. Self-routing from the main attention and auxiliary-proxy routing through an indexer are alternative ways to produce the same type of selected support; all selected routing cells therefore share one visual style, while unselected causal cells remain unfilled.

4.1 Structure-Constrained Sparse Attention

Structure-constrained methods restrict Access using positional relations, distance schedules, or a predefined topology. Their distinguishing property is not a particular mask geometry, but that the admissible support is limited by a pattern established independently of unrestricted query-specific search. Let $\mathcal{P}$ denote either a pattern designed before training or a pattern family discovered from a trained dense model. The instantiated support can be written as

$$\mathcal{P} \in \{\mathcal{P}_{\text{design}}, \mathcal{P}_{\text{observed}}\}, \quad \mathcal{I}_t = \text{Instantiate}(\mathcal{P}; t, z_t), \quad |\mathcal{I}_t| \ll t, \quad (10)$$

where z_t is optional input-dependent information used only to instantiate positions within the permitted family. This distinction separates *architecture-prescribed* patterns, fixed before training so that parameters adapt to the restricted support, from *post-hoc discovered* patterns, inferred from an already-trained dense checkpoint and applied at inference. Figure 4 visualizes these two origins through representative causal attention maps.

4.1.1 Architecture-Prescribed Patterns

Architecture-prescribed patterns determine the attention topology before pretraining or task training, allowing the model parameters to adapt to the restricted support. A common construction combines a local backbone with a small number of long-range edges. Longformer uses a sliding window together with task-dependent global tokens that can read the full sequence and be read by all positions [10]. BigBird combines local, global, and random connections to provide local continuity, global aggregation, and additional long-range graph connectivity [11]. These local–global composites preserve a predictable memory-access pattern, although special nodes and heterogeneous edge types complicate both model design and implementation.

A second line organizes long-range access directly by distance. Sparse Transformer factorizes dense attention into complementary local and strided or fixed patterns, reducing the interaction count to approximately $O(L\sqrt{L})$ for suitable factorizations [70]. LongNet uses progressively increasing segment sizes and dilation rates to form multi-scale Dilated Attention, thereby extending the receptive field while keeping the attention work linear in sequence length under its sparse schedule [71]. PowerAttention similarly uses power-of-two offsets to allocate a fixed budget across multiple distance scales [72]. Distance-structured patterns are regular and budgetable, but their ability to retrieve a remote item depends on the prescribed offsets and on information propagation through depth.

The benefit of architecture-prescribed sparsity is training–inference consistency: the model learns under the same constrained Access used at deployment. The cost is limited input adaptivity. A fixed topology can retain irrelevant regions and omit evidence whose location or density does not match the structural prior. Enlarging windows or adding global edges improves coverage but directly consumes the compute and bandwidth savings that motivate sparsity.

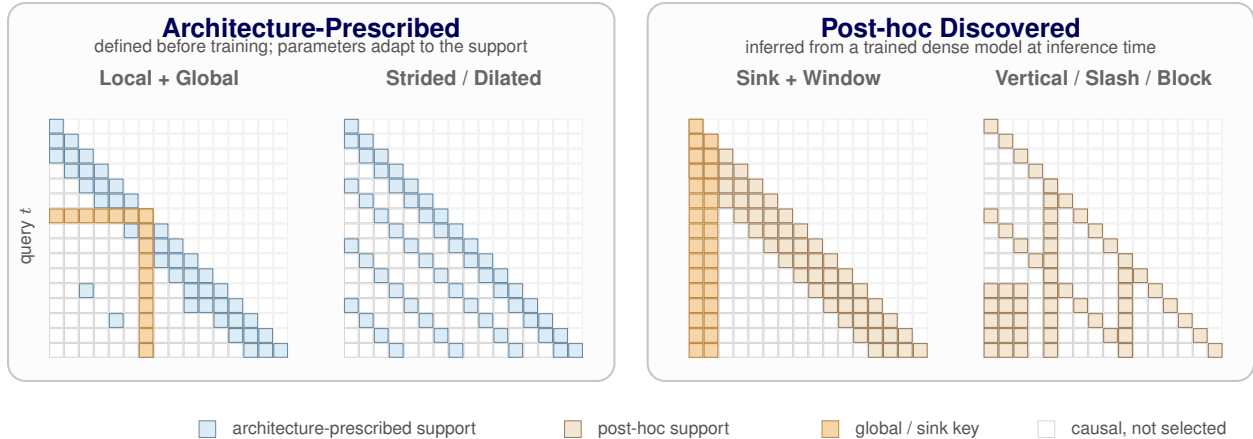


Figure 4: Two origins of structure-constrained support, shown as causal attention maps (rows are queries, columns are keys). Architecture-prescribed patterns—local windows with global or random edges, and strided/dilated schedules—are fixed before training so parameters adapt to them. Post-hoc patterns—sink-plus-window rules and offline-profiled vertical/slash/block families—are inferred from a trained dense model and applied at inference. The global token is drawn in causal-ized form: it attends all earlier positions (its causal row) and is attended by all later ones (its column), never accessing future keys; a sink instead contributes only a column.

4.1.2 Post-hoc Discovered Patterns

Post-hoc methods analyze a trained dense model and convert recurring attention behavior or extrapolation failures into an inference-time rule. StreamingLLM identifies the importance of initial “attention sink” tokens and retains a small sink set together with a rolling recent window [73]. LM-Infinite similarly uses an initial-plus-recent Λ -shaped mask and caps relative-position distances to improve zero-shot length extrapolation [74]. Their runtime Access is globally fixed rather than content-selected: the rule preserves empirically important positions but does not retrieve arbitrary middle-context tokens.

MInference occupies an intermediate point between fixed and fully content-adaptive routing. It profiles dense attention maps to assign each head an A-shape, vertical-slash, or block-sparse pattern family, and then instantiates columns, diagonals, or blocks from the current Q/K values at inference time [75]. The family is fixed by offline analysis, whereas the selected positions retain limited input dependence. This reduces the online search space and supports structured kernels, but the dense checkpoint was not optimized under the resulting mask.

Post-hoc discovery is attractive because it can accelerate an existing model without sparse pretraining. Its central limitation is training–inference mismatch: deleting connections and changing the Softmax normalization domain can alter a computation that was learned as dense. Fixed rules minimize selection overhead but offer little input adaptivity; pattern families recover some adaptivity at the price of runtime statistics and candidate construction. Comparisons should therefore report not only nominal sparsity, but also retained attention mass or support recall, end-to-end latency, and quality across tasks whose relevant evidence has different positional structure.

4.2 Self-Routing Sparse Attention

Self-routing constructs support from the main model’s own representations or execution state by scoring candidates in the same query–key representation space used by the main Softmax Readout, either directly or through deterministic pooling of backbone keys. Candidate evidence may additionally use sampled queries, pooled statistics, online-Softmax state, or a draft model’s attention before being passed to the budgeted selector in Eq. (8). The methods below differ in how they construct this evidence and in how tightly selection is coupled to attention execution. Figure 5 summarizes three representative evidence constructions developed below. Auxiliary-proxy routing, discussed in Section 4.3, instead forms selection scores from separately parameterized index queries and keys that the main attention does not read.

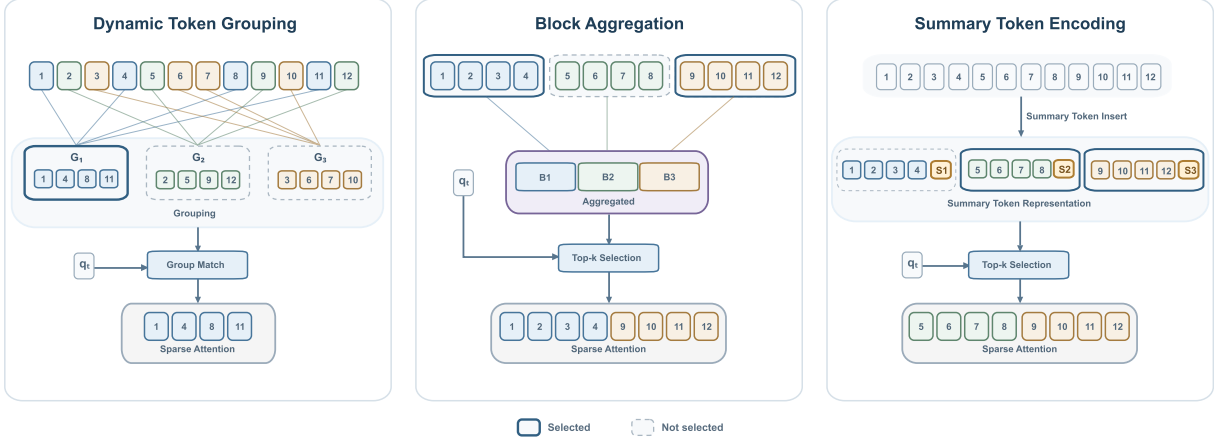


Figure 5: Representative evidence constructions for self-routing Sparse Attention: dynamic token grouping, block aggregation, and in-sequence summary-token encoding.

4.2.1 Dynamic Token Grouping

Dynamic token grouping constructs a content-dependent sparse pattern without first evaluating the full query–key score matrix. A lightweight partitioner maps every query and key independently to one or more groups according to representation similarity. Tokens with matching group assignments are then gathered, causal masking is applied within each group, and ordinary token-level Softmax is evaluated only on those pairs. The groups may contain positions that are far apart in the original sequence, so this mechanism can connect semantically related tokens without prescribing where those tokens must occur.

Reformer realizes this idea with locality-sensitive hashing over shared query–key representations: tokens that collide under a hash attend within the same or adjacent buckets, and multiple hash rounds improve recall [76]. Routing Transformer is a particularly representative learned variant because it exposes the complete grouping pipeline—learned partition, content-dependent assignment, and within-group readout—while retaining standard dot-product attention after routing [77].

Routing Transformer maintains G shared centroids $\{\mu_c\}_{c=1}^G$ and normalizes queries, keys, and centroids onto a common sphere. Conceptually, each query and key is assigned to its most similar centroid:

$$a_t^Q = \arg \max_{c \in \{1, \dots, G\}} \tilde{q}_t \mu_c^\top, \quad a_j^K = \arg \max_{c \in \{1, \dots, G\}} \tilde{k}_j \mu_c^\top, \quad (11)$$

where $\tilde{q}_t$ and $\tilde{k}_j$ are normalized routing vectors. The centroids are updated online by mini-batch spherical k -means, allowing the partition to follow the representation geometry learned by the model. For causal self-attention, the routing support of query t consists of earlier keys assigned to the same centroid:

$$\mathcal{I}_t^{\text{route}} = \left\{ j < t \mid a_j^K = a_t^Q \right\}. \quad (12)$$

The query then performs exact Softmax attention within this support:

$$r_t^{\text{route}} = \text{softmax} \left(\frac{\tilde{q}_t (\tilde{K}_{\mathcal{I}_t^{\text{route}}})^\top}{\sqrt{d}} \right) V_{\mathcal{I}_t^{\text{route}}}. \quad (13)$$

In practice, Routing Transformer assigns each centroid roughly L/G queries and keys with the highest centroid similarity rather than allowing unconstrained cluster sizes; this balances parallel work but can give a token more than one membership. Its models also allocate separate heads to local attention so nearby evidence is preserved when clustering misses it. The resulting cost is $O(LGd + L^2d/G)$ under balanced groups, minimized near $G = \sqrt{L}$ to $O(L^{3/2}d)$. This content adaptivity comes with assignment, sorting, group-size control, and load-balancing overhead, while non-contiguous members produce irregular gathers that are harder to map to accelerator tiles. These costs motivate later methods to fix the physical candidate unit as a page or contiguous block and focus on constructing an accurate low-cost score for each unit.

4.2.2 Block Aggregation

Once candidates are fixed as contiguous blocks, routing reduces to scoring each block and expanding the selected blocks to their constituent KVs. A key distinction is whether those scores are derived from an unchanged dense checkpoint or from representations optimized under sparse support. This is not a strict chronological progression: training-free designs preserve retrofitability, whereas trainable designs trade additional optimization for a smaller dense-to-sparse mismatch.

For an existing dense checkpoint that must be deployed without adaptation, block relevance can be estimated directly from its current query–key geometry. Quest bounds the largest possible query–key product within each page, while XAttention derives block scores from antidiagonal statistics [78, 79]. Their appeal is immediate applicability without modifying model parameters; their limitation is that they approximate the attention behavior of a model never optimized for restricted support, so the dense-to-sparse mismatch can become more consequential as the retained budget shrinks.

When training from scratch or continued adaptation is feasible, the model can instead learn to make block-level evidence reliable under sparse support. MoBA partitions KVs into contiguous blocks and represents each block $\mathcal{B}_u$ by the mean of its backbone keys:

$$\bar{k}_u = \frac{1}{|\mathcal{B}_u|} \sum_{i \in \mathcal{B}_u} k_i. \quad (14)$$

For query q_t , it then computes the relevance of block u by a query-to-mean-key inner product:

$$e_{t,u}^{\text{MoBA}} = q_t \bar{k}_u^\top. \quad (15)$$

MoBA selects the Top- k historical blocks according to $e_{t,u}^{\text{MoBA}}$ and always adds the current block u_t :

$$\mathcal{S}_t^{\text{MoBA}} = \text{TopK}_{u < u_t}(e_{t,u}^{\text{MoBA}}; k) \cup \{u_t\}. \quad (16)$$

Let $\mathcal{I}_t^{\text{MoBA}}$ contain the causally visible token positions in the selected blocks. The final readout applies exact token-level Softmax to their original KVs:

$$o_t^{\text{MoBA}} = \text{softmax}\left(\frac{q_t (K_{\mathcal{I}_t^{\text{MoBA}}})^\top}{\sqrt{d}}\right) V_{\mathcal{I}_t^{\text{MoBA}}}. \quad (17)$$

Because scoring reuses the backbone’s Q/K space, sparse training can adapt those representations to make the coarse block score useful [80]. FlashMoBA preserves this routing rule but makes smaller blocks practical: tiled Top- k avoids materializing the full query–block score matrix, and a gather-and-densify kernel packs irregularly selected queries into on-chip tiles for FlashAttention-style computation [81].

Native Sparse Attention (NSA), introduced by DeepSeek-AI as a hardware-aligned and natively trainable sparse architecture, uses block aggregation within a three-branch design. Let ℓ_c , s_c , and ℓ_s denote the compression-block length, compression stride, and selection-block length, respectively. NSA first maps each completed compression block to a learned compressed key:

$$\tilde{K}_t^{\text{cmp}} = \left\{ \phi_K(k_{i s_c + 1 : i s_c + \ell_c}) \mid 0 \leq i \leq \left\lfloor \frac{t - \ell_c}{s_c} \right\rfloor \right\}. \quad (18)$$

Here ϕ_K is a learned compression MLP with intra-block positional encoding; compressed values $\tilde{V}_t^{\text{cmp}}$ are constructed analogously. The compressed branch computes its query–key attention probabilities as

$$p_t^{\text{cmp}} = \text{softmax}\left(\frac{q_t (\tilde{K}_t^{\text{cmp}})^\top}{\sqrt{d}}\right). \quad (19)$$

NSA reuses these probabilities as routing evidence. When compression and selection blocks have different boundaries, the score of selection block u aggregates the probabilities of the overlapping compression blocks:

$$p_t^{\text{sfc}}[u] = \sum_{m=0}^{\ell_s/s_c - 1} \sum_{r=0}^{\ell_c/s_c - 1} p_t^{\text{cmp}} \left[\frac{\ell_s}{s_c} u - m - r \right], \quad (20)$$

where out-of-range terms are taken as zero. For grouped-query attention, the scores of the H_g query heads sharing one KV head are summed before selection:

$$\hat{p}_t^{\text{sfc}}[u] = \sum_{h=1}^{H_g} p_{t,h}^{\text{sfc}}[u]. \quad (21)$$

The fine-grained branch then retains the Top- n selection blocks according to the aggregated score:

$$\mathcal{S}_t^{\text{NSA}} = \text{TopK}(\hat{p}_t^{\text{slc}}; n). \quad (22)$$

The original-token KVs in $\mathcal{S}_t^{\text{NSA}}$ form the selected branch, while a sliding-window branch preserves recent local context. Input-dependent sigmoid gates weight the compressed, selected, and sliding-window branch outputs, whose weighted sum forms the final NSA readout. Thus, unlike MoBA’s single selected-block path, NSA uses the compressed summaries both as readable memory and as evidence for retrieving full-resolution content [12].

Other trainable variants mainly change how block evidence is summarized. InFLLM-V2 combines multi-stage mean/-max pooling with GQA-group aggregation and allows dense-sparse switching; DashAttention learns hierarchical pooling with α -entmax to allocate a variable support; and COBS compresses second-order statistics to estimate block attention mass [82, 83, 84]. Across these designs, adaptation can reduce dense-to-sparse mismatch, but summary maintenance, block-boundary errors, overfetch, and additional Readout paths remain part of the end-to-end cost.

4.2.3 Summary-Token Encoding

Summary-token methods insert learned tokens into the sequence and use their backbone representations as address memory. Landmark Attention places a landmark after each context block and uses a grouped Softmax in which a token’s effective probability depends on both its own key and its block landmark; the selected landmarks determine which original blocks are loaded [85]. Simplified Sparse Attention inserts gist tokens, trains them under a restricted mask to encode their chunks, and then attends over raw KVs from the selected chunks together with the selected gist representations. Its hierarchical extension adds meta-gists to reduce the cost of scanning all chunk summaries [86].

HiLS targets the attention mass of each distant chunk rather than only its mean or maximum token score [14]. For query q_t and chunk $\mathcal{B}_u$, let $s_{t,j} = q_t k_j^\top / \sqrt{d}$; the exact unnormalized chunk mass is

$$Z_{t,u} = \sum_{j \in \mathcal{B}_u} \exp(s_{t,j}). \quad (23)$$

Computing $Z_{t,u}$ for every chunk would require all token-level query-key products. HiLS therefore appends a landmark token to each chunk and uses its learned query q'_u to construct a compact summary. The landmark query first induces an intra-chunk distribution, whose weighted key average becomes the summary key:

$$\alpha_{u,j} = \frac{\exp(q'_u k_j^\top / \sqrt{d})}{\sum_{r \in \mathcal{B}_u} \exp(q'_u k_r^\top / \sqrt{d})}, \quad k'_u = \sum_{j \in \mathcal{B}_u} \alpha_{u,j} k_j. \quad (24)$$

The entropy of this aggregation distribution supplies a chunk-dependent bias:

$$b'_u = - \sum_{j \in \mathcal{B}_u} \alpha_{u,j} \log \alpha_{u,j}. \quad (25)$$

For the current query, HiLS combines summary relevance and aggregation entropy into a linear surrogate for the chunk’s LogSumExp score:

$$\hat{s}_{t,u} = \frac{q_t (k'_u)^\top}{\sqrt{d}} + b'_u \approx \log Z_{t,u}, \quad \hat{Z}_{t,u} = \exp(\hat{s}_{t,u}) \approx Z_{t,u}. \quad (26)$$

It then retrieves the K distant chunks with the largest surrogate scores:

$$\mathcal{S}_t^{\text{HiLS}} = \text{TopK}_{u \in \mathcal{C}_t}(\hat{s}_{t,u}; K), \quad (27)$$

where $\mathcal{C}_t$ contains historical chunks outside the local window. HiLS computes exact token-level Softmax within each selected chunk, but uses the surrogate masses to allocate probability across chunks. If $Z_{t,\text{loc}}$ is the exact mass of the local window and $c(j)$ denotes the chunk containing token j , then for a token in a selected distant chunk,

$$w_{t,j}^{\text{HiLS}} = \frac{\exp(s_{t,j})}{Z_{t,c(j)}} \frac{\hat{Z}_{t,c(j)}}{\hat{Z}_t}, \quad \hat{Z}_t = Z_{t,\text{loc}} + \sum_{u \in \mathcal{S}_t^{\text{HiLS}}} \hat{Z}_{t,u}. \quad (28)$$

The local window is treated as one group with its exact mass $Z_{t,\text{loc}}$, and the final output is the weighted sum of values over the local window and selected chunks. Because $\hat{Z}_{t,u}$ participates in the forward attention weights rather than only in the discrete Top- K decision, the language-modeling loss can train the landmark summaries end to end. This example illustrates why Access and Readout must be separated: a summary score may merely choose candidates, or it may also determine how probability mass is distributed across candidate groups. Summary tokens remain semantically aligned with the backbone, but they require a specialized sequence layout and training procedure, consume additional token capacity, and can become an information bottleneck when one vector must represent a long or heterogeneous chunk.

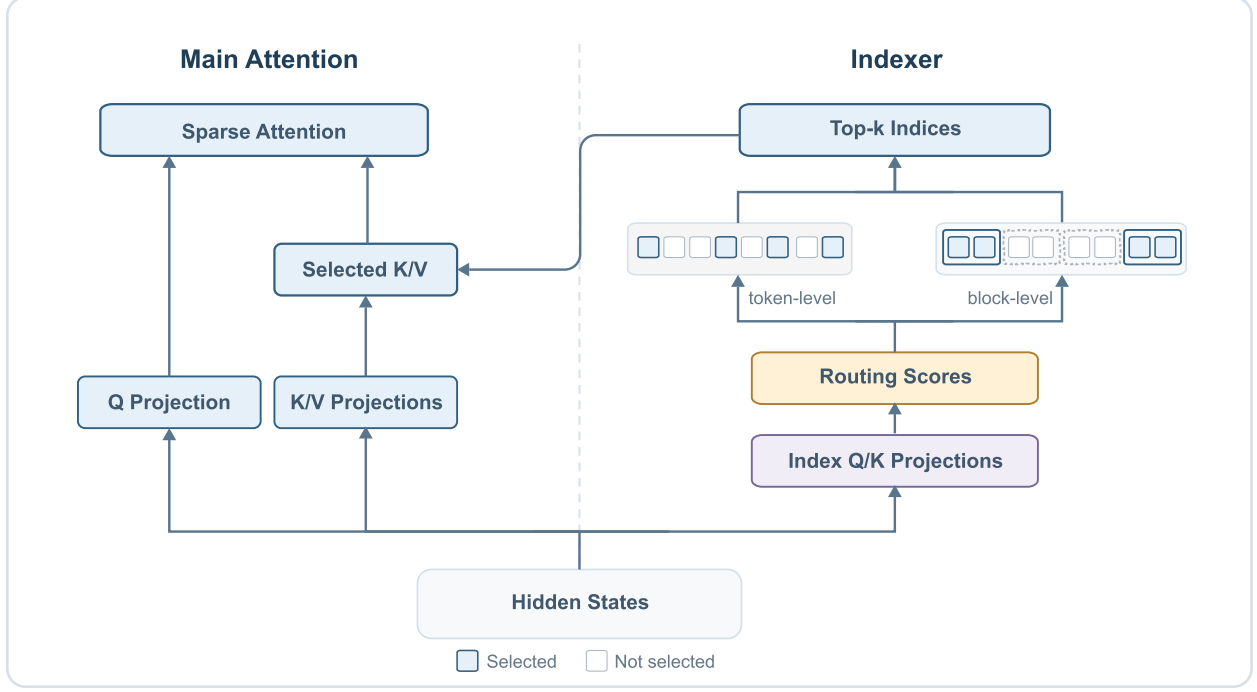


Figure 6: Auxiliary-proxy Sparse Attention within one layer. Shared hidden states feed the Main Attention and Indexer paths; token- or block-level index selections determine which K/V entries reach Sparse Attention.

4.3 Auxiliary-Proxy Sparse Routing

Auxiliary-proxy routing also selects support from the current content, but introduces an address representation whose parameters and training objective are distinct from the main attention projections. A lightweight indexer typically forms the proxy query and key through separate projections of backbone hidden states, for example $q_t^I = h_t W_Q^I$ and $k_u^I = h_u W_K^I$ at token granularity; block-level methods may instead pool these proxy keys or construct one compressed proxy entry per block. The index-specific parameters W_Q^I and W_K^I are learned independently of the main attention projections. The resulting proxy scores $e_{t,u}^I = s_I(q_t^I, k_u^I)$ determine which original K/V entries are selected. Depending on the method, these scores may be discarded after Access or may also influence Readout. Figure 6 makes the internal split explicit: the indexer maps query and candidate states into a compact address space, constructs selection evidence, and applies a budgeted selector; only the returned indices address the original K/V memory. For block routing, evidence can be formed by scoring pooled proxy keys or by aggregating token scores, so block-granular Access does not by itself imply a block-granular evidence scan. Decoupling makes the dimensionality, number of heads, update schedule, supervision, and routing granularity independently tunable. It also creates a second memory whose fidelity to the content-memory relevance relation must be learned and maintained.

4.3.1 Token-Granularity Routing

Token-granularity proxies maintain a separately addressable entry for each historical token. TokenButler projects hidden states and cached keys into a low-dimensional importance space, distills the proxy from dense attention distributions, and amortizes predictions through intervals and neighbor fetching [87].

DeepSeek Sparse Attention (DSA) from DeepSeek-V3.2 introduces a lightweight Lightning Indexer for routing [64]. The indexer has an MQA-style structure in which several index query heads share a single index key per token (left column of Figure 7). For query t and a preceding token u , its proxy score is

$$e_{t,u}^I = \sum_{h=1}^{H_I} w_{t,h}^I \text{ReLU}(q_{t,h}^I (k_u^I)^\top), \quad (29)$$

where H_I is the number of index heads, $q_{t,h}^I$ is the query of index head h , k_u^I is the index key shared by all heads, and the query-dependent scalar weights $w_{t,h}^I$ fuse the per-head scores into one token-level score. The Top- k tokens under

$e_{t,:}^I$ form a single support shared by all attention heads, because the main attention runs MLA in its MQA mode, where each latent KV entry serves every query head; the index scores only determine this support and do not enter Readout. The indexer is supervised by a KL loss toward the main attention scores summed over heads and renormalized over tokens: a short dense warmup trains only the indexer over the full context, after which the loss is restricted to the selected tokens while the whole model adapts to sparse support, and the indexer input is detached so that the indexer and the backbone learn only from the KL and language-modeling losses, respectively.

LongCat Sparse Attention (LSA) extends the DSA Lightning Indexer to relieve two of its deployment bottlenecks, the full-length index scan and the discontinuous KV access induced by token-level selection. Streaming-Aware Indexing reserves part of the support budget for a fixed sink and sliding window while using the remainder for dynamic token selection, improving the contiguity of KV access. Hierarchical Indexing first recalls candidate blocks with approximate scores and then performs fine-grained token selection within those blocks, reducing index computation while retaining token-level final support [16].

SAS learns a continuous context-ranking gate and incorporates the gate score into attention logits, allowing the language-model objective to optimize both ranking under a finite budget and the subsequent Readout [88]. Token-Butler, DSA, and SAS thus respectively illustrate plug-in distillation, native mask-oriented indexing, and end-to-end coupling between proxy evidence and readout weights.

Binary proxies reduce the storage and comparison cost of each address entry. HashAttention learns mappings from queries and key-value pairs to a Hamming space and retrieves pivotal tokens by binary similarity before applying attention to the selected original KVs [89]. HATA trains query and key codes to preserve the relative ordering required for Top- k selection and co-designs binary scoring with a sparse-attention kernel [90]. Binary coding lowers per-entry cost, but it does not change the $O(L)$ number of token-level index entries. The router must still scan or retrieve candidates and then gather KVs from potentially scattered positions.

Token routing minimizes block overfetch and can recover isolated evidence. Its limiting resource at very long context may nevertheless be index bandwidth rather than arithmetic. A low-dimensional or binary proxy is useful only if its reduced comparison cost exceeds the cost of reading the full proxy sequence, performing Top- k , and gathering irregular content memory.

4.3.2 Block-Granularity and Compressed-Entry Routing

Block-granularity routing uses a contiguous unit for final selection and content access. SeerAttention learns a lightweight predictor over pooled Q/K representations and distills block importance from dense attention; the resulting AttnGate chooses original-token KV blocks [91]. SeerAttention-R extends the mechanism to decoding, shares selection within GQA groups, and maintains a compressed key index [92]. SpotAttention learns a calibrated block distribution and uses dual Top- p rules to allocate different budgets across queries and layers [93]. These methods improve access regularity while retaining the full-resolution KVs inside selected blocks, but all three train the router as a plug-in on a frozen pretrained model, so the backbone itself never adapts to block-sparse support.

MiniMax Sparse Attention (MSA) instead trains the backbone under its block-sparse support. MSA is a block-sparse extension of GQA used in MiniMax-M3, a natively multimodal MoE model with about 428B total and 23B activated parameters that applies MSA in 57 of its 60 layers [94, 95]. Its Index Branch shares the MQA-style structure of the DSA Lightning Indexer (middle column of Figure 7) but differs in the role of the index heads: instead of fusing them into one support shared by all attention heads, MSA assigns one index head to each GQA group, so each index head selects support independently for its own group. Index scores are computed at token level, between the index query and every visible index key, whereas selection operates at block level: for index head h , each contiguous KV block u is scored by the maximum of its token scores,

$$e_{t,u,h}^I = \max_{i \in \mathcal{B}_u, i \leq t} \frac{q_{t,h}^I (k_i^I)^\top}{\sqrt{d_I}}, \quad (30)$$

where $\mathcal{B}_u$ is the set of token positions in block u and d_I is the index dimension. The Top- k blocks under $e_{t,:}^I$ are kept together with the block containing the query, over which the query heads of the corresponding GQA group compute exact Softmax attention. Supervision follows the DSA recipe—a KL loss toward the Main Branch attention on the selected tokens, preceded by a full-attention warmup and confined to the index projections by stop-gradients—except that the teacher is averaged over the query heads of each group.

MSA thus uses block-granular Access but a token-granular evidence scan, trading the token-level localization of DSA for regular per-group KV reads. Because the index scan still touches every visible token, its cost continues to grow with context length and ultimately bounds the attainable speedup; regular KV reads do not by themselves reduce the computation and bandwidth consumed by the proxy.

Compressed-entry routing changes the address representation more aggressively. Qwen Sparse Attention (QSA) compresses the index key sequence into micro-block representations before scoring, thereby shrinking the index scan itself rather than only the final selection [13, 96]. In Qwen3.8-Flash-Next (Table 18), QSA replaces the full-attention layers during continued pretraining, with one QSA layer following every three Gated DeltaNet layers. Its indexer shares the MQA-style structure of MSA, but where MSA pools token scores, QSA pools the index keys themselves: the shared key sequence is average-pooled over non-overlapping micro-blocks before positional encoding, so each micro-block receives one content summary and one block position. For query t and micro-block u ,

$$\bar{k}_u^I = \text{AvgPool}(\{k_i^I\}_{i \in \mathcal{B}_u}), \quad e_{t,u}^I = \begin{cases} \sum_{h=1}^{H_I} \text{ReLU}(q_{t,h}^I (\bar{k}_u^I)^\top), & \max \mathcal{B}_u \leq t, \\ -\infty, & \text{otherwise,} \end{cases} \quad (31)$$

so per-head scores are summed without the query-dependent weights of Eq. (29), and only fully observed micro-blocks are scored. Even though the core attention uses GQA, QSA does not select per group as MSA does: the fused score $e_{t,u}^I$ yields a single selection that all GQA groups share. The Top- k micro-blocks under $e_{t,\cdot}^I$ are expanded to their original token positions and joined with the tokens of the current incomplete micro-block, over which core attention reads original-token KVs. Training follows the two stages of DSA—indexer-only dense distillation, then sparse training of the whole model with the KL restricted to the selected blocks—except that the token-level teacher distribution is max-pooled within each micro-block and renormalized to match the block-level scores. Because the index scan shrinks by the micro-block size while content stays at token resolution, QSA compresses addresses without compressing content. In short, DSA and MSA both score every visible token and differ in whether index heads are fused into token scores or token scores are max-pooled into per-group block scores, whereas QSA pools the keys before scoring and is the only one of the three that shortens the index scan.

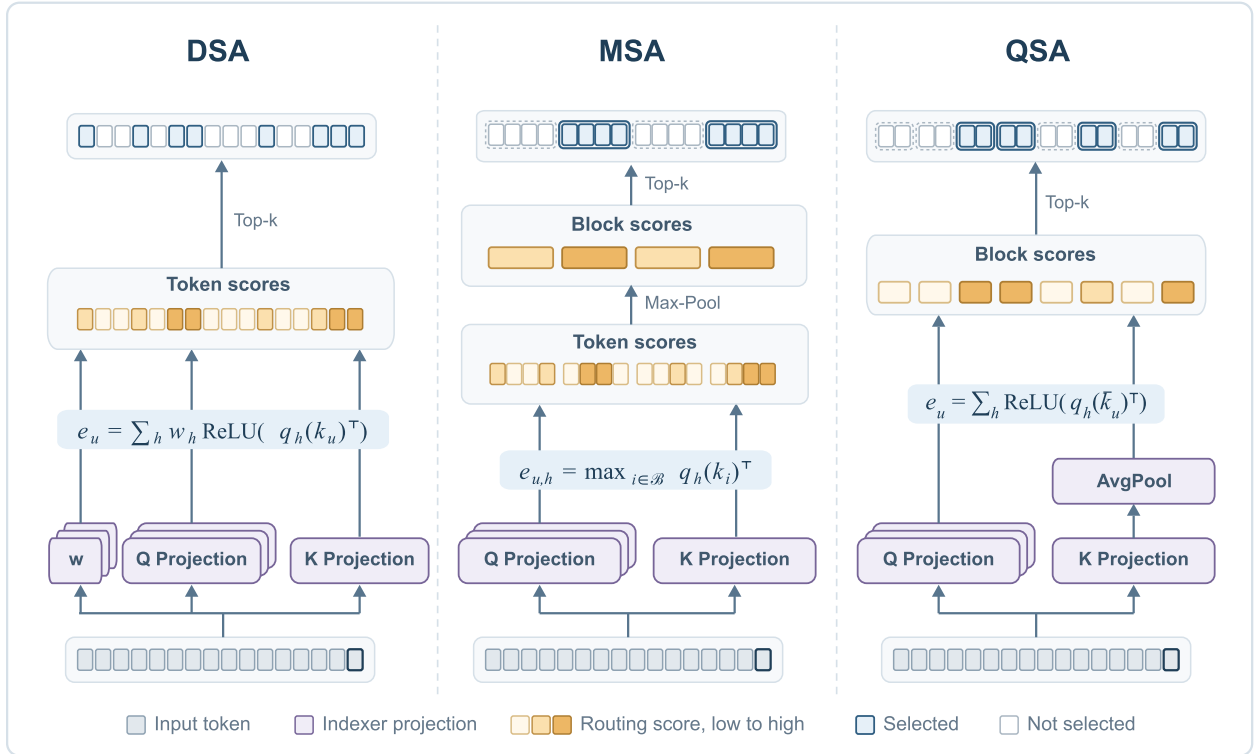


Figure 7: Indexer structures of DSA, MSA, and QSA, which differ mainly in where aggregation enters the index-score computation. The in-figure formulas are simplified; Eqs. (29)–(31) give the full definitions.

Compressed Sparse Attention (CSA) goes further and compresses content as well [50]. Introduced in DeepSeek-V4, where it alternates with Heavily Compressed Attention (HCA) layers that compress more aggressively and attend densely, CSA merges the KV entries of every m tokens into one entry by learned, overlapping weighted pooling,

applies the same compression to the index keys, and scores the compressed blocks with the DSA indexer:

$$\bar{c}_u = \sum_{i \in \mathcal{B}_{u-1} \cup \mathcal{B}_u} \pi_{u,i} \odot c_{u,i}, \quad e_{t,u}^I = \sum_{h=1}^{H_I} w_{t,h}^I \text{ReLU}(q_{t,h}^I (\bar{k}_u^I)^\top), \quad (32)$$

where $c_{u,i}$ is a per-token projection serving as both key and value, $\pi_{u,i}$ are channel-wise softmax weights over the current and preceding blocks, and $\bar{k}_u^I$ is the compressed index key. The Top- k compressed entries, together with an uncompressed sliding window that covers the query’s incomplete block, form the eligible content view used by Readout. Because this view holds compressed entries rather than token KVs, CSA reduces index length, attention work, and resident content memory at the cost of item-level fidelity. QSA and CSA together show why Memory Representation must distinguish address compression from content compression.

Block selection offers fewer routing units and more regular gathers, but it reads irrelevant tokens whenever importance is concentrated inside a block. If evidence is still computed at token resolution, the proxy scan remains linear in the number of tokens. If both the index and content are compressed by a factor m , the scanned sequence can shrink from L to approximately L/m , but the reduction is purchased with coarser routing resolution and, for compressed content, reduced recoverability of individual details.

4.4 Temporal and Cross-Layer Reuse

Once content-adaptive Access can produce a useful support, a complementary optimization is to avoid recomputing similar decisions at every decoding step and layer. Two distinct objects can be reused. Temporal support reuse uses a history of prior query–support relations to propose candidates for a new decoding query, after which a method may rerank or refresh them. Cross-layer reuse lets a non-anchor layer inherit selected positions from an anchor layer; more aggressive designs also let it read K/V content produced by a source layer. These mechanisms change when Access decisions and content states are refreshed rather than defining a new evidence representation. They can therefore be layered onto structure-, self-, or proxy-routed candidate generation and are treated here as cross-cutting lifecycle attributes rather than a mutually exclusive family.

4.4.1 Temporal Support Reuse

Temporal support reuse exploits the observation that nearby or semantically similar decoding queries often retrieve many of the same historical tokens. Rather than accepting an earlier support as the current answer, these methods use past selections only to construct a smaller candidate set. The current query then recomputes exact QK scores over those candidates, selects its own support, and applies ordinary sparse Softmax to the original KVs. Temporal reuse therefore reduces the cost of discovering candidates without reusing stale attention weights or discarding the full KV cache.

ReTopK is a representative recall-before-rerank design [97]. For each query head, it maintains a bounded FIFO cache of normalized historical queries $\bar{q}_\tau = q_\tau / \|q_\tau\|_2$ together with their selected supports $\hat{\mathcal{L}}_\tau$. Given a new query, it first computes cosine similarity to each cached query,

$$\rho_{t,\tau} = \bar{q}_t \bar{q}_\tau^\top, \quad (33)$$

and denotes the indices of the R most similar cache entries by $\mathcal{R}_t$. Their historical supports are merged with a recent window $\mathcal{L}_t$ to form the candidate set

$$\mathcal{A}_t = \mathcal{L}_t \cup \bigcup_{\tau \in \mathcal{R}_t} \hat{\mathcal{L}}_\tau. \quad (34)$$

The recent window keeps newly appended tokens eligible even though they cannot yet appear in a historical support. ReTopK next evaluates the current query against only the recalled candidate keys and reranks them:

$$\hat{\mathcal{L}}_t = \text{TopK}_{i \in \mathcal{A}_t} \left(\frac{q_t k_i^\top}{\sqrt{d}}; K \right). \quad (35)$$

The selected indices are then used in the candidate-local Softmax of Eq. (9). Thus, ReTopK reuses only historical support identities for candidate recall; all retained scores, attention probabilities, and value aggregation are recomputed from the current query.

The main failure mode is support drift: similar queries need not require identical evidence, and errors can accumulate when supports produced by reuse are inserted back into the cache. ReTopK therefore falls back to full-history exact Top- K when the maximum cached-query similarity is below a threshold and periodically performs an exact refresh.

Its efficiency depends on the cache capacity, number R of recalled supports, recent-window width, fallback rate, and refresh interval. If these quantities remain bounded, reranking touches at most the union of R supports and the recent window rather than the full history; if fallback is frequent or the union becomes large, the advantage correspondingly shrinks.

4.4.2 Cross-Layer Reuse

The simplest cross-layer strategy shares selected positions. TidalDecode observes persistence in selected positions across adjacent layers: a small set of selection layers performs full attention and generates indices, while intervening layers reuse them [98]. Cascade similarly computes exact Top- k only at calibrated anchor layers and uses dynamic programming to choose where those anchors should be placed [99].

IndexCache applies this principle directly to the learned indexers of DSA. It assigns every layer one of two roles: an F (Full) layer runs its indexer over the full history and caches fresh Top- k indices, whereas an S (Shared) layer omits its indexer and inherits the indices from the nearest preceding F layer. If $c_\ell \in \{F, S\}$ denotes the role of layer ℓ , the support is

$$\mathcal{I}_t^{(\ell)} = \begin{cases} \text{TopK}(e_{t,:}^{I,(\ell)}; k), & c_\ell = F, \\ \mathcal{I}_t^{(f(\ell))}, & c_\ell = S, \end{cases} \quad f(\ell) = \max\{j < \ell : c_j = F\}. \quad (36)$$

The first layer is always Full, and each later Full layer overwrites the temporary index cache, so reuse adds no persistent index buffer beyond the one already required by DSA. The training-free variant greedily chooses which indexers to retain by measuring language-modeling loss on a calibration set, because uniformly spaced Full layers can remove unusually important indexers. The training-aware variant instead distills each retained indexer from the attention distributions of all layers that will share its output, encouraging one support to serve the whole layer group rather than only its anchor layer [15]. GLM-5.2 deploys this mechanism under the name IndexShare, with one indexer shared by every four sparse-attention layers; GLM-5.3 retains the same base architecture and therefore the same cross-layer indexing design [68, 69].

More extensive reuse shares content memory as well as discrete indices. YOIO/CLSA starts from a self-decoder/cross-decoder architecture: the self-decoder writes one cross-layer KV memory, a single-head query-aware indexer computes token-level Top- k routing once over that memory, and every cross-decoder layer reads its own query-dependent values from the same selected support. A multi-layer distillation target averaged over decoder layers and attention heads trains the shared indexer to preserve tokens that are jointly useful across the stack [100].

Compressed Sparse Attention 2 (CSA2) was introduced by DeepSeek-AI in the technical report *DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression* as the successor to the CSA mechanism used in DeepSeek-V4 [101]. It targets two remaining long-context costs jointly: storing separate global KVs at every layer and repeatedly scanning the full index at every layer. CSA2 therefore shares main KVs and indexer keys across depth while allowing the schedule for refreshing sparse indices to be controlled separately.

Every CSA2 layer is statically assigned one of three modes. A Full layer computes fresh main KVs and indexer keys, scans all causally visible positions, and produces fresh Top- k indices. In the decoder, the first Full layer also builds a larger candidate pool $\mathcal{P}_t^{\text{CSA2}}$ that is reused by later indexing layers. A Reindex layer reuses the main KVs and indexer keys from the most recent Full layer, but forms its own index query and refreshes the Top- k support within that shared candidate pool. A Reuse layer skips index scoring and inherits the latest support produced by either a Full or Reindex layer. If $m_\ell \in \{F, R, U\}$ is the mode of layer ℓ and $r(\ell)$ is its nearest preceding index-producing layer, the reuse schedule is

$$\mathcal{I}_t^{(\ell)} = \begin{cases} \text{TopK}_{i \leq t}(e_{t,i}^{I,(\ell)}; k), & m_\ell = F, \\ \text{TopK}_{i \in \mathcal{P}_t^{\text{CSA2}}}(e_{t,i}^{I,(\ell)}; k), & m_\ell = R, \\ \mathcal{I}_t^{(r(\ell))}, & m_\ell = U. \end{cases} \quad (37)$$

All three modes still compute their own main query and local sliding-window KVs. Full layers refresh shared content, address entries, and support; Reindex layers refresh only the support against reused address entries; and Reuse layers refresh none of them. YOIO/CLSA therefore provides the simpler shared-memory baseline before CSA2 introduces selective reindexing and hierarchical candidate narrowing; this conceptual progression also matches their publication order.

HySparse alternates occasional full-attention layers with groups of sparse layers. It extracts block-level maximum attention scores from a full layer, reuses them as selection evidence for later sparse layers, shares the full layer’s KVs, and preserves local information through a separate sliding-window branch [102]. HySparse2 refines the sparse-attention path in three ways: it replaces block-level retrieval with token-level selection to avoid block overfetch, reuses

both the full layer’s KVs and selected indices across the following sparse layers, and forces a recent window into the selected token set. Local and retrieved tokens therefore use one unified sparse-attention operation over a shared KV cache rather than separate global and sliding-window branches [103]. This design retains fine-grained retrieval while amortizing index computation and KV storage across each group of layers.

Reuse amortizes selection and can also reduce duplicated memory when index and content states are shared. The corresponding loss is specificity. Temporal reuse can lag behind changes in the current query; cross-layer reuse can suppress differences in what shallow and deep layers need to retrieve. Selection or reindex frequency, reuse span, representation compatibility, and correction policy must therefore be reported alongside accuracy and throughput. A claimed speedup should also separate savings in router computation from savings in index traffic, KV traffic, and resident memory.

4.5 Summary

Across these families, the design trajectory reflects a shift in *who decides where to attend*. The earliest designs fixed the support structurally—local, strided, or dilated patterns set before training [70, 10, 11, 71]—and later sink-plus-window rules or profiled pattern families applied structural constraints to trained dense checkpoints [73, 75]. Self-routing restored content adaptivity by letting the backbone score candidates from its own representations, progressing from hashing or clustering [76, 77] and training-free block estimation on a dense checkpoint [78, 79] to training the model *under* its own sparsity, where learned block summaries drive selection [12, 80]. Auxiliary-proxy routing then moved address computation into a separate, distilled index, tunable in dimensionality and granularity and coarsened from tokens to blocks and compressed entries to shrink the index itself [91, 64]. As context windows reach one million tokens and beyond, this per-layer index scan becomes the bottleneck, pushing the frontier from constructing a support to managing its lifecycle—sharing address and content memory and reusing selections across steps and layers [50, 101, 100].

This trajectory has steadily narrowed the gap to dense attention. Training-free sparsity is ultimately an approximation and inherits a training–inference mismatch, so its quality degrades once the retained budget drops below the evidence a task needs. Methods trained under their own sparse support behave differently: because the parameters adapt to a finite candidate set, natively trained selective attention and distilled-then-continued indexers now report quality on par with dense attention—and, at long context, occasionally above it—while cutting both prefill and per-step decoding cost [12, 80, 64]. This parity is conditional rather than universal: it holds when the router is trained end-to-end, the budget matches the task’s evidence density, and the reported speedup accounts for evidence construction, index traffic, and gathering rather than only the final sparse matrix multiplication. Within those conditions, trained sparse attention is a credible option for long-context deployment rather than merely a lossy fallback, although its margin over dense attention remains task-, budget-, and implementation-dependent.

Accordingly, sparse attention has moved from research prototypes into production LLMs, where deployed designs use auxiliary-proxy routing at token (DSA), block (MSA), or compressed-entry (QSA and CSA) granularity, and a subset of recent releases adds cross-layer reuse. DeepSeek-V3.2 pairs each latent token with a low-dimensional index key scored by a Lightning Indexer, aligned to dense attention by distillation and then trained under sparse support [64]; DeepSeek-V4 aggregates index keys and KV entries into a compressed global memory combined with a sliding window [50]; and DeepSeek-V4.1-flash organizes layers into Full, Reindex, and Reuse modes that share indexer keys and KVs along depth [101]. Qwen3.8-flash-next compresses token routing keys into micro-block indices that expand to original-token KVs [96], MiniMax scores token-level index keys and max-pools them into block selections [94], and later GLM designs incorporate cross-layer index sharing [68]. LongCat-2.0 and LongCat-Flash-Lite-Sparse deploy LSA, combining streaming-aware support, hierarchical index scoring, and cross-layer index reuse [16, 104]. The emerging recipe—a learned, compact index, optionally compressed across positions, aligned through native training or distillation and continued training, and increasingly reused across depth—shows that the mechanism-level developments surveyed here now underpin mainstream long-context architectures.

5 Linear Attention

Softmax Attention preserves contextual history as an enumerable collection of token- or chunk-level key–value representations, whereas Sparse Attention primarily restricts which of these explicit memory units are eligible for a query. Linear Attention changes the memory substrate itself: by exploiting factorizable query–key interactions, it accumulates historical key–value associations into one or more recurrently maintained associative states without retaining the complete token-wise KV history. The canonical formulation maintains a context-length-independent state, while later variants enlarge or partition the associative space, preserve multiple states over different temporal ranges, or construct block-level associative summaries. Linear Attention therefore trades item-wise historical addressability for recurrent

compression, shifting the central design problem from selecting among stored tokens to determining how compressed associative memory is represented, updated, accessed, and read.

Because the term *Linear Attention* covers several related formulations, we focus here on causal mechanisms that represent contextual history through one or more recurrently maintained associative states. Starting from the kernel-factorized formulation of Linear Transformer, we examine how this memory is accumulated, retained, corrected, expanded, accessed, and read. This perspective also includes gated Linear RNNs whose principal operation is associative state editing. Methods developed primarily from structured state-space dynamics are discussed in Section 6, while architectures that combine Linear Attention with explicit attention or other heterogeneous sequence mixers are examined in Section 7.

Linear Transformer provides the canonical operator. Suppose a kernelized similarity admits the separable form $K(q_t, k_i) = \phi(q_t)^\top \phi(k_i)$. For normalized causal Linear Attention, historical key–value information can be accumulated into an associative matrix and a normalization state,

$$S_t = S_{t-1} + \phi(k_t)v_t^\top, \quad z_t = z_{t-1} + \phi(k_t), \quad (38)$$

and read as

$$o_t = \frac{\phi(q_t)^\top \hat{S}_t}{\phi(q_t)^\top \hat{z}_t + \varepsilon}, \quad (39)$$

where $(\hat{S}_t, \hat{z}_t)$ denotes the state visible under the model’s causal read–write convention and ε represents an optional numerical stabilizer. The usual normalized form assumes a feature map and kernel for which the denominator is well defined; later recurrent variants may instead use unnormalized state contractions, output normalization, or learned gates. During autoregressive decoding, the canonical formulation maintains only (S_t, z_t) , while parallel or chunkwise algorithms can expose the same recurrence during training [17, 19].

Within the framework introduced in Section 2, the development of Linear Attention begins with a fundamental change in Memory Representation and Memory Update. Its canonical formulation replaces an enumerable token-wise history with an associative matrix state and recurrently incorporates new key–value associations into this compressed representation. Later methods refine the state transition through retention, correction, erasure, and controlled writing, while enlarging or reorganizing the maintained memory through dense dimensional expansion, sparse address spaces, multiple temporal states, and block-level associative summaries. These developments also reshape the other stages of memory processing. A differentiated state organization requires Access to determine which maintained units are exposed to the current query, Readout to decode and aggregate the eligible state information, and Integration to coordinate completed readouts when multiple heads, states, or memory paths contribute to the module output. The five-dimensional view therefore reveals a coupled design trajectory: changes in how associative memory is represented and updated progressively alter what can be accessed, how it can be read, and how the resulting contextual information is incorporated into the network.

Following this trajectory, we first examine Memory Update within a single associative state, from additive accumulation to increasingly controlled forms of retention, correction, erasure, and writing. We then turn to Memory Representation and study how associative memory is enlarged or reorganized through dense dimensional expansion, sparse address spaces, temporal state collections, and block-level summaries. On this basis, we analyze how differentiated memory units introduce new Access and Readout choices, including routing among maintained states, normalized state decoding, and aggregation across multiple summaries. We finally consider how completed readouts are integrated within a layer and how memory-derived signals may be coordinated across network depth. Throughout the section, the functional analysis is paired with the corresponding computational implications, including persistent state growth, routing and merging overhead, parallel training form, and the maturity of the available empirical evidence.

5.1 Memory Update: From Accumulation to Controlled State Editing

For comparison across single-state methods, we use an analytical decomposition into retain/decay, erase/correct, and write/commit. This decomposition identifies distinguishable update roles; it does not assert that every method implements three independent modules or applies them in an identical computational order. For a model with H attention heads, the complete memory is $S_t \in \mathbb{R}^{H \times d_k \times d_v}$, and head h maintains $S_t^h \in \mathbb{R}^{d_k \times d_v}$. To simplify notation, the head index is omitted below and $S_t \in \mathbb{R}^{d_k \times d_v}$ denotes a single-head state. Its update is summarized as

$$\bar{S}_{t-1} = \alpha_t \odot S_{t-1}, \quad S_t = \bar{S}_{t-1} - E_t + W_t, \quad (40)$$

where $\bar{S}_{t-1}$ is the retained old state, E_t is the association erased or corrected from that state, and W_t is the current write.

5.1.1 Retain / Decay

Retain / Decay controls how much of the old state remains before new information is written. Existing methods differ in both the source and the granularity of α_t . The coefficient can be a fixed parameter after training or can be generated dynamically from the current input through $f_\alpha(x_t)$. Its granularity can be a head-wise scalar $\alpha_t^h \in \mathbb{R}$ or a channel-wise vector $\alpha_t^h \in \mathbb{R}^{d_k}$ acting on the d_k state rows. The development begins with complete retention without decay, then introduces fixed decay with predefined time scales, and finally moves to input-dependent, fine-grained retention. Table 4 summarizes this progression.

Table 4: Evolution of Retain / Decay mechanisms in Linear Attention.

Retain category	Period	Unified form	Representative methods
No decay	2020–2025	$\alpha_t^h = 1$	Linear Transformer, DeltaNet, DeltaProduct
Fixed head-wise scalar	2023–2024	$\alpha_t^h \in \mathbb{R}$	RetNet, Lightning Attention-2
Fixed channel-wise vector	2024	$\alpha_t^h \in \mathbb{R}^{d_k}$	RWKV-5
Input-dependent head-wise scalar	2024	$\alpha_t^h = f_\alpha(x_t) \in \mathbb{R}$	GDN
Input-dependent channel-wise vector	2023–2026	$\alpha_t^h = f_\alpha(x_t) \in \mathbb{R}^{d_k}$	GLA, RWKV-6/7, HGRN2, KDA, GDN2, EDA

No decay. Linear Transformer, DeltaNet, and DeltaProduct set $\alpha_t^h = 1$, so historical state does not decay actively with time [17, 19, 105, 106]. This preserves the accumulated history, but early information continues to occupy the finite state space, and the model cannot vary the retention strength of different historical components according to time or input content.

Fixed decay. Fixed decay introduces stable time scales through predefined retention coefficients. RetNet and the fixed-decay Lightning Attention-2 formulation use head-wise scalar decay, so different heads cover histories of different lengths; RWKV-5 uses channel-wise time decay, allowing different state channels within the same head to retain information at different rates [18, 107, 108].³ Compared with undecayed accumulation, fixed decay continually reduces the weight of earlier history and forms multiscale memory, but its forgetting rate does not change with input content at inference time.

Input-dependent decay. Input-dependent decay makes the retention coefficient a function of the current input. GLA uses a channel-wise vector gate, whereas GDN uses an input-dependent head-wise scalar, allowing different tokens to regulate historical retention globally or by channel [110, 20]. HGRN2 also uses an input-dependent channel-wise forget gate and assigns a retention lower bound that increases with layer depth. At layer ℓ ,

$$g_t^{(\ell)} = \sigma\left(f_g(x_t^{(\ell)})\right), \quad \alpha_t^{(\ell)} = \beta_\ell + (\mathbf{1} - \beta_\ell) \odot g_t^{(\ell)} \in (0, 1)^{d_k}, \quad (41)$$

where $g_t^{(\ell)}$ is generated from the current input and the channel-wise lower bound $\beta_\ell \in [0, 1]^{d_k}$ increases elementwise and monotonically with depth. This constraint allows lower layers to update local information more rapidly while higher layers retain longer historical time scales [111]. RWKV-6/7, KDA, GDN2, and EDA also generate input-dependent channel-wise decay $\alpha_t \in (0, 1)^{d_k}$ to control retention in individual state rows [108, 112, 113, 114, 115]. Retention thus develops from a fixed temporal prior into token-dependent and channel-aware control, with cross-layer constraints further organizing different time scales.

5.1.2 Erase / Correct

Erase / Correct describes which associations are removed or revised in the old state before new information is written. Let $X_{t-1}^e \in \mathbb{R}^{d_k \times d_v}$ be the source state actually read by the erase operation, let $R_t^e, U_t^e \in \mathbb{R}^{d_k \times m_t^e}$ be the read directions and state-update directions, and let m_t^e be the effective erase rank. The total erase term is

$$E_t = U_t^e \left((R_t^e)^\top X_{t-1}^e \right) \in \mathbb{R}^{d_k \times d_v}. \quad (42)$$

Erase mechanisms evolve from no explicit removal, to delta correction in which read and update directions are coupled, and then to designs that use different parameters for the two directions or combine multiple erase components. Table 5 summarizes this progression.

³RWKV-4 also uses fixed channel-wise time decay, but it maintains normalized vector-valued accumulators rather than the matrix-valued associative state assumed in this subsection; it is therefore omitted from the comparison tables [109].

Table 5: Evolution of Erase / Correct mechanisms in Linear Attention.

Erase category	Period	Unified form	Representative methods
No explicit erase	2020–2024	$m_t^e = 0, E_t = 0$	Linear Transformer, RetNet, GLA, RWKV-5/6, HGRN2, Lightning Attention-2
Coupled corrective erase	2021–2025	$m_t^e = 1, U_t^e = \beta_t k_t, R_t^e = k_t$	DeltaNet, GDN, KDA, DeltaProduct
Decoupled erase	2025–2026	U_t^e and R_t^e need not be aligned and/or $m_t^e > 1$	RWKV-7, GDN2, EDA

No explicit erase. When $m_t^e = 0$, no independent erase term is constructed and $E_t = 0$. Linear Transformer [17], RetNet [18], GLA [110], RWKV-5/6 [108], HGRN2 [111], and Lightning Attention-2 [107] do not read the old content at the current address and remove it directionally before writing. In the unified decomposition, HGRN2 contains only $\bar{S}_{t-1} = D_t S_{t-1}$ and no address-specific removal, so it also belongs to this category.

Coupled corrective erase. DeltaNet uses the same direction to read and modify the old association at the current address. For $k_t \in \mathbb{R}^{d_k}$ and scalar correction rate $\beta_t \in \mathbb{R}$,

$$X_{t-1}^e = \bar{S}_{t-1}, \quad m_t^e = 1, \quad U_t^e = \beta_t k_t, \quad R_t^e = k_t, \quad (43)$$

which gives $E_t = \beta_t k_t (k_t^\top \bar{S}_{t-1})$. Because R_t^e and U_t^e are both determined by k_t , the old association is read and modified along aligned directions. GDN and KDA retain this rank-one correction, while DeltaProduct performs multiple corrections of the same type within one token [19, 105, 20, 113, 106].

Decoupled erase. RWKV-7, GDN2, and EDA no longer require every erase component to use identical read and state-update directions. GDN2 defines a key-side gate $b_t \in \mathbb{R}^{d_k}$ and uses

$$X_{t-1}^e = \bar{S}_{t-1}, \quad m_t^e = 1, \quad U_t^e = k_t, \quad R_t^e = b_t \odot k_t. \quad (44)$$

Here $b_t \odot k_t$ determines the content read from the old state, while k_t determines the state-update direction, producing an asymmetric rank-one erase [114].

EDA performs an independent pre-erase followed by delta correction. Let $e_t, k_t \in \mathbb{R}^{d_k}$ be the pre-erase and correction directions and $\gamma_t, \beta_t \in \mathbb{R}$ their strengths. Expanding the two operations gives

$$X_{t-1}^e = \bar{S}_{t-1}, \quad U_t^e = [\gamma_t e_t \quad \beta_t k_t], \quad R_t^e = [e_t \quad k_t - \gamma_t (e_t^\top k_t) e_t], \quad m_t^e = 2. \quad (45)$$

The first component performs pre-erasure along e_t , and the second performs correction along k_t after pre-erasure; together they form a rank-two total erase [115].

RWKV-7 uses a normalized removal key $\hat{k}_t \in \mathbb{R}^{d_k}$ and a vector-valued removal rate $b_t \in \mathbb{R}^{d_k}$,

$$X_{t-1}^e = S_{t-1}, \quad m_t^e = 1, \quad R_t^e = \hat{k}_t, \quad U_t^e = b_t \odot \hat{k}_t. \quad (46)$$

The read direction is defined by R_t^e , whereas U_t^e adjusts the state-update direction through channel-wise removal strength [112, 113].

5.1.3 Write / Commit

Write / Commit specifies the location, content, and strength with which current information is committed. Let $A_t^w \in \mathbb{R}^{d_k \times r_t^w}$ and $C_t^w \in \mathbb{R}^{d_v \times r_t^w}$ denote write directions and contents, where r_t^w is the effective write rank. The write term is

$$W_t = A_t^w (C_t^w)^\top \in \mathbb{R}^{d_k \times d_v}. \quad (47)$$

Write evolves from direct writing without additional regulation, to corrective writing controlled by an update rate, and then to decoupled writing with independent control of write direction or content. Table 6 summarizes the progression.

Table 6: Evolution of Write / Commit mechanisms in Linear Attention.

Write category	Period	Unified form	Representative methods
Direct write	2020–2024	$A_t^w = k_t, C_t^w = v_t$	Linear Transformer, RetNet, GLA, RWKV-5/6, Lightning Attention-2
Corrective write	2021–2025	$A_t^w = \beta_t k_t, C_t^w = v_t$	DeltaNet, GDN, KDA, DeltaProduct
Erase–write decoupling	2025–2026	Separate control of erase and write address or content	RWKV-7, GDN2, EDA

Direct write. The earliest Linear Attention methods directly add a rank-one key–value association,

$$r_t^w = 1, \quad A_t^w = k_t, \quad C_t^w = v_t, \quad W_t = k_t v_t^\top. \quad (48)$$

Linear Transformer establishes this form [17]. RetNet, GLA, RWKV-5/6, and Lightning Attention-2 use the same direct outer-product write or an equivalent recurrence [18, 110, 108, 107]. HGRN2 sets $A_t^w = \mathbf{1} - \alpha_t$ and $C_t^w = i_t$, where $i_t \in \mathbb{R}^{d_v}$ is candidate content, giving $W_t = (\mathbf{1} - \alpha_t) i_t^\top$; equivalently, one may relabel its write address as $k_t = \mathbf{1} - \alpha_t$ [111].

Corrective write. Delta-style methods use a scalar update rate to control commit strength,

$$r_t^w = 1, \quad A_t^w = \beta_t k_t, \quad C_t^w = v_t, \quad W_t = \beta_t k_t v_t^\top. \quad (49)$$

The coefficient β_t determines how strongly the current association enters the state. DeltaNet establishes this rate-controlled commit; GDN and KDA retain scalar-controlled writing; and DeltaProduct generates multiple write components within one token [19, 105, 20, 113, 106].

Erase–write decoupling. Recent methods decouple the erase and write roles through separate address or content controls. GDN2 uses

$$r_t^w = 1, \quad A_t^w = k_t, \quad C_t^w = w_t \odot v_t, \quad (50)$$

where $w_t \in \mathbb{R}^{d_v}$ is a value-side gate controlling the committed content channel by channel [114]. EDA retains the corrective write $A_t^w = \beta_t k_t, C_t^w = v_t$, and $W_t = \beta_t k_t v_t^\top$, while decoupling it from the independently addressed pre-erase [115]. RWKV-7 uses a distinct add key with $A_t^w = k_t, C_t^w = v_t$, and $W_t = k_t v_t^\top$ [112, 113].

5.1.4 Representative Recurrent Update Rules

Retain, Erase, and Write jointly determine the full recurrence. Table 7 lists representative methods chronologically and marks their Decay, Erase, and Write types. All equations use the single-head orientation $S_t \in \mathbb{R}^{d_k \times d_v}$ and retain only the core update terms. Let $I \in \mathbb{R}^{d_k \times d_k}$ and $D_t = \text{Diag}(\alpha_t)$; when decay is scalar, $D_t = \alpha_t I$.

The formulas use unified notation to emphasize Update structure and omit method-specific normalization, feature generation, and output gating. In HGRN2, $\alpha_t \in (0, 1)^{d_k}$ is the channel-wise forget gate and $i_t \in \mathbb{R}^{d_v}$ is candidate content; $\mathbf{1} - \alpha_t$ also forms its direct-write direction. In DeltaProduct, R is the number of delta transformations performed for one token and $\mathcal{T}_{t,r}$ is the r th update. The method-specific symbols and decoupling relations for RWKV-7, GDN2, and EDA are defined above.

5.1.5 Loss-Based Optimization View

The same recurrent rules admit a complementary comparison through equivalent learning objectives that reproduce their state updates. Prior work has interpreted recurrent associative-memory updates as online optimization over an internal state, including fast-weight delta rules, test-time training and regression, and optimization-based analyses of gated Linear Attention [19, 116, 117, 113]. We therefore compare representative methods by identifying a per-token objective whose unit-step gradient update yields the corresponding recurrence,

$$S_t = S_t^- - \nabla_{S_t^-} \mathcal{L}_t(S_t^-), \quad (51)$$

where the optimization state S_t^- can be the previous state, a decayed state, or an intermediate state produced by an earlier optimization step.

Table 7: Representative recurrent update rules.

Method	Decay	Erase	Write	Year	Unified update rule
Linear Transformer [17]	No decay	No explicit erase	Direct	2020	$S_t = S_{t-1} + k_t v_t^\top$
DeltaNet [19, 105]	No decay	Coupled corrective	Corrective	2021	$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
RetNet [18]	Fixed head-wise	No explicit erase	Direct	2023	$S_t = \alpha^h S_{t-1} + k_t v_t^\top$
GLA [110]	Input-dependent channel-wise	No explicit erase	Direct	2023	$S_t = D_t S_{t-1} + k_t v_t^\top, D_t = \text{Diag}(f_\alpha(x_t))$
RWKV-5 [108]	Fixed channel-wise	No explicit erase	Direct	2024	$S_t = \text{Diag}(\alpha^h) S_{t-1} + k_t v_t^\top$
RWKV-6 [108]	Input-dependent channel-wise	No explicit erase	Direct	2024	$S_t = D_t S_{t-1} + k_t v_t^\top, D_t = \text{Diag}(f_\alpha(x_t))$
HGRN2 [111]	Input-dependent channel-wise	No explicit erase	Direct	2024	$S_t = D_t S_{t-1} + (\mathbf{1} - \alpha_t) i_t^\top$
GDN [20]	Input-dependent head-wise	Coupled corrective	Corrective	2024	$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
Lightning Attention-2 [107]	Fixed head-wise	No explicit erase	Direct	2024	$S_t = \alpha^h S_{t-1} + k_t v_t^\top$
DeltaProduct [106]	No decay	Coupled corrective	Corrective	2025	$S_t = (\mathcal{T}_{t,R} \circ \dots \circ \mathcal{T}_{t,1})(S_{t-1})$
RWKV-7 [112]	Input-dependent channel-wise	Decoupled	Erase-write decoupled	2025	$S_t = [D_t - (b_t \odot \hat{k}_t) \hat{k}_t^\top] S_{t-1} + k_t v_t^\top$
KDA [113]	Input-dependent channel-wise	Coupled corrective	Corrective	2025	$S_t = (I - \beta_t k_t k_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top$
GDN2 [114]	Input-dependent channel-wise	Decoupled	Erase-write decoupled	2026	$S_t = [I - k_t (b_t \odot k_t)^\top] D_t S_{t-1} + k_t (w_t \odot v_t)^\top$
EDA [115]	Input-dependent channel-wise	Decoupled	Erase-write decoupled	2026	$S_t = (I - \beta_t k_t k_t^\top) (I - \gamma_t e_t e_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top$

Table 8: Survey-derived equivalent per-token objectives corresponding to representative update rules.

Method	Per-token objective $\mathcal{L}_t$	Resulting update
Linear Transformer [17]	$-\langle S_{t-1}^\top k_t, v_t \rangle$	$S_t = S_{t-1} + k_t v_t^\top$
RetNet [18]	$-\langle S_{t-1}^\top k_t, v_t \rangle + \frac{1}{2} \ \sqrt{1 - \alpha^h} S_{t-1}\ _F^2$	$S_t = \alpha^h S_{t-1} + k_t v_t^\top$
GLA [110]	$-\langle S_{t-1}^\top k_t, v_t \rangle + \frac{1}{2} \ \sqrt{I - D_t} S_{t-1}\ _F^2$	$S_t = D_t S_{t-1} + k_t v_t^\top$
HGRN2 [111]	$-\langle S_{t-1}^\top (\mathbf{1} - \alpha_t), i_t \rangle + \frac{1}{2} \ \sqrt{I - D_t} S_{t-1}\ _F^2$	$S_t = D_t S_{t-1} + (\mathbf{1} - \alpha_t) i_t^\top$
DeltaNet [19, 105]	$\frac{\beta_t}{2} \ S_{t-1}^\top k_t - v_t\ _2^2$	$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
GDN [20]	$\frac{\beta_t}{2} \ \tilde{S}_{t-1}^\top k_t - v_t\ _2^2$, optimized at $\tilde{S}_{t-1} = \alpha_t S_{t-1}$	$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
KDA [113]	$\frac{\beta_t}{2} \ \tilde{S}_{t-1}^\top k_t - v_t\ _2^2$, optimized at $\tilde{S}_{t-1} = D_t S_{t-1}$	$S_t = (I - \beta_t k_t k_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top$
DeltaProduct [106]	$\mathcal{L}_{t,r} = \frac{\beta_{t,r}}{2} \ S_{t,r-1}^\top k_{t,r} - v_{t,r}\ _2^2, r = 1, \dots, R$	$S_t = (\mathcal{T}_{t,R} \circ \dots \circ \mathcal{T}_{t,1})(S_{t-1})$
EDA [115]	$\mathcal{L}_t^e = \frac{\gamma_t}{2} \ \tilde{S}_{t-1}^\top e_t\ _2^2$, then $\mathcal{L}_t^w = \frac{\beta_t}{2} \ (S_t^e)^\top k_t - v_t\ _2^2$, with $\tilde{S}_{t-1} = D_t S_{t-1}$	$S_t = (I - \beta_t k_t k_t^\top) (I - \gamma_t e_t e_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top$

For DeltaProduct, each optimization step is $\mathcal{T}_{t,r}(S) = (I - \beta_{t,r} k_{t,r} k_{t,r}^\top) S + \beta_{t,r} k_{t,r} v_{t,r}^\top$, with $S_{t,0} = S_{t-1}$. For EDA, the first objective produces $S_t^e = (I - \gamma_t e_t e_t^\top) \tilde{S}_{t-1}$, which is then used as the optimization state of the second objective.

Table 8 reveals two broad stages in the development of single-state updates. Early additive methods maximize the correlation between the state prediction and the current value. Their differences arise from the regularization applied to the old state: Linear Transformer has no forgetting penalty, RetNet introduces isotropic fixed decay, GLA replaces it with an input-dependent channel-wise penalty, and HGRN2 further couples the write address to the complementary forget gate. These objectives produce direct writes whose magnitude does not depend on the value already predicted at the current address.

Delta-style methods replace correlation maximization with reconstruction-error minimization, making the write explicitly residual dependent. DeltaNet performs one correction on the previous state. GDN and KDA move the same correction to a decayed optimization state, with GDN using a head-wise scalar and KDA using channel-wise decay. DeltaProduct increases optimization depth by fitting multiple key-value targets sequentially within one token. EDA instead expands the objective sequence: it first fits a zero target at an independently selected erase address and then

fits the current value at the write address. The progression therefore changes not only the form of the loss, but also the state at which it is optimized, the number of optimization steps, and whether erase and write share the same address and target.

5.2 Capacity Expansion

By 2025, decay and delta-style editing enabled a single state to use bounded memory more effectively, but the complete history was still compressed into one $d_k \times d_v$ matrix. Better update rules reduce conflicts but cannot provide an unlimited number of independent storage locations; directly enlarging a dense state also increases every token’s read and write cost. Capacity Expansion therefore relaxes the assumption that each head maintains only one dense state and enlarges available memory through sparse selection.

Standard Linear Attention maintains one matrix state $S_t \in \mathbb{R}^{d_k \times d_v}$ per head. Capacity Expansion generalizes it to M states,

$$\mathcal{S}_t = \left\{ S_t^{(m)} \in \mathbb{R}^{d_k \times d_v} \right\}_{m=1}^M, \quad \mathcal{S}_t \in \mathbb{R}^{M \times d_k \times d_v}. \quad (52)$$

Flattening the state and key dimensions gives an equivalent row matrix of shape $(Md_k) \times d_v$. SSE and SDM both increase capacity by adding available state rows, but SSE retains a group structure whereas SDM uses a larger flat memory table. Under this unified row-memory view, routing determines which groups or rows are eligible for the current query and therefore belongs to Access, while weighting and aggregating the selected rows into one contextual representation belong to Readout. If a grouped formulation instead produces separate group- or state-specific readouts before combining them, that subsequent combination belongs to Integration. The distinction therefore depends on the analytical granularity: groups may be treated as internal partitions of one expanded state or as distinct readout paths; for SSE and SDM in this subsection, we use the former interpretation unless separate completed readouts are made explicit.

5.2.1 Sparse State Expansion (SSE)

Sparse State Expansion (SSE) enlarges the associative memory by maintaining multiple groups of state rows [21]. Each group provides a separate region of the expanded state space, while a learned router activates only a small number of groups for the current token. Within the active groups, separate coefficients determine how strongly individual rows participate in writing and reading. The resulting organization is hierarchical: routing first identifies relevant groups and then distributes the update or readout within those groups.

This hierarchy gives SSE a strong structural prior. Group-level selection reduces the number of state regions considered by each token and provides a comparatively regular execution pattern. Because read and write operate within the same group organization, the structure can also help preserve a relationship between where information is stored and where later queries search for it. At the same time, predefined group boundaries restrict direct competition across the full memory space. Rows in an inactive group cannot participate in the current operation, even when their contents might be relevant, and the fixed partition may limit how memory units reorganize as their roles evolve during training.

SSE therefore expands capacity through a balance between specialization and regularity. Increasing the number of groups enlarges the persistent memory without requiring every token to interact with every row, but the effective benefit depends on whether the router distributes information across groups, avoids repeatedly overloading a small subset, and selects groups that remain useful for later retrieval.

5.2.2 Sparse Delta Memory (SDM)

Sparse Delta Memory (SDM) removes the predefined group boundaries and organizes the expanded state as a flat collection of recurrent memory rows [22]. Product-Key Memory routers retrieve a limited number of rows for writing and reading. Rows selected for writing receive delta-style updates, while unselected rows preserve their previous contents. A separate read router identifies the rows that contribute to the current output.

The flat organization gives SDM greater addressing freedom than a fixed grouped structure. Any row can, in principle, compete for the current write or read operation, and the write and read supports can be generated independently. This separation allows the system to distinguish where incoming information should be stored from where a later query should search. It also makes the enlarged memory less dependent on a manually imposed partition of the row space.

The same flexibility introduces a more demanding coordination problem. A row may be written frequently but rarely retrieved, while another may be retrieved before receiving sufficiently relevant updates. Independently learned routers can also develop incompatible address conventions, weakening the connection between storage and later recall. More-

over, the practical efficiency of a large flat state depends on fast retrieval, balanced row utilization, and memory access patterns that do not offset the savings obtained from sparse activation.

5.2.3 Comparison and Open Design Space

Table 9 compares SSE and SDM. Both expand a dense matrix into row memory and use sparse read/write operations to control per-token cost; SSE uses fixed groups for hierarchical routing, whereas SDM performs independent retrieval directly in a flat row space.

Table 9: Comparison of SSE and SDM under a unified row-memory view.

Aspect	SSE	SDM
Unified state	$S_t \in \mathbb{R}^{N \times d_v}$, $N = Md_k$; each row is a d_v -dimensional memory unit	
State organization	M fixed groups with d_k rows per group	Flat row table without group boundaries
Write selection	Group Top- K_g plus within-group coefficients	Independent PKM write router selecting Top- W rows
Read selection	Read over selected groups and row support	Independent PKM read router selecting Top- R rows
Read/write relation	Shared group structure; strongly related supports	Independent read and write routers
Capacity expansion	Increase the number of groups	Directly increase the number of flat rows
Routing characteristic	Hierarchical and regular, with a group prior	Flat and flexible, but more dependent on fast approximate retrieval

Fixed groups reduce the search range and regularize routing, but a predefined partition can restrict dynamic reorganization among memory units. Flat routing provides greater addressing freedom, but must sustain low-cost, high-recall retrieval over a much larger row space. A hierarchical combination of grouped and flat routing could provide both coarse partitioning and fine-grained global selection.

Coordination between Read and Write also remains unresolved. Shared or related supports keep written content aligned with later retrieval but may prevent a query from recalling other memory regions. Fully independent routers are more flexible, yet can produce uneven write utilization, rows that are rarely read, or memory units retrieved before they are adequately trained. Explicit read–write consistency objectives, load balancing, and state-utilization regularization may improve this trade-off.

As row count increases, routing cost, cross-layer memory organization, and state lifetime become increasingly important. Potential directions include reusing routing decisions across neighboring layers, allocating or reclaiming rows according to usage frequency, and allowing different layers to access shared collections of state memory. Capacity Expansion therefore requires the joint design of Memory Representation, sparse Access, Readout, and update dynamics rather than merely increasing state size.

5.3 Temporal Expansion

Capacity Expansion increases rows within the associative space, but history may still be compressed into one temporally uniform memory. When recent details and distant information remain mixed in the same state, different temporal ranges share the same representation resolution and cannot be independently emphasized during readout. Temporal Expansion instead preserves multiple states along the token dimension. If $S_t^{(m)}$ summarizes a contiguous historical interval, then

$$\mathcal{S}_t = \left\{ S_t^{(m)} \right\}_{m=1}^{M_t}. \quad (53)$$

The query first produces a state-specific Readout from each retained state, after which Integration combines these readouts,

$$o_t = \sum_{m=1}^{M_t} \lambda_{t,m} \text{Read}\left(q_t, S_t^{(m)}\right), \quad (54)$$

where M_t is the number of retained segment states and $\lambda_{t,m}$ controls the contribution of state m during Integration.

The central questions are how tokens are partitioned into segments, whether old states are retained or merged as the context grows, and how the current query combines information from different temporal intervals. Log-Linear Attention [23] uses a deterministic multiscale hierarchy, Dynamic Linear Attention [24] uses content-dependent segmentation and merging, whereas Multi-Head Linear Attention [118] preserves a flat collection of fixed-granularity token-block

summaries. These mechanisms all expand memory along the token dimension, but differ in temporal organization, state-count control, and cross-state Integration.

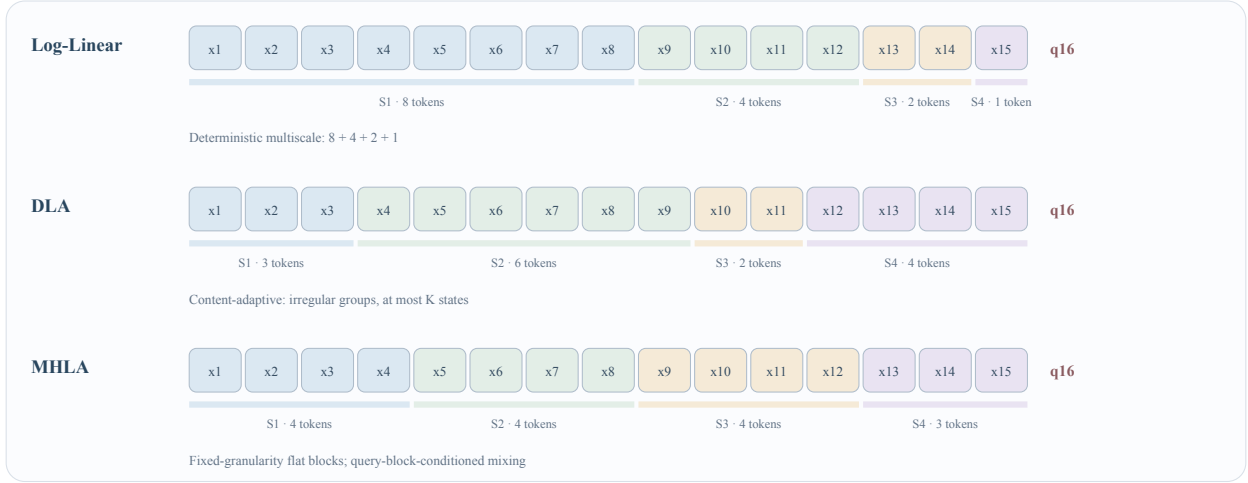


Figure 8: Comparison of temporal grouping strategies in Log-Linear Attention, DLA, and MHLA. Log-Linear Attention deterministically organizes history into multiscale intervals, DLA forms irregular content-adaptive segments under a bounded state budget, and MHLA preserves a flat collection of fixed-granularity token blocks with query-block-conditioned mixing.

5.3.1 Log-Linear Attention

Log-Linear Attention uses a deterministic multiscale organization to avoid compressing the entire history into one state [23]. A Fenwick-tree or power-of-two schedule maintains $O(\log t)$ states. As new tokens arrive, shorter segments are merged according to a binary-carry rule, producing contiguous historical groups such as $8 + 4 + 2 + 1$. Recent information is preserved in short segments at high resolution, while distant history is compressed into longer segments, and the number of states grows only logarithmically with sequence length. Its limitation is that segmentation depends entirely on position, so a fixed merge can cross an important semantic boundary.

5.3.2 Dynamic Linear Attention (DLA)

DLA replaces fixed temporal partitioning with content-dependent segmentation [24]. It uses a State Information Score to determine whether the current token should continue to accumulate into an existing state or create a new segment state when the information changes substantially. When the number of states reaches an upper bound K , DLA merges adjacent states with the lowest information density. Relative to Log-Linear Attention, it allocates more of the limited state budget to regions with dense semantic change while keeping memory bounded by $M_t \leq K$. The cost is that boundaries and merges depend on dynamic scores, and history that has already been merged generally cannot recover its original temporal resolution.

5.3.3 Multi-Head Linear Attention

Multi-Head Linear Attention (MHLA) partitions the token sequence into fixed blocks and preserves a separate key-value summary for each block [118]. Its token-level “heads” therefore differ from conventional feature heads, which partition the channel dimension. Rather than merging all block contributions into one global state, MHLA allows each query block to form a learned mixture of the retained summaries, followed by the usual query-key interaction within each block. From the temporal-memory perspective, MHLA is a flat expansion with approximately uniform chunk resolution: it preserves greater distinction among historical intervals than a single recurrent state, but unlike Log-Linear Attention and DLA, it does not intrinsically merge older summaries or bound their number. Its efficiency therefore depends on controlling both the number of retained blocks and the cost of inter-block mixing.

5.3.4 Comparison and Development Trends

Log-Linear Attention, DLA, and MHLA all preserve multiple associative summaries over different token intervals, but organize temporal resolution differently. Log-Linear Attention uses a deterministic multiscale hierarchy, DLA creates

and merges segments according to content, and MHLA retains a flat collection of fixed-granularity blocks. Table 10 compares their segmentation criteria, state-management rules, and readout mechanisms.

Table 10: Comparison of representative Temporal Expansion mechanisms.

Aspect	Log-Linear Attention	DLA	MHLA
Segmentation basis	Position-dependent Fenwick or power-of-two schedule	Content-dependent State Information Score	Fixed token-block partition
Segment form	Multiscale contiguous intervals	Unequal-length semantic intervals	Flat, usually equal-granularity blocks
Number of states	$M_t = O(\log t)$	$M_t \leq K$	Determined by the number of retained blocks
State management	Hierarchical binary-carry merging	Dynamic creation and information-based merging	Independent block summaries without intrinsic merging
Temporal resolution	Fine for recent and coarse for distant history	Allocated according to semantic change	Approximately uniform across blocks
Cross-state Integration	Query-dependent integration of temporal-scale readouts	Integration of readouts from dynamically constructed segment states	Query-block-dependent integration of block-summary readouts
Principal advantage	Predictable complexity and multiscale coverage	Higher resolution around semantic transitions	Preserves independently distinguishable block summaries
Principal limitation	Fixed merges may cross semantic boundaries	Depends on learned boundary and merging decisions	State count and mixing cost can grow with the number of blocks

The three methods illustrate complementary approaches to temporal memory organization. Log-Linear Attention varies representation resolution according to temporal distance, DLA allocates resolution according to content change, and MHLA preserves a uniform flat partition while differentiating historical blocks during readout. These choices expose a broader trade-off among temporal resolution, bounded memory growth, segmentation adaptivity, and query-dependent retrieval. They are also potentially composable: fixed block summaries could be hierarchically merged or dynamically consolidated, while richer query-conditioned routing could be applied over the resulting bounded collection of temporal states.

5.4 Auxiliary Coordination and Routing Enhancements

The preceding subsections concern the construction and organization of associative memory itself. Memory Update methods change how information is retained, corrected, erased, or written within a recurrent state. Capacity Expansion increases the number of available state units, while Temporal Expansion preserves multiple states that summarize different historical intervals. A separate set of recent methods instead focuses on how Linear Attention computations are coordinated across broader structural axes such as feature heads and network layers. These methods do not form a single mechanism family: they are grouped here because their principal contribution lies in coordinating an identifiable Linear Attention computation rather than introducing another general state-update or state-organization rule.

Feature-head coordination. Softmax Linear Attention (SLA) introduces token-dependent competition across the feature heads of a Linear Attention layer [119]. A key-side gate controls the relative strength with which the current token is written into different head states, while a query-side gate controls the relative contribution of those heads to the current output. Unlike the mechanisms in Section 5.1, SLA does not primarily redesign the recurrence within each state; instead, it allocates write strength across a bank of existing states and controls how their completed outputs are integrated. Its direct effects therefore lie in Memory Update and Integration. Because the gates are continuous and need not exclude a head from computation, they should not automatically be interpreted as sparse Access decisions.

Cross-depth coordination. Cross-Layer Value Routing (CLVR) operates along network depth [120]. Rather than changing the number of recurrent states or the retain-correct-write rule within a layer, it exposes an internal value associated with the current write operation to subsequent layers through the residual stream. Later layers can therefore use a memory-derived signal that would otherwise remain internal to the layer that produced it.

CLVR differs from Capacity and Temporal Expansion because it does not primarily create additional persistent memory units. It also differs from the Hybrid Architectures examined in Section 7, because it preserves the host Linear Attention recurrence rather than combining heterogeneous sequence mixers or independent memory paths. Within the five-dimensional framework, CLVR is best understood as cross-layer coordination around the memory operator. It affects how memory-derived information enters later computation, but the routed value is not necessarily a completed readout in the strict sense used to define Integration in Section 2. CLVR should therefore be distinguished from both ordinary residual propagation and explicit fusion of completed memory-path outputs.

Taken together, SLA and CLVR expose two distinct coordination axes. SLA redistributes writing and output contribution across feature heads, whereas CLVR propagates a memory-derived signal across network depth. Their relationship

to the preceding categories is complementary rather than hierarchical. State-update methods determine how an individual associative state changes; Capacity and Temporal Expansion determine which memory units are maintained; and the mechanisms discussed here determine how state computations or internal memory signals are coordinated across feature partitions and network layers. This distinction also explains why these methods may affect several analytical dimensions without constituting new top-level memory substrates.

5.5 Summary

Linear Attention replaces an enumerable token-wise KV history with recurrently maintained associative memory. Its development begins with changes to Memory Representation and Memory Update, and subsequently extends to the organization, access, readout, and coordination of multiple memory units.

The evolution of state updating seeks to compress historical information more effectively within a finite associative state. The canonical additive update continually superposes new key–value associations, which can lead to interference as the state accumulates information. Retention mechanisms regulate how strongly previous content persists, delta-style correction revises the association stored at a particular address, and more differentiated erase–write mechanisms provide finer control over what is removed and committed. These developments primarily refine Memory Update while determining which historical distinctions remain preserved in Memory Representation.

As sequence length increases, state expansion relaxes the reliance on a single uniformly compressed memory. Capacity Expansion enlarges the associative space through additional dimensions, groups, or memory rows, whereas Temporal Expansion maintains multiple states that summarize different historical intervals. These changes directly enrich Memory Representation, but also introduce corresponding Access, Readout, and Integration questions: the model must determine which maintained units are eligible for the current query, how each eligible state is decoded, and how the resulting state-specific readouts are combined. The effective benefit of expansion therefore depends not only on nominal state size, but also on routing accuracy, read–write coordination, temporal resolution, and the recoverability of stored information.

Other emerging directions coordinate Linear Attention computations across broader structural axes. SLA allocates writing across feature heads and integrates their outputs, whereas CLVR propagates memory-derived signals across network depth. Both complement state updating and expansion without defining new memory substrates. Together with the capacity- and temporal-expansion mechanisms discussed above, they show that Linear Attention is developing from a compact associative operator toward a more organized memory system whose practical value depends on jointly balancing compression, capacity, temporal resolution, access cost, readout quality, and implementation efficiency.

6 State Space Models

State Space Models (SSMs) process a sequence by evolving a hidden state as each input arrives and reading that state to produce a contextual representation. Their central design problem is to preserve useful features of the input history through dynamics that can be learned and computed efficiently. Structured transitions and input-dependent control address the state-dynamics problem [25, 26, 27], while later variants enrich the read–write interfaces to the retained state [121, 122].

In the memory-centric framework of Section 2, these designs primarily shape Memory Representation and Memory Update, while later variants also make Readout more explicit. Figure 9 illustrates the basic update at one sequence position: the previous state and current input determine a new state, from which the layer reads a contextual representation. We begin with structured time-invariant dynamics and selective control, including their sequence-processing algorithms, and then examine how input–output interfaces, head organization, and update geometry extend the basic state-space layer.

6.1 State Dynamics and Selective Control

An SSM state is a set of activations carried between sequence positions. Model weights encode what is learned across training examples, whereas this state records information about the sequence currently being processed. The transition determines which state components persist and how they interact; the input and readout maps connect that state to the surrounding network. Time-invariant and input-conditioned SSMs differ in whether these operators are shared across positions or generated in response to the current input.

6.1.1 Structured Time-Invariant Dynamics

A basic discrete state-space kernel can be written as

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad r_t = Ch_t, \quad (55)$$

where $x_t \in \mathbb{R}^m$ is the current input, $h_t \in \mathbb{R}^n$ is the memory state, and $r_t \in \mathbb{R}^p$ is the contextual readout. The transition $\bar{A}$ retains and mixes existing state components, $\bar{B}$ writes the new input, and C extracts information for the current computation. The bars denote discrete-time parameters, obtained, for example, by discretizing a continuous-time system. Equation (55) isolates the state-space kernel; a complete layer can also include a direct input–output branch, nonlinearities, gates and output projections.

For a time-invariant kernel, learned operators are shared across positions and act on a continually changing sequence state. Starting from $h_0 = 0$, unrolling the recurrence gives

$$r_t = \sum_{i=1}^t C\bar{A}^{t-i}\bar{B}x_i. \quad (56)$$

Each input is written through $\bar{B}$, propagated for the elapsed number of steps, and read through C . The coefficient $C\bar{A}^{t-i}\bar{B}$ depends on the lag $t - i$, yielding a convolutional view for whole-sequence processing and a recurrent view for incremental decoding. The two views expose the same dynamics to different computational settings: training can process a sequence collectively, while decoding updates the state as new tokens arrive.

HiPPO provides an analytic foundation for constructing history-preserving dynamics [123]. It represents an input history through coefficients of a polynomial approximation under a chosen measure over the past. Intuitively, the state tracks a set of features of the historical signal, with the measure determining how different parts of the past contribute and how the coefficients evolve over time. Different measures yield dynamics with different time dependencies; here the construction provides operators and initializations for the structured SSMs that follow. The approximation viewpoint makes the role of state dimension concrete: it controls the number of coefficients available to represent the history.

LSSL introduced trainable state-space layers [124]. S4 made long-sequence computation practical through a structured transition parameterization, commonly initialized from HiPPO, while learning dynamics and input–output projections from task data [25]. In a suitable basis, S4 represents the transition using diagonal and low-rank components, providing structure for efficient kernel computation. In Equation (56), the transition participates in every lag coefficient. Its structure therefore affects both which temporal features the layer can express and how efficiently the convolution kernel can be constructed and applied. These developments turned a prescribed historical approximation into a trainable sequence-processing component: training adapts the operators, and inference applies them while updating the state for each new sequence.

S4D simplified the transition parameterization to diagonal structure [125]. In a diagonal system, each state coordinate propagates through its own transition coefficient, while the input and readout maps combine these coordinates with the network channels. S5 instead organized the layer as one multi-input multi-output system and used parallel scans [126]. These choices illustrate how temporal dynamics and the organization of the recurrent unit jointly shape an SSM layer.

A scan exploits associative composition of successive state transformations. Two updates can be composed into a transformation over their combined interval, and these interval transformations can be combined hierarchically. This permits parallel evaluation of the sequence state updates. Transition structure determines the cost of representing and composing those transformations; it is central to making the scan practical. Convolution and scan thus offer distinct sequence-processing routes, while recurrent decoding carries the current state forward one position at a time.

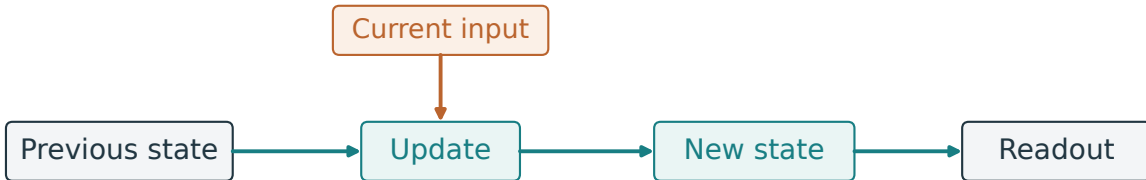


Figure 9: A state-space update at one sequence position. The new state supplies both the current readout and the memory carried to the next position.

6.1.2 Input-Conditioned State Dynamics

A time-invariant kernel applies the same temporal mixing rule to every input sequence. Language modeling often calls for content-dependent retention: an association may need to survive intervening distractors, an irrelevant token may require little writing, and a context change may call for faster replacement of old information. Selective dynamics let the input regulate these operations.

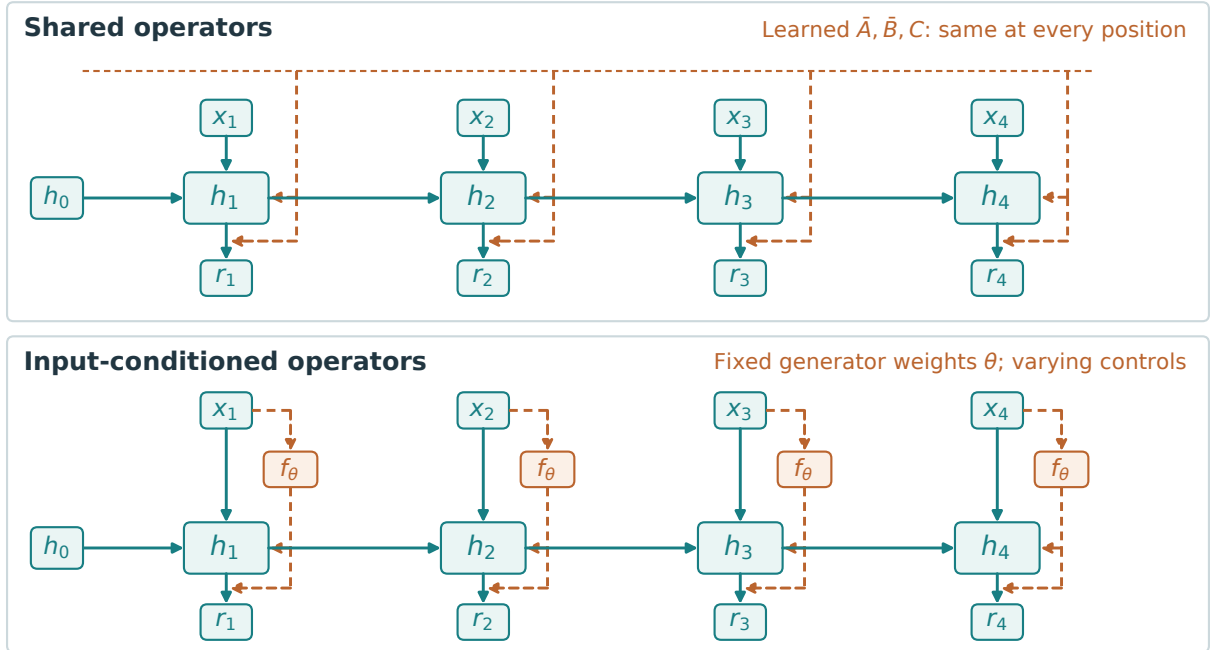
Mamba makes the step size Δ_t , input parameter B_t , and readout parameter C_t functions of the current input [26]. Its underlying state matrix A is learned during training; the effective discrete transition $\bar{A}_t = \exp(\Delta_t A)$ varies with the generated step size. The step size controls how far the dynamics advance, the input map controls writing, and the readout map modulates the contribution of state directions to the current representation. During inference, the state and generated controls vary across positions, while the weights of the networks producing those controls remain fixed. Figure 10 contrasts the resulting control paths with shared operators.

State updates with fixed or input-conditioned operators

Both lanes update state on the same input sequence, with learned parameters fixed at inference.

Solid teal: content / state

Dashed orange: control parameters



Mamba example: $(\Delta_t, B_t, C_t) = f_\theta(x_t)$; $\bar{A}_t = \exp(\Delta_t A)$.
Schematic control dependencies.

Figure 10: Shared and input-conditioned operators applied to the same input sequence. Both systems update their state at every position. The upper lane reuses learned transition, write and read operators; the lower lane generates control quantities from each input using fixed generator weights. In Mamba [26], the generated step size modulates the effective discrete transition. Solid paths carry content or state, and dashed orange paths indicate control dependencies.

Input conditioning also changes the available computation. The contribution of an earlier token now depends on controls generated along the intervening sequence, so a single lag-based convolution is insufficient for the general selective kernel. Mamba instead uses a structured selective scan [26]. Mamba-2 develops block-oriented computation through Structured State Space Duality (SSD), connecting a constrained state-space kernel to an attention-like matrix formulation [27]. This connection makes the recurrence’s structure available to efficient matrix-based sequence computation.

For a Mamba-2 head, let $H_t \in \mathbb{R}^{N \times P}$ be a matrix state with memory dimension N and value-channel width P . Writing the projected head input as $u_t \in \mathbb{R}^P$, its core recurrence and readout take the form

$$H_t = a_t H_{t-1} + b_t u_t^\top, \quad r_t = H_t^\top c_t, \quad (57)$$

where a_t is a scalar transition shared within the head, $b_t, c_t \in \mathbb{R}^N$ are write and read directions, and the write scale is absorbed into b_t .⁴ From a zero initial state,

$$r_t = \sum_{i=1}^t \underbrace{(c_t^\top b_i)}_{\text{read-write match}} \underbrace{\left(\prod_{j=i+1}^t a_j \right)}_{\text{intervening decay}} u_i. \quad (58)$$

The contribution of an earlier input factors into its match with the current read direction and the intervening transitions. The head-wise scalar transition enables this factorization, relating the SSM to the associative computation used in Linear Attention. Denote the scalar coefficient of u_i in Equation (58) by M_{ti} . These coefficients form a lower-triangular matrix: row t corresponds to a readout position and column i to a source position. On the diagonal, the transition product is empty and equals one.

Figure 11 highlights the contribution of u_2 to r_4 . The input is written along b_2 , propagated by a_3 and a_4 , and read along c_4 , giving $M_{42} = (c_4^\top b_2)a_3a_4$. Multiplying this coefficient by u_2 gives that input’s contribution to the readout; the intervening states also contain contributions from other inputs. Whole-sequence algorithms exploit the matrix structure in blocks, while incremental decoding evaluates the same kernel using only its current recurrent state.

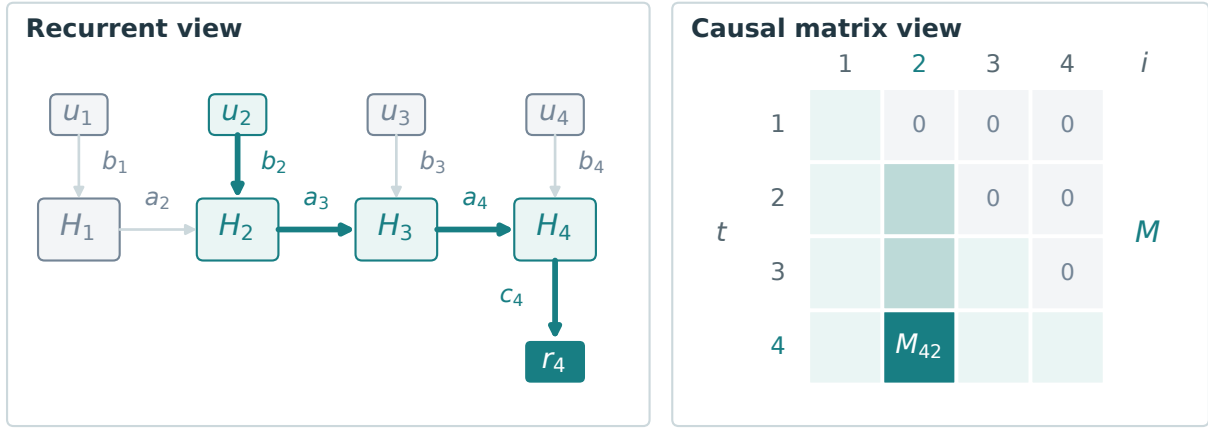


Figure 11: Two views of the SSD kernel in Equations (57)–(58) [27]. The highlighted recurrent path and matrix entry M_{42} identify the same contribution, from input u_2 to readout r_4 .

Table 11 compares the dynamics and computational interfaces of these representative kernels.

Table 11: Representative SSM dynamics and computational interfaces. The Mamba-2 SSD kernel is a selective parameterization with a head-wise scalar transition.

Kernel choice	State control	Computational interface
Learned time-invariant dynamics: S4, S4D, S5 [25, 125, 126]	Shared operators govern input writing, temporal propagation and readout.	Convolution or structured scans process sequences; recurrent decoding applies the same learned dynamics to the evolving state.
Selective dynamics: Mamba [26]	Input-generated step sizes and write/read maps regulate the effective operators.	A selective scan processes the input-dependent recurrence.
SSD kernel: Mamba-2 [27]	A head-wise scalar transition combines with input-dependent write and read directions.	Recurrent and structured-matrix views support block computation and state-based decoding.

For a fixed number of heads and fixed state dimensions N and P , the Mamba-2 kernel maintains context-length-independent state and per-step decoding work, while total sequence work grows linearly with the number of positions [27]. The $N \times P$ head state determines the retained storage, while the sequence algorithm determines how updates are scheduled and mapped to hardware. Increasing the state dimensions raises the storage and per-step arithmetic costs. The practical design therefore couples a useful state parameterization with an efficient implementation of its updates and readouts.

⁴Here inputs and readouts are column vectors. Equivalently, a row read map $C_t^{\text{row}} = c_t^\top$ gives $r_t^\top = C_t^{\text{row}} H_t$.

6.2 State Organization and Read–Write Refinements

State dynamics specify how information evolves; the surrounding interface determines what can be written and extracted at each position. H3 used associative-recall and induction-head tasks to study shortcomings of early SSMs and introduced multiplicative interactions to strengthen the relevant computation [127]. Zoology further showed that aggregate language-modeling quality can conceal substantial differences in associative recall [128]. These results motivate closer examination of the operations connecting inputs, states and queries.

6.2.1 Read–Write Interfaces

The input and output ports of the specified recurrent unit give one useful description of this interface. In Equation (55), scalar ports $m = p = 1$ define a single-input single-output (SISO) system with an n -dimensional internal state. A multi-input multi-output (MIMO) system has vector-valued ports. State size determines the internal representation, while port dimensions determine the number of input and output components connected to that representation.

S5 uses a joint MIMO system in the time-invariant setting [126]. By comparison, the P columns in the Mamba-2 kernel of Equation (57) can be read as P scalar-port recurrences sharing coefficients. Each column receives one scalar component of u_t , maintains N state values, and produces one component of r_t . The head has NP state values in total. Stating the recurrent unit at this level separates the value-channel width from both its internal memory dimension and the extra read–write directions considered next.

The MIMO variant of Mamba-3 expands the read–write channels available to the principal state [121]. Whereas Equation (57) uses one read vector c_t , a matrix $C_t^{\text{multi}} \in \mathbb{R}^{N \times R}$ supplies R directions,

$$Y_t^{\text{multi}} = H_t^\top C_t^{\text{multi}} \in \mathbb{R}^{P \times R}, \quad (59)$$

and these channels are combined into the layer output. Writing is similarly extended by combining multiple outer-product contributions. The construction increases the work performed with a principal state: each read direction projects the same retained information differently, and the resulting channels provide a richer interface to the network. For the read projection shown above, direct multiplication requires work proportional to $NP R$, exposing how the number of directions changes the arithmetic. When state movement dominates decoding time, this additional arithmetic can improve hardware utilization; realized latency depends on the implementation, device and batch size.

MIMOMamba develops a different multichannel construction by generalizing scalar SSD to matrix-valued attention [122]. Within a head, transition and input–output matrices are parameterized through a polynomial algebra generated from shared base matrices. Their commuting structure supports the associated matrix-valued factorization. Affine state-update composition supplies the associativity used by parallel scans, while the polynomial construction supplies the additional algebraic structure for this particular dual formulation. Mamba-3 MIMO and MIMOMamba thus enrich state interfaces through different choices of parameter sharing and operator structure.

Figure 12(a,b) contrasts increasing the state dimensions with adding read–write directions to the same principal state. Together, the interfaces above determine the calculations performed on the retained state and the channels through which its information reaches the layer output. Their value depends on the information preserved by the dynamics: a richer projection exposes additional views of the state, while the transition and writing process determine the historical distinctions encoded there.

6.2.2 State Organization and Update Geometry

The layer can further organize temporal processing across heads. HADES interprets Mamba-2’s multi-head recurrences as an adaptive filter bank [129]. It combines shared filters for global low-pass behavior with expert filters for local high-pass behavior, as sketched in Figure 12(c). The shared component carries more slowly varying information; expert components emphasize more rapidly changing contributions. This organizes complementary temporal characteristics within the SSM layer, alongside the choice of read–write interfaces.

The geometry of the injected updates provides another way to improve how the state evolves. MuonSSM augments an SSM with a momentum-based pathway and lightweight Newton–Schulz iterations on low-rank input injections [130]. The momentum pathway carries information across updates, while the orthogonalization procedure conditions their directional structure. Figure 12(d) illustrates this change in the geometry of input injections. In the state-space recurrence, this intervention targets the contribution written into memory, keeping the learned temporal transition as a separate component. It therefore adds update processing to the choices of retention and input–output mapping already described.

These designs operate at different locations in the state computation. Head organization distributes temporal characteristics across interacting components; update conditioning shapes the signal entering the state. This distinction helps

explain how an SSM layer can be refined while retaining recurrent execution. The transition’s expressiveness remains important as well: state-tracking analyses identify limitations of particular transition classes under specified architectural and numerical assumptions [131]. Examining the transition, interface and update together is consequently useful for understanding the behavior of a concrete SSM.

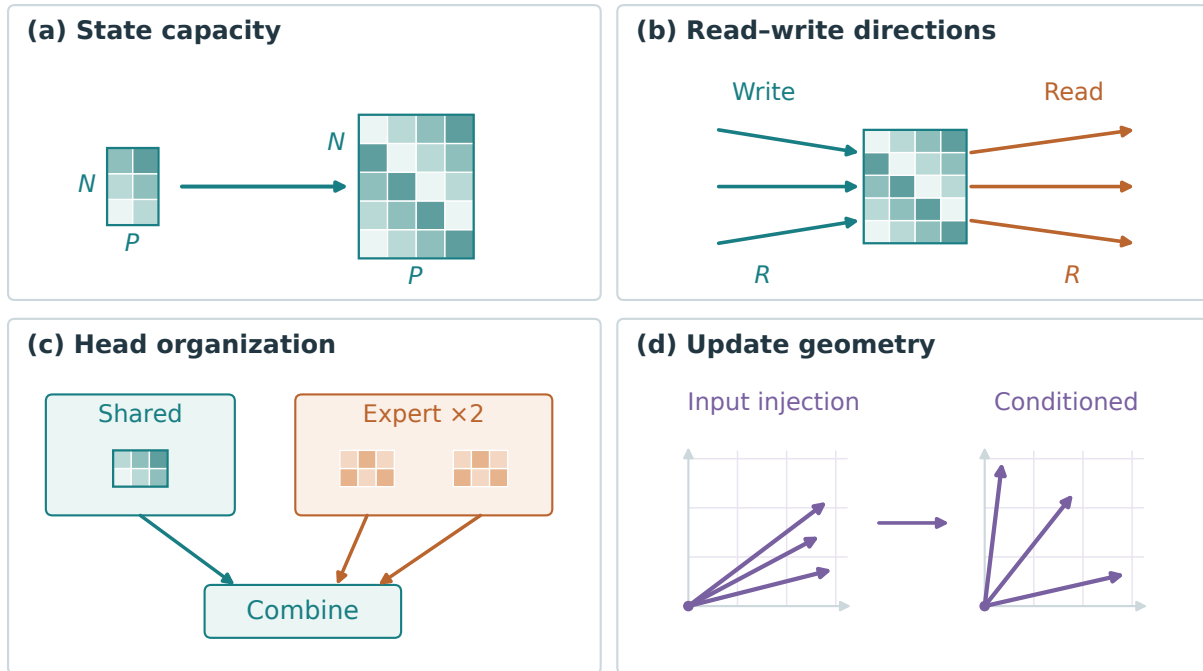


Figure 12: Four ways to refine an SSM layer: (a) enlarge the state, (b) add read–write directions, (c) combine shared and expert temporal components as in HADES [129], and (d) condition input-injection directions as in MuonSSM [130]. Panels (c,d) are conceptual sketches of the mechanisms described in the text.

6.3 Summary

SSM sequence layers combine a retained state with learnable temporal dynamics. Structured time-invariant models provide tractable ways to represent and propagate historical features. Selective models let input-generated controls modulate state evolution, writing and reading, and SSD connects a constrained selective kernel to matrix-based computation. Subsequent designs enrich the channels connected to the state, organize its temporal processing across heads, and condition the geometry of incoming updates. Table 12 summarizes these SSM-specific choices and their computational roles.

These mechanisms also supply components for hybrid architectures. Attamba uses SSMs to form block-level keys and values [132], while DART retains local SSM block-state contributions for query-time key and value generation [133]. Samba combines Mamba with sliding-window attention [134]. The following chapter examines how such state-based components are coordinated with other sequence-processing paths.

Table 12: SSM design questions, mechanisms and roles. The fixed-size decoding state assumes a fixed head count and fixed state dimensions.

Design question	SSM mechanism	Computational or state effect
How should the state propagate and adapt?	Structured dynamics and input-dependent controls.	Temporal propagation, selective writing and readout.
How should the recurrence be computed?	SSD recurrent and matrix views; block computation.	Structured sequence computation and a fixed-size decoding state.
How should the state communicate?	Multichannel interfaces: S5, Mamba-3 MIMO and MIMOMamba.	Richer input–output interaction with the retained state.
How should heads and updates be organized?	HADES head organization; MuonSSM update geometry.	Complementary temporal roles; conditioned input-injection directions.

7 Hybrid Architecture: Composing Heterogeneous Memory Mechanisms

This section takes composition granularity as its organizing principle and examines how heterogeneous attention and state-based memory mechanisms are combined at the layer, head, branch, and token levels. Hybrid Architecture is treated as a compositional design space rather than as a fifth memory operator parallel to Softmax Attention, Sparse Attention, Linear Attention, and State Space Models. At a coarse granularity, a hybrid may arrange largely self-contained modules across network depth; at finer granularities, it may redesign the internal organization of heads, branches, token routing, memory transitions, and fusion interfaces. Accordingly, this section focuses on where heterogeneous mechanisms are combined and how their interactions change across different structural granularities.

Softmax Attention, Sparse Attention, Linear Attention, and State Space Models provide complementary capability–cost profiles. Explicit attention preserves direct token-level addressability, Sparse Attention restricts the candidate set, and Linear Attention and SSMs propagate history through bounded recurrent states. Hybrid designs combine these capabilities so that fine-grained retrieval, local interaction, sparse access, and efficient long-range propagation need not be provided by one sequence mixer.

We organize Hybrid Architectures by the structural granularity at which heterogeneous mechanisms are combined. *Layer-wise Hybrid* assigns different mixers across network depth; *Head-wise Hybrid* partitions mechanisms across heads or channel groups within a layer; *Branch-wise Hybrid* maintains and coordinates multiple memory paths inside a block; and *Token-wise Hybrid* assigns tokens or chunks to different memory representations or sequence-mixing operations. These granularities may coexist within one architecture and together extend contextual memory from a single substrate or processing path to a heterogeneous organization of explicit and recurrent memories. Under the five-dimensional view in Section 2, such composition first broadens Memory Representation and, depending on how tightly the paths are coupled, may further coordinate how information is updated or transferred, which memory interfaces are exposed, how their contents are read, and how the resulting features are integrated. Composition granularity therefore identifies the structural scope of hybridization, while the five dimensions describe its consequences for memory processing.

Figure 13 summarizes where heterogeneous mechanisms are combined and qualitatively relates each composition granularity to its typical intervention scope under the five-dimensional view. The following subsections compare Layer-wise designs by layer-allocation policy, Head-wise designs by head budget, Branch-wise designs by interaction interface, and Token-wise designs by routing signal. Tables 13–16 then report the corresponding categories, periods, and representative methods before the text examines their motivations and implementations.

7.1 Layer-wise Hybrid

Layer-wise Hybrid assigns different sequence mixers to different depths of a network, combining mechanisms such as Softmax or Linear Attention, SSMs, local attention, and Sparse Attention within a single backbone. The principal distinction is the source of the layer-allocation decision. Some architectures determine module positions before training through a fixed ratio or periodic schedule. Others begin with a pretrained model and use layer behavior to decide which attention modules should be retained or replaced. In the latter group, the selection signal may come directly from diagnosis of the original layers, from representation differences after a candidate replacement module has been aligned, or from teacher–student discrepancy during iterative distillation. Table 13 therefore distinguishes Predefined Allocation, Direct Diagnosis Selection, Alignment-Based Selection, and Iterative Distillation-Guided Selection.

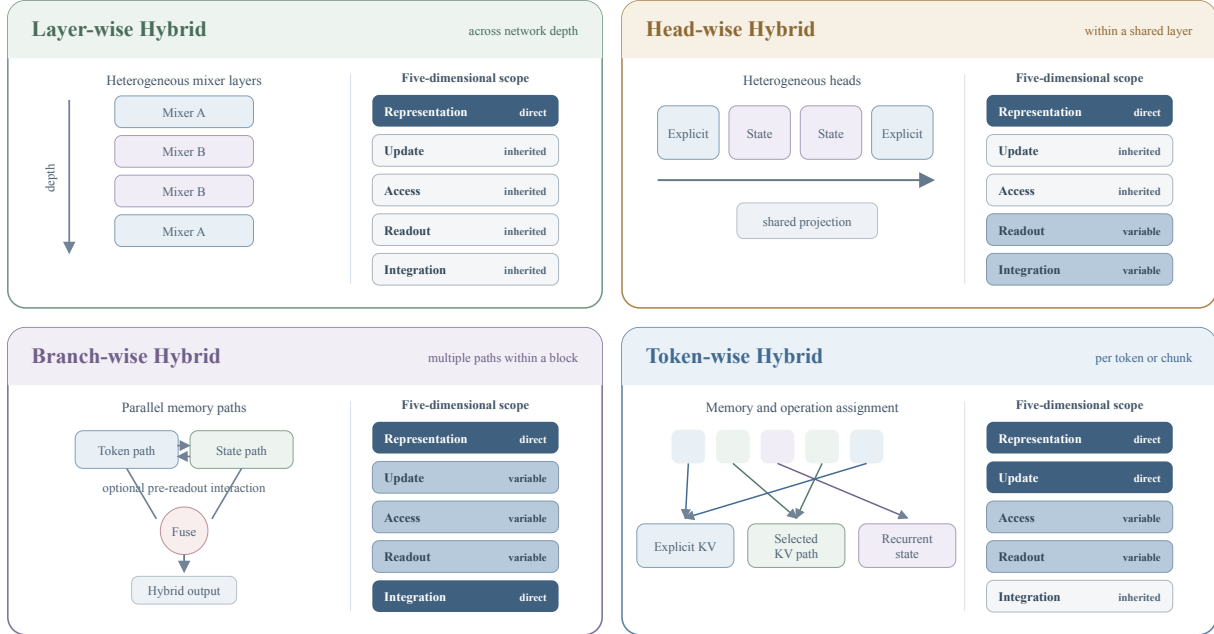


Figure 13: Four composition granularities of Hybrid Architecture and their typical intervention scopes under the five-dimensional memory-centric view. The schematics show where heterogeneous mechanisms are combined. The five-cell columns correspond to Memory Representation, Memory Update, Access, Readout, and Integration. Dark, medium, and light blocks denote dimensions commonly modified directly by hybrid composition, affected in a method-dependent manner, or primarily inherited from the constituent mechanisms, respectively. The profiles are qualitative summaries rather than quantitative literature counts.

Table 13: Layer-wise Hybrid designs grouped by layer-allocation policy.

Layer allocation	Training strategy	Period	Representative methods
Predefined allocation	From-scratch training	2024–2026	Griffin [135]; Jamba [28]; Samba [134]; Zamba [136]; Zamba2 [137]; HySparse [102]; HySparse2 [103]
Direct diagnosis selection	Pretrained-model transfer	2024–2026	LightTransfer [138]; Priming [139]
Alignment-based selection	Pretrained-model transfer	2025–2026	Jet-Nemotron / PostNAS [140]; HypeNet / HALO [141]
Iterative distillation-guided selection	Pretrained-model transfer	2025	KL-Guided Layer Selection [142]

7.1.1 Predefined Allocation

Many native Hybrid models use manually specified layer structures. Griffin [135] configures gated linear recurrent blocks and local-attention blocks at an approximate ratio of 2:1, whereas Jamba [28] uses Mamba as its dominant path and retains full-attention layers at an approximate ratio of 1:7 relative to Mamba layers. Samba likewise adopts a predetermined layer-wise arrangement of Mamba, sliding-window attention, and MLP blocks, using recurrent propagation as the backbone while periodically providing direct local retrieval [134]. These designs control where different context-processing capabilities appear across depth without fusing their readouts inside one block.

Zamba and Zamba2 periodically invoke attention modules that are shared across depth within a Mamba backbone. Zamba repeatedly uses one shared attention block. Zamba2 retains parameter sharing but alternates between two shared attention blocks and adds LoRA projection modules to provide lightweight depth-specific adaptation at different invocation sites [136, 137]. HySparse similarly organizes the model with a fixed full/sparse layer ratio. Within its sparse layers, however, it computes global sparse attention and local sliding-window attention in parallel, while reusing the KV representations and selection indices of a preceding full-attention layer [102]. HySparse2 keeps this predefined organization but partitions the backbone into a self-decoder that interleaves full attention with sliding-

window attention and a cross-decoder that interleaves full attention with token-level sparse attention, connecting the two through full-attention KV bridging so that each cross-decoder full-attention layer projects its keys and values from self-decoder hidden states [103]. Its detailed selection and cross-layer reuse mechanism is treated in Section 4.4.

7.1.2 Direct Diagnosis Selection

Direct diagnosis methods estimate the replaceability of attention layers in a pretrained model without performing complete replacement-module alignment and repeated distillation before every selection decision. LightTransfer identifies “lazy” layers that depend less on global attention and prioritizes their conversion to streaming or local attention [138]. Priming uses a block-importance score to select Transformer layers that should be retained and applies spectral initialization when converting the remaining attention blocks into SSM modules [139]. Both derive their selection signal directly from properties of the original model, but use different migration procedures: lightweight adaptation in LightTransfer and training-free conversion in Priming.

7.1.3 Alignment-Based Selection

Alignment-based methods first reduce the representation gap between Softmax Attention and a candidate efficient module, and then evaluate the effect of replacement at different depths. Jet-Nemotron performs hidden-state alignment before PostNAS jointly searches full-attention placement, the efficient attention operator, and hardware-aware hyperparameters; the resulting Hybrid model is subsequently fine-tuned [140]. HypeNet/HALO performs attention-weight transfer and hidden-state alignment before selecting the full-attention layers to retain, followed by distillation and fine-tuning to recover the converted model’s capability [141]. Both follow an alignment–selection–adaptation sequence, differing mainly in the search space and subsequent optimization procedure.

7.1.4 Iterative Distillation-Guided Selection

KL-Guided Layer Selection alternates student adaptation with layer selection. It first performs attention-weight transfer and hidden-state alignment, and distills an initial Linear Attention student. Softmax Attention is then restored one layer at a time, with the resulting reduction in teacher–student KL divergence used to estimate that layer’s importance; distillation continues after each selection round [142]. The selection signal therefore reflects the behavior of an already adapted student rather than statistics of the original model or a one-time alignment result.

This progression moves Layer-wise Hybrid design from uniform periodic schedules specified during architecture construction toward layer-specific allocation informed by pretrained-model behavior. In parallel, model construction expands beyond training from scratch to adaptation, distillation, fine-tuning, and training-free conversion. These training strategies support the allocation decision, but remain distinct from the architectural question of which mechanism is deployed at each depth and from the memory-level Access rule used inside that mechanism.

7.2 Head-wise Hybrid

Head-wise Hybrid assigns different heads or channel groups within the same layer to distinct sequence mixers, allowing full-context retrieval, streaming or sparse token interaction, and state-based propagation to coexist at every depth. Its central question is how the head budget is apportioned among these mechanisms and whether the assignment is predefined, selected from head function, adapted to the current input, or varied across network depth. Table 14 groups existing methods into Fixed Head Allocation, Function-Aware Head Selection, Input-Adaptive Head Routing, and Depth-Adaptive Head Allocation.

7.2.1 Fixed Head Allocation

Fixed-allocation methods specify the ratio of the two head types during architecture design and keep it identical or approximately identical across the Hybrid layers of a given configuration. Hymba places attention and SSM heads in parallel within a symmetrically formulated fusion module, while its final configurations use an SSM-dominant allocation in which attention heads occupy no more than approximately one fifth of the Mamba heads [29]. Falcon-H1 uses a more SSM-heavy asymmetric allocation; in its 7B configuration, the attention-to-SSM head budget is approximately 1:2 [143]. Both are native Head-wise Hybrid architectures rather than methods that choose particular positions from a pretrained set of full-attention heads.

7.2.2 Function-Aware Head Selection

DuoAttention separates pretrained attention heads according to their long-context functions. It learns a scalar gate for each head by minimizing the output deviation between full attention and a gated mixture of full and streaming

Table 14: Head-wise Hybrid designs grouped by the policy used to determine the head allocation.

Head allocation	Period	Allocation characteristic	Representative methods
Fixed head allocation	2024–2025	A predefined ratio is used across Hybrid layers.	Hymba [29]; Falcon-H1 [143]
Function-aware head selection	2024–2026	Heads with retrieval or long-range functions retain full attention, while the remaining heads use streaming, sparse, or recurrent paths.	DuoAttention [144]; HydraHead [145]
Input-adaptive head routing	2026	A test-time router assigns heads to full-attention or sparse-attention modes according to the input, allowing the overall sparsity ratio to vary dynamically.	Elastic Attention [146]
Depth-adaptive head allocation	2026	The hybrid ratio is varied across depth.	Head-wise Hybridization [147]

attention, and then binarizes the gates for deployment. Retrieval heads retain full attention and a complete KV cache, whereas streaming heads retain only attention sinks and recent tokens [144]. DuoAttention therefore combines full-context and structurally sparse Softmax Attention within each layer, using an optimization-based estimate of which heads require global retrieval.

HydraHead extends function-aware selection across different sequence-mixing families. It analyzes the roles of attention heads in retrieval and long-range information processing, preserves the head positions whose functions depend more strongly on full attention, and converts the remaining heads to GDN paths [145]. The resulting architecture can retain a small aggregate full-attention budget—for example, approximately one full-attention head for every seven GDN heads—but this ratio describes overall resource allocation. Both DuoAttention and HydraHead produce a fixed head assignment after the identification or conversion stage; they differ in whether the efficient heads use streaming attention or a recurrent GDN mechanism.

7.2.3 Input-Adaptive Head Routing

Elastic Attention combines full attention and streaming sparse attention within the same layer, but does not fix one head assignment for every input. A lightweight attention router uses the current sequence representation to assign individual heads to full-attention or sparse-attention computation modes at test time, allowing the model-level sparsity ratio to adapt to the input’s sensitivity to sparse retrieval [146]. This distinguishes Elastic Attention from offline function-aware selection: DuoAttention and HydraHead identify a deployment topology before inference, whereas Elastic Attention changes the active full/sparse head allocation across inputs.

7.2.4 Depth-Adaptive Head Allocation

Head-wise Hybridization further allows the full-attention/GDN budget to vary with depth. For layer ℓ ,

$$H_\ell^{\text{FA}} + H_\ell^{\text{Eff}} = H, \quad H_\ell^{\text{Eff}} : H_\ell^{\text{FA}} = k_\ell : 1, \quad (60)$$

where k_ℓ changes across the network so that each depth receives a different amount of exact token-retrieval capacity according to its functional requirements [147]. Function-aware selection asks which head positions should preserve full attention, input-adaptive routing asks which computation mode each head should use for the current input, and depth-adaptive allocation asks how much full-attention capacity should be assigned at each depth.

7.3 Branch-wise Hybrid

Branch-wise Hybrid maintains multiple comparatively complete information streams with distinct Memory Representations, Updates, or Readout rules inside the same block. Head-wise Hybrid instead partitions heads or channel groups within a common multi-head structure and typically concatenates their readouts before a shared output projection. We distinguish Branch-wise designs by whether the paths are combined after producing contextual readouts $r_t^{(p)}$ or after independently forming path outputs $o_t^{(p)}$. The fusion operator alone does not determine the category; the defining property is that the architecture preserves identifiable memory paths rather than partitioning head or channel capacity within one shared module. Table 15 summarizes the two interfaces.

Table 15: Branch-wise Hybrid designs grouped by the interface at which heterogeneous memory paths are combined.

Fusion interface	Unified form	Period	Representative methods
Readout-level branch fusion	$r_t^{\text{hyb}} = \text{Fuse}_r(\{r_t^{(p)}\})$	2022–2026	Block-Recurrent Transformer [148]; Memorizing Transformer [149]; Infini-attention [30]; DART [133]
Output-level branch fusion	$o_t^{\text{hyb}} = \text{Fuse}_o(\{o_t^{(p)}\})$	2025	Titans-MAG [150]

7.3.1 Readout-Level Branch Fusion

Readout-level designs combine contextual representations produced by distinct paths before a shared output transformation:

$$r_t^{\text{hyb}} = \text{Fuse}_r(\{r_t^{(p)}\}_{p \in \mathcal{P}_t}; x_t), \quad o_t^{\text{hyb}} = \text{Integration}(r_t^{\text{hyb}}; x_t). \quad (61)$$

The methods in this category differ in the memories read by each path, the degree of cross-path dependence, and the fusion rule applied to their readouts.

Block-Recurrent Transformer maintains a token stream and a fixed set of recurrent state vectors. In the token direction, token self-attention and token-to-state cross-attention produce parallel readouts that are concatenated before a shared projection. The state direction mirrors this structure: state self-attention and state-to-token cross-attention are combined before gated recurrent updating. It therefore performs readout-level fusion in both directions while jointly producing the current token representations and the recurrent state for the next block [148]. Unlike the associative matrix used by standard Linear Attention, its recurrent memory consists of multiple explicit state vectors that are read through ordinary attention.

Memorizing Transformer provides a boundary case for the same interface. It separately reads the local KV context and a top- k external kNN memory, then combines the two per-head attention results through a learned gate before the usual multi-head output transformation [149]. Although the external datastore lies outside the model-internal memory emphasized by the core taxonomy, its gated readout interface is directly comparable to internal branch fusion.

Infini-attention implements the same ordering with model-internal memories. For each head, it first produces a local causal attention context A_{dot} and a compressive-memory readout A_{mem} , and then computes

$$A = \sigma(\beta) \odot A_{\text{mem}} + (1 - \sigma(\beta)) \odot A_{\text{dot}}. \quad (62)$$

The fused head contexts are subsequently concatenated and projected by the shared multi-head output matrix, making this a clear readout-level fusion [30].

DART couples its paths more tightly because the State-Memory Attention (SMA) branch retrieves from chunk-state memories produced by the Mamba-2 scan and shares the native SSM read vector. Its final combination nevertheless remains at the readout interface: the SMA readout R_t is added as a scalar-gated residual correction to the SSM readout $C_t H_t$,

$$r_t^{\text{DART}} = C_t H_t + G_t R_t, \quad G_t = \text{SiLU}(U_t W_G). \quad (63)$$

The resulting representation then continues through the remaining block transformations [133]. Thus, Block-Recurrent Transformer, Memorizing Transformer, Infini-attention, and DART use different memory substrates and fusion operators, but all combine path readouts before the enclosing module completes its output transformation.

7.3.2 Output-Level Branch Fusion

When each branch first forms its own output, Hybrid-level Integration instead acts on $\{o_t^{(p)}\}$:

$$o_t^{\text{hyb}} = \text{Fuse}_o(\{o_t^{(p)}\}_{p \in \mathcal{P}_t}; x_t). \quad (64)$$

Titans-MAG follows this pattern. Its short-term branch applies sliding-window attention, while its neural long-term memory branch independently produces a memory output from the same input. The two branch outputs are normalized with learned vector-valued weights and combined through a nonlinear gate,

$$o_t^{\text{MAG}} = \text{Gate}(o_t^{\text{SWA}}, o_t^{\text{mem}}), \quad (65)$$

so the fusion directly defines the output of the Hybrid module rather than an intermediate readout awaiting a shared output projection [150].

7.4 Token-wise Hybrid

Token-wise Hybrid uses token position, a content score, or a learned policy to decide which memory or sequence-mixing path receives a token. Let z_i denote this architecture-level routing decision for token i . Depending on the method, the token may remain in recent KV memory, move to sparse historical KV, be compressed into a recurrent state, or be assigned to a different sequence-mixing operation. The routing decision changes memory-level Access only when it changes which represented memory units or state interface are exposed to the current query; routing that merely selects an operator remains mechanism allocation. Table 16 groups the methods by routing signal into Temporal-Boundary Allocation, Content-Score Allocation, and Learned Operation Allocation.

Table 16: Token-wise Hybrid designs grouped by the signal used to determine token allocation.

Token allocation	Unified selection rule	Period	Representative methods
Temporal-boundary allocation	$z_i = f_{\text{time}}(t - i, w)$: deterministic allocation by token age, position, or local-window boundary	2024–2025	LoLCATs [151]; Native Hybrid Attention [31]
Content-score allocation	$z_i \in \{KV_{\text{recent}}, KV_{\text{salient}}, S_{\text{compressed}}\}$: recent tokens remain local; departing tokens enter salient historical KV or a compressed state according to self-recall error or self-saliency	2025–2026	LoLA [152]; STILL [153]
Learned operation allocation	$z_i = \arg \max_{o \in \mathcal{O}} p_{\theta}(o x_i, \mathcal{H}_i)$: a learned or search-derived policy assigns a Softmax or Linear/Recurrent operation	2026	NAtS-L [154]

7.4.1 Temporal-Boundary Allocation

Temporal-boundary methods make a deterministic allocation according to token age, position, or a local-window boundary. LoLCATs applies exact Softmax Attention to the most recent w tokens and accumulates tokens that leave the window into a Linear Attention state; local tokens and the long-term state are then read through a jointly normalized expression [151]. Native Hybrid Attention similarly preserves exact KV representations for recent tokens, updates older history into a fixed number of long-term slots, and applies one Softmax readout over the local KV and the slots [31]. Both use a temporal boundary to determine memory lifecycle, but represent remote history with an associative matrix state and gated slots, respectively.

7.4.2 Content-Score Allocation

Content-score methods further distinguish which tokens leaving the local window should remain explicitly addressable. Their allocation can be summarized as

$$z_i \in \{KV_{\text{recent}}, KV_{\text{salient}}, S_{\text{compressed}}\}. \quad (66)$$

All recent tokens first remain in local KV memory. When a token leaves the window, LoLA uses self-recall error to determine whether it should enter sparse historical KV or be compressed into a Linear Attention state [152]. STILL performs a similar division using self-saliency: high-saliency history remains accessible through Sparse Attention, while the remaining information is written into a linear state [153]. These designs extend a fixed temporal lifecycle into content-dependent allocation of memory fidelity.

7.4.3 Learned Operation Allocation

NAtS-L does not only decide how a token should be stored after it leaves a local window. It searches, at token or chunk granularity, whether the unit should use a Softmax Attention or GDN operation:

$$z_i = \arg \max_{o \in \mathcal{O}} p_{\theta}(o | x_i, \mathcal{H}_i), \quad \mathcal{O} = \{\text{Softmax}, \text{GDN}\}. \quad (67)$$

Tokens assigned to the Softmax operation retain explicit KV representations, whereas those assigned to GDN propagate through a recurrent state [154]. Routing thus progresses from a predetermined time boundary and hand-designed content score to learned or search-derived assignment of sequence-mixing operations.

7.5 Summary

The preceding comparison shifts the question of efficient sequence modeling from whether one mechanism can fully replace Softmax Attention to how heterogeneous mechanisms can divide and coordinate contextual processing. Hybrid Architecture is therefore not an additional sequence mixer, but a compositional design paradigm in which explicit token

retrieval, sparse access, local interaction, and recurrent propagation can be assigned to different structural units within the same model.

Hybrid designs distribute complementary capabilities across progressively finer and more tightly coordinated granularities. Layer-wise methods arrange largely self-contained mixers across network depth. Head-wise methods place heterogeneous retrieval and state-propagation capabilities within a shared layer, Branch-wise methods coordinate comparatively complete memory paths at the readout or output interface, and Token-wise methods determine how individual tokens or chunks are represented or processed. These granularities are composable rather than mutually exclusive, allowing one architecture to coordinate complementary mechanisms across several structural axes.

The central technical development is a shift from module coexistence toward deeper coordination of heterogeneous memory paths. Coarse Layer-wise composition primarily controls where each mechanism is used, whereas finer-grained designs can additionally couple memory representations, lifecycle decisions, readout rules, and output formation. Head-wise allocation divides representational and retrieval capacity inside a layer; Branch-wise fusion combines multiple contextual readouts or independently formed path outputs; and Token-wise allocation determines whether particular information remains explicitly addressable or is incorporated into a compressed recurrent state. The scope of complementarity therefore expands from arranging separate modules to jointly organizing how heterogeneous memories are formed, accessed, and combined.

8 Attention Designs in Publicly Documented LLM Architectures: Evolution, Coordination, and Frontier Adoption

The preceding sections examine attention and related sequence-mixing mechanisms at the method level. This section turns to their use in publicly documented autoregressive language-model backbones and asks three architecture-level questions: whether attention design is converging toward one dominant form, how heterogeneous mechanisms are combined within individual models, and which attention structures are represented among high-performing open-weight models.

Our first evidence set is a longitudinal inventory of 59 release-level architecture records spanning 14 major model lineages from 2022 through September 22, 2026. Models released together are grouped when they share the same language-model attention backbone, whereas separately released versions remain separate records; a documented change in the attention backbone also defines a separate record. The inventory is based primarily on official papers, technical reports, model cards, and released configurations. Although several included releases are natively multimodal, the unit of analysis remains the autoregressive language-model backbone. We include such models only when their causal language backbone is publicly documented and relevant to the evolution of attention design, and exclude vision or audio encoders, modality projectors, cross-modal interfaces, and other modality-specific components from the classification. The chapter therefore remains an analysis of LLM attention architectures rather than a survey of multimodal architectures. The complete inventory and source mapping are provided in Supplementary Table S1.

Our second evidence set is a frozen comparison of high-performing open-weight models from the Artificial Analysis Intelligence Index v4.3.2, captured on September 22, 2026 [155, 156, 157]. It provides a cross-sectional view of attention designs near the open-weight performance frontier, while the longitudinal inventory characterizes architectural development over time. Both analyses are descriptive: the inventory is purposively curated rather than exhaustive or market-share weighted, and leaderboard scores depend on model scale, training, post-training, reasoning budget, and systems implementation in addition to attention design. We therefore use these evidence sets to characterize adoption and coexistence, not to infer that a particular attention mechanism causes higher model quality.

Figure 14 visualizes the 58 classifiable records in the longitudinal inventory; Kimi k1.5 is omitted because its attention architecture is not separately disclosed. Family-level records with distinct scale-specific variants may appear as multiple nodes, so the figure is not a one-to-one count of release records. Node fills encode attention composition, outlines summarize context-length tiers, and labels are limited to representative releases to preserve readability.

8.1 Diversification Rather Than Convergence in Attention Design

The longitudinal inventory shows diversification rather than convergence on one attention type. Dense MHA evolved toward MQA and GQA to reduce KV-head redundancy while preserving full token-level retrieval, and GQA remains a widely used backbone in recent Llama [56], Qwen [58], Mistral [158], MiniMax [159], MiniCPM [160], and K2-Horizon releases [161]. In parallel, MLA [5, 162, 163] compresses per-token KV representations, local and sparse attention [64, 16] restrict explicit access, and recurrent mechanisms such as Gated DeltaNet [164], Lightning Attention [165], KDA [66], and Mamba [166] replace part of repeated token retrieval with fixed-size state. These approaches



Figure 14: Attention designs across the classifiable architecture inventory. For natively multimodal systems, only the autoregressive language-model backbone is classified. Fill colors encode attention composition, split fills denote heterogeneous designs, and outlines indicate broad context-length tiers. The increasing concentration of split-fill nodes in recent years indicates the growing use of Hybrid architectures; the figure is descriptive and does not imply performance rankings or causal relationships.

reduce different costs and retain different memory interfaces, so they continue to coexist rather than forming a single replacement sequence.

Model lineages also change direction rather than following a monotonic progression. DeepSeek develops from MLA to MLA-based sparse retrieval and then to CSA/HCA and CSA2 [5, 64, 50, 101]; Qwen moves from MHA and GQA to recurrent-state architectures combined with full attention in Qwen3-Next, Qwen3.5, and Qwen3.6, and with sparse attention in Qwen3.8-Flash-Next [167, 59, 164, 60, 61, 62, 13]; MiniMax moves from Lightning/Softmax hybrids to full GQA and then to dense/sparse attention [165, 159, 95]; and LongCat moves from MLA in LongCat-Flash and LongCat-Flash-Lite to LSA in LongCat-2.0 and LongCat-Flash-Lite-Sparse [162, 163, 16, 104]. The resulting landscape is therefore best understood as several persistent design routes—shared-KV dense attention, latent KV compression, restricted explicit access, and recurrent-state propagation—that are increasingly recombined in different models.

8.2 Increasing Hybridization and Emerging Cross-Layer Reuse

Hybrid composition becomes more common in the recent inventory, primarily through predetermined layer-wise schedules. Of the 59 release-level records, 58 disclose enough information for classification: 32 retain one principal attention or memory family, while 26 are classified as Hybrid. These are release-level records rather than 26 distinct architectural patterns. Table 17 shows that Hybrid records increase from none in 2022–2023 to 9 of 25 in 2024–2025 and 17 of 27 in 2026. The dominant forms are local/global explicit-attention schedules and recurrent-state/explicit-attention schedules, in which complementary capabilities are assigned to different layers and coordinated through the hidden-state stream.

Cross-layer reuse introduces a distinct form of coordination by extending the depth-wise lifetime of memory and routing artifacts. Across the documented cases, GLM-5.2 and GLM-5.3 reuse retrieval indices or top- k candidate decisions through IndexShare/IndexCache [68, 69, 15]; DeepSeek-V4.1-Flash additionally reuses selected global KV representations and indexer keys through the Full, Reindex, and Reuse modes of CSA2 [101]; and LongCat-2.0 and LongCat-Flash-Lite-Sparse use LSA’s Cross-Layer Indexing to reuse one layer’s selected token set across subsequent layers [16, 104]. Hybrid composition describes where complementary mechanisms are placed; cross-layer reuse describes whether artifacts produced at one depth remain available to later layers. The latter is currently documented in five records, but it extends coordination from module placement to the depth-wise persistence of memory and routing artifacts.

Table 17: Temporal distribution of Single-family, Hybrid, and cross-layer-coordinated architectures in the publicly documented model inventory. Single-family and Hybrid are mutually exclusive labels; cross-layer coordination is an overlapping attribute.

Period	Records	Single-family	Hybrid	Cross-layer
2022–2023	6	6 (100%)	0	0
2024–2025	25	16 (64%)	9 (36%)	0
2026*	27	10 (37%)	17 (63%)	5 (19%)

*Records available through September 22, 2026. Percentages use the classifiable records within each period as the denominator. Kimi k1.5 is excluded because its attention architecture is not separately documented. Cross-layer coordination should not be added to the Single-family and Hybrid columns. The five cross-layer records are GLM-5.2, GLM-5.3, DeepSeek-V4.1-Flash, LongCat-2.0, and LongCat-Flash-Lite-Sparse; the two GLM releases and the two LongCat releases each share a base architectural pattern. DeepSeek-V4.1-Flash is counted as Hybrid in the mutually exclusive Single-family/Hybrid classification and also carries the overlapping cross-layer attribute.

8.3 Attention Designs among High-Performing Open-Weight Models

The frozen Artificial Analysis comparison provides a cross-sectional view of the attention structures represented near the open-weight performance frontier. Table 18 reports the eleven high-performing open-weight endpoints retained in the September 22, 2026 snapshot. Reasoning settings such as “max” and “xhigh” are treated as endpoint configurations rather than separate architectures. The endpoint selection follows the Artificial Analysis Intelligence Index v4.3.2 snapshot, while release dates, parameter scales, and attention structures are based on the corresponding public model documentation and the source mapping in Supplementary Table S1 [155, 157].

Table 18: Attention structures of the eleven selected high-performing open-weight model endpoints in the Artificial Analysis snapshot captured on September 22, 2026. Rows are ordered by release date in reverse chronological order. Model size reports total parameters; disclosed active parameters are shown in parentheses [157].

Model endpoint	Release	Model size	Attention structure
MiMo-V2.6-Pro	2026-09	1.02T (42B active)	60 SWA-GQA + 10 global-GQA layers [168]
DeepSeek-V4.1-Flash (max)	2026-09	552B (8B active prefill; 16B active decode)	CSA2 with SWA; cross-layer KV/index/top- <i>k</i> reuse
K2-Horizon-375B-A23B	2026-09	375B (23B active)	full GQA
GLM-5.3-Flash	2026-08	320B (18B active)	34 KDA + 11 compressed-indexer DSA layers
GLM-5.3 (max)	2026-08	744B (40B active)	MLA-based DSA; IndexShare/IndexCache
Qwen3.8-Flash-Next	2026-08	176B; 125B main (6B active)	3 GDN : 1 QSA
DeepSeek-V4 Pro 0813 (max)	2026-08	1.6T (49B active)	CSA/HCA layer-wise hybrid
Qwen3.8-2.4T-A95B	2026-08	2.4T (95B active)	3 Gated DeltaNet : 1 gated full-GQA layer
Qwen3.8-27B (xhigh)	2026-08	27B	3 Gated DeltaNet : 1 gated full-GQA layer [169]
Kimi K3 (max)	2026-07	2.8T (104B active)	3 KDA : 1 Gated MLA
MiniMax-M3	2026-06	428B (23B active)	3 dense-attention + 57 MSA layers

The frontier comparison remains architecturally heterogeneous. The selected models include full GQA, latent-compressed sparse attention, block-sparse attention, local/global or compressed dense/sparse hybrids, and recurrent-state layers combined with full, latent, or sparse explicit retrieval. Hybrid structures are prominent but not universal: K2-Horizon retains full GQA and GLM-5.3 uses a single DSA family, while the other endpoints combine distinct access or state-propagation routes. At the same time, every selected architecture retains an explicit token-retrieval path, indicating that recurrent state and sparsity currently complement rather than uniformly replace token-addressable attention. The table demonstrates coexistence at the performance frontier; its chronological ordering does not rank attention mechanisms or imply that any particular mechanism causes higher model quality.

8.4 Summary

The longitudinal inventory and the frozen frontier comparison support the same conclusion: current LLM attention design is diversifying rather than converging. Hybrid records have become more common and are still organized mainly through fixed layer-wise composition, while cross-layer reuse remains a limited but conceptually distinct extension that increases the depth-wise persistence of selected memory and routing artifacts. High-performing open-weight

models preserve this diversity and continue to combine efficient state propagation or sparse access with explicit token retrieval.

9 Synthesis Across Mechanisms, Architectures, and Future Directions

The preceding chapters reveal that the evolution of attention-centered sequence architectures can be understood, at least in part, as an expanding redesign of contextual memory rather than simply as a succession of attempts to replace one operator. Explicit token memories, recurrent associative states, structured state-space models, and hybrid architectures begin from different technical formulations, yet repeatedly confront a related set of design questions: which memory units are maintained, how new information updates them, which represented information is made available to a query, how that information is read, and how multiple readouts are coordinated. Across the literature reviewed in this survey, the design focus consequently expands from optimizing an isolated interaction rule toward organizing the representation, lifecycle, and use of contextual memory as a whole.

Building on this perspective, this section develops a three-level synthesis that traces the evolution of memory control from mechanism design to architectural coordination and future memory systems. At the *mechanism level*, explicit-memory and recurrent-state methods begin from different memory representations and computational forms, yet both gradually extend the scope of control from their respective core concerns to a broader set of functions spanning Memory Representation, Memory Update, Access, Readout, and Integration. The ranges of memory functions explicitly controlled by the two routes consequently show a growing degree of overlap. At the *architecture level*, an important direction is to integrate memory mechanisms with complementary capabilities within a single model. This complementarity is currently realized primarily through predetermined layer-wise composition, while recent systems begin to extend coordination beyond module placement to the cross-layer sharing, reuse, and refresh of selected memory and routing artifacts. At the *forward-looking level*, we further synthesize these mechanism- and architecture-level developments into a stateful multidimensional memory-routing hypothesis spanning temporal scope, network depth, substrate type, and representation resolution.

9.1 Mechanism Level: Distinct Starting Points and Expanding Control Scopes

At the mechanism level, the surveyed literature suggests two broad routes for organizing contextual memory. The *explicit-memory route* retains an enumerable memory interface while reducing the cost of representing or accessing its units. The *state-based route* compresses history into recurrently maintained states while improving their capacity, update control, and recoverability. These routes represent different predominant memory forms and design pressures rather than mutually exclusive model classes.

Explicit-memory route. As reviewed in Sections 3 and 4, this route represents contextual memory through token KVs, shared or latent token representations, bounded slots, blocks, or lower-resolution summaries. Its primary emphasis lies in Memory Representation and Access: methods reduce redundancy across heads, channels, or layers, alter the resolution of stored history, restrict the candidate set exposed to a query, or reuse representations and retrieval decisions. Its control scope also extends to Memory Update when bounded or multiresolution memories merge, replace, or consolidate historical information; to Readout when routing scores, summary estimates, or modified attention maps affect the aggregation of eligible memory; and to Integration through head coordination, output gating, or the combination of multiple readout components. The route therefore retains an enumerable memory interface while extending control from representation and access efficiency to the broader lifecycle and use of the represented memory.

State-based route. As reviewed in Sections 5 and 6, this route replaces a growing list of token-wise memories with one or more recurrently maintained distributed states. Its primary emphasis lies in Memory Representation and Memory Update: methods determine how historical information is compressed, accumulated, retained, corrected, overwritten, or selectively propagated. Its control scope expands through additional rows, slots, partitions, temporal summaries, and state groups that differentiate memory capacity and temporal resolution; selection among these units makes Access more explicit; conditioned, multi-channel, or multi-state projections enrich Readout; and gating or coordination among completed state readouts makes Integration a direct design consideration. The route therefore extends from constructing and updating a compressed state toward organizing how multiple state units are represented, maintained, exposed, read, and coordinated.

Figure 15 summarizes the distinct starting points and expanding control scopes of the two routes. Darker cells indicate each route’s primary emphasis, whereas lighter cells mark functions that become more explicit in subsequent designs. The columns are functional dimensions rather than a mandatory execution sequence.

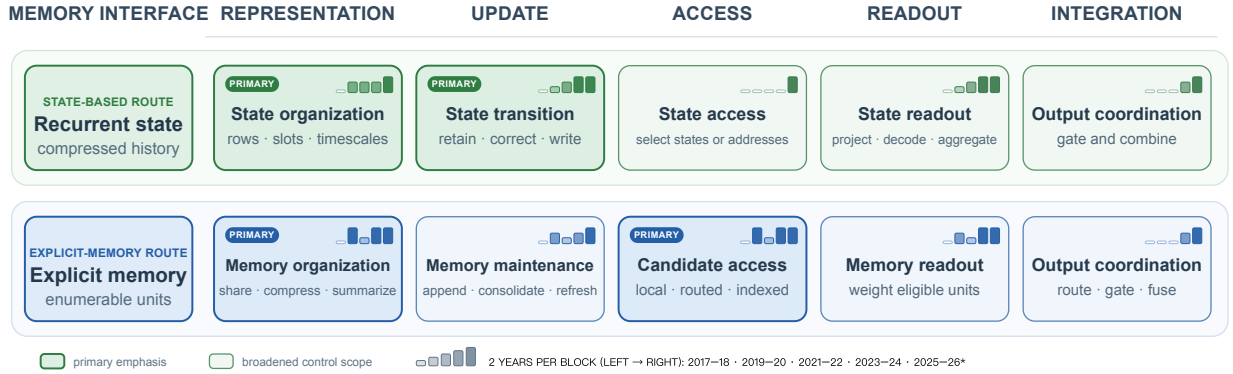


Figure 15: Mechanism-level synthesis of the expanding control scopes of explicit-memory and state-based methods. For the five functional dimensions, the upper-right mini-bars represent 2017–2018, 2019–2020, 2021–2022, 2023–2024, and 2025–2026 from left to right; taller and darker bars indicate greater qualitative activity among the representative methods discussed in Sections 3–6. The 2026 coverage extends through September.

Synthesis 1: Growing Overlap in the Scope of Memory Control

Explicit-memory and state-based methods retain different memory interfaces and computational forms, yet both expand from their respective initial emphases toward a broader and increasingly overlapping set of memory functions. The convergence lies in the scope of explicit design control across Memory Representation, Memory Update, Access, Readout, and Integration, rather than in a shared memory substrate or computational mechanism.

This growing overlap in control scope provides the mechanism-level basis for the architecture-level development examined next. As Representation, Update, Access, Readout, and Integration become more explicit and coordinated design variables, hybrid architectures can combine complementary memory capabilities across layers, heads, branches, and tokens rather than requiring one mechanism to satisfy every quality–cost trade-off. The next subsection examines how this complementarity is realized in publicly documented LLM architectures and how coordination begins to extend from module placement to the cross-layer lifecycle of selected memory and routing artifacts.

9.2 Architecture-Level Synthesis: From Complementary Placement to Memory-Artifact Lifecycles

The inventory analysis in Section 8.2, summarized in Table 17, reveals two ways in which contextual memory is increasingly organized along network depth: the layer-wise composition of heterogeneous memory mechanisms and the cross-layer reuse of selected memory and routing artifacts. Layer-wise composition determines what form of memory processing occurs at each representational stage, whereas cross-layer reuse determines which artifacts produced at one stage remain available to later stages. Together, these developments bring the construction, maintenance, and reuse of contextual memory along network depth into the architecture-level design space.

Within the current sample, Hybrid records become more common and are organized mainly through predetermined layer-wise schedules. Explicit attention supports fine-grained query-dependent retrieval, Sparse Attention restricts the candidate budget, and recurrent Linear Attention or SSM layers propagate compressed history with bounded or slowly growing decoding state. Coordinated through the hidden-state stream, these mechanisms apply different memory operations as a representation moves through the network. Layer-wise composition therefore organizes complementary memory capabilities across depth, allowing different representational stages to contribute different forms of retrieval, compression, and state propagation.

Layer-wise composition changes the memory processing performed at each depth; cross-layer reuse changes the lifetime of the resulting artifacts. In the documented cases, selected KV representations, index states, or candidate decisions remain available across depth instead of being independently reconstructed at every layer. This introduces a distinction between *capability placement*—which memory functions operate at different depths—and *artifact-lifecycle coordination*—which memory or routing artifacts persist across layers and when they are reused or refreshed. In the current inventory, layer-wise Hybrid composition is already common, whereas explicit cross-layer artifact reuse is

confined to five release-level records corresponding to three architectural patterns documented across GLM [68, 69, 15], DeepSeek [101], and LongCat [16, 104] systems.

Synthesis 2: Network Depth as an Emerging Dimension of Memory Organization

Layer-wise composition determines what memory processing occurs at each depth, while cross-layer reuse determines what memory artifacts persist across depth. Together, they make network depth an emerging dimension along which contextual memory is constructed and managed.

Viewed together, these developments shift the architecture-level question from which mechanism each layer should use to how memory should be organized across the network as a whole. As representations move through successive layers, memory may be retrieved, compressed, transformed, or propagated by different mechanisms, while selected intermediate artifacts may remain available to later stages. Network depth thus becomes an additional coordinate for organizing where contextual memory is produced, transformed, maintained, reused, and refreshed. This perspective motivates the forward-looking multidimensional memory organization considered next.

9.3 Forward-Looking Hypothesis: Stateful Multidimensional Memory Routing

The mechanism- and architecture-level syntheses above motivate a broader design space for contextual memory. At the mechanism level, explicit-memory and state-based methods increasingly control overlapping sets of memory functions. At the architecture level, complementary capabilities are commonly distributed across layers, while a small number of recent systems begin to extend the lifecycle of selected memory and routing artifacts across network depth. Taken together, these developments suggest that persistent memory could be organized and controlled jointly across multiple architectural coordinates, rather than being defined only by temporal position or by the mechanism of one layer.

9.3.1 Multidimensional Memory as an Address Space

We represent the admissible addresses of persistent memory by four coordinates:

$$\mathcal{A}_t = \mathcal{T}_t \times \mathcal{L} \times \mathcal{S} \times \mathcal{G}, \quad \mathcal{M}_t = \{M_t[a] \mid a \in \mathcal{A}_t\} \in \mathfrak{M}_\rho, \quad (68)$$

where an address $a = (\tau, \ell, s, g)$ identifies the temporal scope, network depth, memory substrate, and representation granularity of a memory unit, and $M_t[a]$ denotes the memory unit stored at address a . The temporal coordinate $\tau \in \mathcal{T}_t$ may refer to a token, segment, or longer historical interval. The depth coordinate $\ell \in \mathcal{L}$ identifies the layer or representational stage at which the unit is produced or maintained. The substrate coordinate $s \in \mathcal{S}$ distinguishes forms such as explicit KV, an associative state, or a structured recurrent state. The granularity coordinate $g \in \mathcal{G}$ distinguishes fine-resolution representations from coarser summaries of the same or related temporal scope.

These coordinates describe an address space for memory organization rather than requiring one physically materialized Cartesian tensor. A model could instantiate only a selected set of addresses and could associate different Update and Readout rules with different substrates. The same historical interval might, for example, be represented by fine-grained token KVs, a block-level representation, or a compressed recurrent summary. Memories produced at different depths could likewise preserve information associated with different levels of representation and remain available to selected later computations.

Such an organization allows persistent memory units to assume differentiated roles. Fine-resolution explicit representations may support precise query-dependent retrieval, while compressed summaries or recurrent states may provide broader historical coverage at lower storage or access cost. Architecture design would consequently determine not only which memory forms are present, but also where they are produced, how long they persist, and at what granularity they remain available.

9.3.2 Sparse Write and Sparse Read

Uniformly updating and reading every admissible memory unit would diminish the efficiency gained from differentiated memory roles. We therefore distinguish a selected write-address set $\mathcal{W}_t$ from the eligible memory view $\mathcal{C}_t$ exposed to the current query. Sparse Write can be expressed as

$$\mathcal{W}_t \subseteq \mathcal{A}_t, \quad \mathcal{M}_t^+ = \text{Update}_\rho(\mathcal{M}_t^-, x_t; \mathcal{W}_t), \quad (69)$$

where $\mathcal{M}_t^-$ and $\mathcal{M}_t^+$ denote the memory before and after the current update. The selected addresses may correspond to particular temporal scopes, network depths, substrates, or representation granularities. The Update rule remains

specific to each selected substrate: it may append an explicit representation, modify a recurrent state, merge information into a summary, or apply a substrate-specific retention or correction rule. Sparse Write therefore controls where incoming information is maintained without requiring heterogeneous memory units to share one update mechanism.

Following the analytical framework in Section 2, the complete Sparse Read pathway is written as

$$\mathcal{C}_t = \text{Access}_\rho(q_t, \widehat{\mathcal{M}}_t), \quad r_t = \text{Readout}_\rho(q_t, \mathcal{C}_t), \quad o_t = \text{Integration}_\rho(r_t; x_t). \quad (70)$$

Here, $\widehat{\mathcal{M}}_t$ denotes the memory visible to the read path under the mechanism’s read–write convention, and $\mathcal{C}_t$ is the eligible memory view produced by Access. It may contain selected units from different temporal scopes, depths, substrates, and granularities, together with the local metadata required by their Readout rules. Readout converts this eligible view into contextual information, while Integration transforms or coordinates one or more completed readouts into the module output.

Sparse Write and Sparse Read are coupled through Memory Representation. Write routing determines which memory addresses receive new information and therefore constrains what can remain available to later queries. Read routing allocates retrieval over the resulting memory organization. Writing to units that are rarely accessed wastes storage and update effort, whereas repeatedly accessing units that receive insufficient or unsuitable updates weakens retrieval quality. Their budgets and routing policies may benefit from joint coordination over the same multidimensional address space.

9.3.3 Stateful Routing of Memory Lifecycles

This organization resembles a multidimensional Memory-MoE in which persistent memory units assume different functional roles and routing determines which units participate in writing and reading. The central distinction from a conventional feed-forward MoE is persistence. Conventional expert routing primarily selects the computation applied to the current representation; memory routing would additionally determine which information is retained, modified, and made available to future queries. A write-routing decision could therefore affect both the current update and the future state of the memory system.

Hypothesis: Stateful Multidimensional Memory Routing

A future architecture may organize persistent memory jointly across temporal scope, network depth, substrate type, and representation granularity. Separate write and read routing would determine which memory addresses preserve incoming information and which represented units contribute to a query. Because write decisions change the information available to later computation, routing would coordinate not only current resource allocation, but also the future lifecycle of contextual memory.

Such a system would introduce several coupled design questions. Memory units would require compatible interfaces across representational levels, while routing would need to balance memory occupancy, update frequency, retrieval fidelity, interference, and hardware efficiency. Persistent write decisions would also create delayed dependencies between information stored at one step and its value to later queries. Feature alignment, memory ownership, staleness control, recovery from harmful writes, load balancing, and long-horizon credit assignment would consequently become central considerations. The hypothesis would ultimately need to be evaluated by whether multidimensional memory routing improves the practical quality–efficiency frontier under matched memory, computation, and routing budgets.

9.3.4 Concrete and Testable Research Directions

Building on the hypothesis above, we outline four feasible directions that progressively expand the scope of memory control. They begin with choosing suitable representations for heterogeneous context, then separate large-scale memory construction from query-time retrieval, extend memory control to incremental lifecycle management, and finally coordinate memory and computation according to the current task state. Existing systems provide partial evidence for each step, while the proposals below identify concrete extensions that could be implemented and tested without realizing the complete multidimensional architecture at once.

Workload-aware differentiated memory representation. Different input regions need not share a single memory representation. Long contexts may combine large documents, code repositories, tool specifications, recent interactions, and execution logs, which differ in stability, structural hierarchy, update frequency, and required retrieval precision.

Existing systems already expose parts of this design space: Qwen Sparse Attention uses compressed micro-block addresses to select original-token content, HCA and CSA combine coarser global representations with fine-grained local access, and Engram assigns stable patterns to deterministic conditional lookup rather than repeatedly reconstructing them through neural computation [13, 96, 50, 170]. A concrete next step is a typed memory constructor that maps each input region to one of a small number of representations—exact, compressed, indexed, or recurrent—using observable attributes such as source type, expected stability, hierarchy, and precision requirements. A repository, for example, could retain a coarse repository summary, file- and symbol-level indexes, and exact KVs only for active code regions. Restricting the initial routing space to these representation choices under an explicit capacity budget would make the substrate and granularity coordinates directly testable.

Asymmetric memory construction and retrieval. Large-scale memory construction and query-time retrieval need not use the same computation. Input-heavy workloads may provide extensive background context but require only short or incremental outputs, making uniform processing across prefill and decoding inefficient. DeepSeek-V4.1-Flash offers direct evidence for this asymmetry: its Causal Encoder–Decoder activates fewer parameters during prefill than decoding and combines CSA2 cross-layer KV/index reuse with aggressive cache compression; QSA similarly lowers long-input scanning cost by compressing routing keys before selecting original-token KVs [101, 96]. A practical extension is a multiresolution construction pipeline in which a lightweight prefill path produces summaries and routing addresses while retaining exact payloads only for precision-sensitive regions. During decoding, each query would first inspect coarse memory and escalate to finer blocks or original tokens only when the coarse readout is insufficient. This coarse-to-fine interface would make prefill cost, persistent-memory size, and retrieval precision separately controllable, extending differentiated Representation into Memory Update, Access, and Readout.

Incremental memory lifecycle and reusable artifacts. Memory maintenance should scale with the amount of changed context rather than with total context length. A document collection may add one section, a repository may modify one function, and an execution trace may append only a few observations; rebuilding all associated memory would repeat work over unchanged content. Cross-layer systems provide initial evidence that memory artifacts can persist: IndexCache reuses retrieval indices, CSA2 defines Full, Reindex, and Reuse modes for KV and index artifacts, and YOIO/CLSA shares one routing decision across multiple downstream layers [15, 101, 100]. The next step is conditional lifecycle control. At each layer or context update, a lightweight controller could choose *reuse*, *refresh*, *transform*, or *invalidate* for each representation or routing artifact. Reuse could apply when hidden-state drift and candidate-set change remain small; refresh when query–index agreement falls below a threshold; transformation when an artifact must be adapted to a new representation space; and invalidation after a source-version change. For a code edit, only the modified function, its file summary, and affected index entries would be refreshed, while unrelated repository memory remains reusable. This would turn temporal scope and depth from static address labels into dynamic coordinates of memory maintenance.

Task-conditioned joint memory and computation routing. Memory and computation policies should ultimately adapt together to the current task state. Broad planning may favor repository- or document-level summaries, precise editing may require local exact tokens, execution may emphasize recent logs and tool outputs, and verification may require multi-source evidence. Elastic Attention and depth-adaptive Hybrid Architecture proposals already vary the allocation of attention and recurrent paths according to input or depth, while Mixture-of-Depths dynamically allocates computation under a fixed budget [146, 147, 171]. A tractable realization could begin with a finite menu of configurations rather than an unrestricted controller. Each configuration would specify a memory substrate, retrieval granularity, attention mode, computation-depth budget, and write-back action; a compact task-state representation would then select among them. Planning could invoke broad coarse-grained retrieval, editing fine-grained local memory, and verification multi-source Readout and Integration. The policy could first be distilled from task annotations or fixed heuristics and later optimized under memory and computation constraints. By jointly selecting how memory is represented, updated, accessed, read, and integrated, this direction provides a constrained path toward complete stateful multidimensional memory routing.

10 Conclusion

This survey has examined four mechanism-centered research lines—Softmax Attention, Sparse Attention, Linear Attention, and State Space Models—together with Hybrid Architecture that organizes these mechanisms within larger models. Rather than treating these lines only as alternative approaches to reducing computational complexity, we have analyzed how they balance fine-grained addressability, compressed historical coverage, memory capacity, update control, and practical computation.

To provide a comparable basis for this analysis, we introduced a five-dimensional memory-centric lens: **Memory Representation**, **Memory Update**, **Access**, **Readout**, and **Integration**. These dimensions identify what historical information remains represented, how the represented memory changes, what becomes eligible for a query, how eligible memory is read, and how one or more completed readouts are transformed or coordinated into the module output. They are analytical roles rather than a universal computational factorization: one operation may affect several dimensions, and the chapter-level research lines remain historically and technically overlapping. Within this view, Softmax Attention reduces redundancy or historical resolution while retaining a separately addressable explicit-memory interface and normalized query–key Readout; Sparse Attention controls which explicit memory units become eligible for a query; Linear Attention develops recurrent associative states through more precise update, capacity, and temporal organization; SSMs structure compressed history through learned and input-conditioned dynamics; and Hybrid Architectures allocate complementary capabilities across layers, heads, branches, and tokens.

At the mechanism level, explicit-memory and recurrent-state methods retain different predominant memory interfaces, but the ranges of memory functions they explicitly control increasingly overlap. Explicit-memory methods extend beyond representation and access efficiency toward bounded update, readout modulation, and output coordination. State-based methods extend beyond state representation and recurrent update toward multiple memory units, selective access, richer readout, and coordinated integration. The shared development is therefore not convergence on one memory substrate or operator, but an expansion in the scope over which memory representation, update, access, readout, and integration are jointly considered.

At the architecture level, our curated inventory of 59 release-level architecture records spanning 14 major model lineages—drawn primarily from text-centered LLMs and, where relevant, from the autoregressive language backbones of natively multimodal models—shows continued coexistence rather than a universal replacement path. Single-family GQA, MLA, and Sparse Attention backbones remain in use, while Hybrid designs increasingly combine complementary memory capabilities. Within the documented Hybrid architectures, predetermined layer-wise composition remains the principal organizational pattern. A smaller set of recent Sparse Attention systems additionally extends coordination across network depth by sharing or reusing selected KV representations, index states, or candidate decisions. This cross-layer reuse is an additional architectural attribute rather than a subtype of Hybrid composition, and the current evidence remains concentrated in a small number of architecture records.

Taken together, the mechanism-level review and architecture-level analyses also show why attention-centered architectures should not be evaluated through asymptotic complexity alone. Explicit memories retain fine-grained addressability but incur storage and data-movement costs; sparse retrieval depends on index quality, candidate recall, and irregular execution; recurrent states face capacity, interference, and recoverability limits; and heterogeneous designs introduce placement, routing, synchronization, and kernel-design challenges. Meaningful comparison should therefore consider model quality, persistent memory, prefill and decoding cost, retrieval behavior, update stability, routing overhead, and realized hardware utilization under matched budgets and implementation conditions.

The mechanism-level and architecture-level syntheses jointly motivate a forward-looking hypothesis in which persistent contextual memory is organized across temporal scope, network depth, substrate type, and representation granularity. Layer-wise composition makes depth a coordinate of capability placement, while emerging cross-layer reuse suggests that depth may also become a coordinate of memory- and routing-artifact persistence. Coordinated Sparse Write and Sparse Read would determine which memory addresses receive information and which represented units contribute to a query, thereby controlling not only current computation but also the future lifecycle of memory. This remains a design hypothesis whose value depends on whether such organization improves the practical quality–efficiency frontier without introducing prohibitive routing, optimization, memory-management, or hardware costs. The memory-centric lens developed in this survey provides a common vocabulary for relating historically distinct research lines and formulating such testable questions. Overall, efficient sequence architecture design is increasingly concerned with the joint organization, lifecycle, and selective use of contextual memory rather than the optimization of an isolated attention operator.

A Supplementary Materials: Publicly Documented Architecture Inventory

This appendix reports the 59 release-level architecture records used in Section 8. *Release record* is the unit counted in Table 17: models released together are grouped when they share the same language-model sequence-mixing architecture, whereas separately released versions remain separate records. *Model lineage* identifies the broader model family used to obtain the 14-lineage count. The *Attention / mixer composition* column gives only the principal sequence-mixing mechanisms and, where useful, their layer ratio. A record is labeled Hybrid when its language backbone instantiates two or more distinguishable sequence-mixing or access regimes in separate layers, branches, heads, or token paths; multiple support components jointly forming a single sparse-attention candidate set do not by themselves constitute Hybrid composition. Records with insufficient architectural disclosure are marked Undisclosed and excluded from the Non-Hybrid/Hybrid percentages. Cross-layer reuse is reported separately in Table 17 and the accompanying discussion. For multimodal systems, only the autoregressive language-model backbone is classified. The inventory is purposively curated rather than exhaustive or market-share weighted.

Table S1: The 59 publicly documented release-level architecture records analyzed in Section 8, with the lineage and Hybrid labels used to derive Table 17.

Release	Model lineage	Release record	Attention / mixer composition	Hybrid	Source
2026-09	MiMo	MiMo-V2.6-Pro-RL	SWA-GQA + global GQA (6:1)	Yes	[168]
2026-09	DeepSeek	DeepSeek-V4.1-Flash	SWA + CSA2	Yes	[101]
2026-09	MiniCPM	MiniCPM5-2B family	Full GQA	No	[160]
2026-09	K2	K2-Horizon-375B-A23B	Full GQA	No	[161]
2026-08	Qwen	Qwen3.8-2.4T-A95B	Gated DeltaNet + full GQA (3:1)	Yes	Model card; config
2026-08	Qwen	Qwen3.8-27B	Gated DeltaNet + gated full GQA (3:1)	Yes	[169]
2026-08	Qwen	Qwen3.8-Flash-Next	Gated DeltaNet + QSA (3:1)	Yes	[13]
2026-08	GLM	GLM-5.3	MLA-based DSA	No	[69, 15]
2026-08	GLM	GLM-5.3-Flash	KDA + compressed-indexer DSA (approximately 3:1)	Yes	[172]
2026-08	LongCat	LongCat-Flash-Lite-Sparse	LSA	No	[16]
2026-07	Kimi	Kimi K3	KDA + Gated MLA	Yes	[66, 173]
2026-07	Gemma	Gemma 4	SWA + global full Attention (4:1 or 5:1)	Yes	[174]
2026-06	LongCat	LongCat-2.0	LSA	No	[16, 104]
2026-06	MiniMax	MiniMax-M3	Dense Attention + MSA	Yes	[95, 94]
2026-06	DeepSeek	DeepSeek-V4	CSA + HCA (compressed dense Attention + SWA)	Yes	[50]
2026-06	GLM	GLM-5.2	MLA-based DSA	No	[68, 15]
2026-05	Step	Step-3.7-Flash	Full GQA + SWA-GQA (1:3)	Yes	Model page; config
2026-05	MiniMax	MiniMax-M2	Full GQA	No	[159]
2026-04	Qwen	Qwen3.6-27B	Gated DeltaNet + gated full Attention (3:1)	Yes	[62]
2026-04	Qwen	Qwen3.6-35B-A3B	Gated DeltaNet + gated full Attention (3:1)	Yes	[61]
2026-04	Mistral	Mistral Medium 3.5-128B	Full GQA	No	[175]
2026-02	Step	Step-3.5-Flash	Full GQA + SWA-GQA (1:3)	Yes	Tech. report; config
2026-02	Qwen	Qwen3.5	Gated DeltaNet + gated full Attention (3:1)	Yes	[60]
2026-02	MiniCPM	MiniCPM-SALA	Lightning Attention + InfLLM-v2 Sparse Attention (3:1)	Yes	[176]
2026-02	GLM	GLM-5	MLA-based DSA	No	[67]
2026-01	LongCat	LongCat-Flash-Lite	MLA	No	[163]
2026-01	MiMo	MiMo-V2-Flash	SWA + global Attention (5:1)	Yes	[177]
2025-12	Nemotron	Nemotron 3	Mamba + full Attention + MLP-only blocks	Yes	[166]
2025-12	DeepSeek	DeepSeek-V3.2	MLA-based DSA	No	[64]
2025-09	LongCat	LongCat-Flash family	MLA	No	[162]
2025-09	MiniCPM	MiniCPM4.1	InfLLM-v2 Sparse GQA	No	[178, 82]
2025-09	Qwen	Qwen3-Next	Gated DeltaNet + gated full Attention (3:1)	Yes	[164, 20, 44]
2025-08	Nemotron	Nemotron Nano 2	Mamba-2 + full Attention	Yes	[179]
2025-07	Kimi	Kimi K2	MLA	No	[65]
2025-06	MiniCPM	MiniCPM4	InfLLM-v2 Sparse GQA	No	[180, 82]

Continued on the next page

Table S1 continued from the previous page

Release	Model lineage	Release record	Attention / mixer composition	Hybrid	Source
2025-06	MiniMax	MiniMax-M1	Lightning Attention + Softmax Attention (7:1)	Yes	[165]
2025-05	Qwen	Qwen3	Full GQA	No	[59]
2025-05	MiMo	MiMo-7B	Full GQA	No	[181]
2025-04	Llama	Llama 4 Scout / Maverick	RoPE chunked GQA + NoPE full GQA (3:1)	Yes	[57]
2025-04	Nemotron	Nemotron-H	Mamba + full Attention + MLP-only blocks	Yes	[182]
2025-03	Mistral	Mistral Small 3.1-24B	Full GQA	No	[158]
2025-03	Gemma	Gemma 3	Local Attention + global Attention (5:1)	Yes	[183]
2025-01	MiniMax	MiniMax-01 / Text-01	Lightning Attention + Softmax Attention (7:1)	Yes	[107]
2025-01	Kimi	Kimi k1.5	Undisclosed	Undisclosed	[184]
2024-12	DeepSeek	DeepSeek-V3	MLA	No	[63]
2024-09	MiniCPM	MiniCPM3-4B	MLA	No	[185]
2024-08	Gemma	Gemma 2	Local Attention + global Attention (1:1)	Yes	[186]
2024-07	Qwen	Qwen2	Full GQA	No	[58]
2024-06	GLM	GLM-4 family	Full GQA	No	[187]
2024-05	DeepSeek	DeepSeek-V2	MLA	No	[5]
2024-04	Mistral	Mixtral-8x22B-v0.1	Full GQA	No	[188]
2024-04	Llama	Llama 3 family	Full GQA	No	[56]
2024-03	Gemma	Gemma 1	MQA / MHA (scale-dependent)	No	[189]
2023-12	Mistral	Mixtral-8x7B-v0.1	Full GQA	No	[190]
2023-09	Qwen	Qwen	Full MHA	No	[167]
2023-09	Mistral	Mistral-7B-v0.1	SWA-GQA	No	[191]
2023-07	Llama	Llama 2	MHA / GQA (scale-dependent)	No	[192]
2023-02	Llama	LLaMA	Full MHA	No	[193]
2022-10	GLM	GLM-130B	Full MHA	No	[194]

References

- [1] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017, pp. 5998–6008. arXiv: 1706.03762. URL: <https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need>.
- [2] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *arXiv preprint arXiv:2005.14165* (2020). arXiv: 2005.14165. URL: <https://arxiv.org/abs/2005.14165>.
- [3] Yushi Bai et al. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding”. In: *arXiv preprint arXiv:2308.14508* (2023). arXiv: 2308.14508. URL: <https://arxiv.org/abs/2308.14508>.
- [4] Cheng-Ping Hsieh et al. “RULER: What’s the Real Context Size of Your Long-Context Language Models?”. In: *arXiv preprint arXiv:2404.06654* (2024). arXiv: 2404.06654. URL: <https://arxiv.org/abs/2404.06654>.
- [5] DeepSeek-AI et al. “DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model”. In: *arXiv preprint arXiv:2405.04434* (2024). arXiv: 2405.04434. URL: <https://arxiv.org/abs/2405.04434>.
- [6] Chenglong Chu et al. “Kwai Summary Attention Technical Report”. In: *arXiv preprint arXiv:2604.24432* (2026). arXiv: 2604.24432. URL: <https://arxiv.org/abs/2604.24432>.
- [7] Noam Shazeer. “Fast Transformer Decoding: One Write-Head is All You Need”. In: *arXiv preprint arXiv:1911.02150* (2019). arXiv: 1911.02150. URL: <https://arxiv.org/abs/1911.02150>.
- [8] Joshua Ainslie et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 4895–4901. DOI: 10.18653/v1/2023.emnlp-main.298. arXiv: 2305.13245. URL: <https://aclanthology.org/2023.emnlp-main.298/>.
- [9] William Brandon et al. “Reducing Transformer Key-Value Cache Size with Cross-Layer Attention”. In: *arXiv preprint arXiv:2405.12981* (2024). arXiv: 2405.12981. URL: <https://arxiv.org/abs/2405.12981>.
- [10] Iz Beltagy, Matthew E. Peters, and Arman Cohan. “Longformer: The Long-Document Transformer”. In: *arXiv preprint arXiv:2004.05150* (2020). arXiv: 2004.05150. URL: <https://arxiv.org/abs/2004.05150>.
- [11] Manzil Zaheer et al. “Big Bird: Transformers for Longer Sequences”. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 17283–17297. arXiv: 2007.14062. URL: <https://proceedings.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html>.
- [12] Jingyang Yuan et al. “Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 23078–23097. DOI: 10.18653/v1/2025.acl-long.1126. arXiv: 2502.11089. URL: <https://aclanthology.org/2025.acl-long.1126/>.
- [13] Zihan Qiu et al. “On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability”. In: *arXiv preprint arXiv:2608.30320* (2026). arXiv: 2608.30320. URL: <https://arxiv.org/abs/2608.30320>.
- [14] Xiang Hu et al. “Hierarchical Sparse Attention Done Right: Toward Infinite Context Modeling”. In: *arXiv preprint arXiv:2607.02980* (2026). arXiv: 2607.02980. URL: <https://arxiv.org/abs/2607.02980>.
- [15] Yushi Bai et al. “IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse”. In: *arXiv preprint arXiv:2603.12201* (2026). arXiv: 2603.12201. URL: <https://arxiv.org/abs/2603.12201>.
- [16] Wen Zan et al. “LongCat Sparse Attention: Taming the Lightning via Streaming-aware Hierarchical Cross-Layer Indexing”. In: *arXiv preprint arXiv:2608.01662* (Aug. 2026). arXiv: 2608.01662 [cs.CL]. URL: <https://arxiv.org/abs/2608.01662>.
- [17] Angelos Katharopoulos et al. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 5156–5165. arXiv: 2006.16236. URL: <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [18] Yutao Sun et al. “Retentive Network: A Successor to Transformer for Large Language Models”. In: *arXiv preprint arXiv:2307.08621* (2023). arXiv: 2307.08621. URL: <https://arxiv.org/abs/2307.08621>.
- [19] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. “Linear Transformers Are Secretly Fast Weight Programmers”. In: *arXiv preprint arXiv:2102.11174* (2021). arXiv: 2102.11174. URL: <https://arxiv.org/abs/2102.11174>.

- [20] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. “Gated Delta Networks: Improving Mamba2 with Delta Rule”. In: *International Conference on Learning Representations*. 2025. arXiv: 2412.06464. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/4904fad153f6434a7bcf04465d4be2cc-Abstract-Conference.html.
- [21] Yuqi Pan et al. “Scaling Linear Attention with Sparse State Expansion”. In: *arXiv preprint arXiv:2507.16577* (2025). arXiv: 2507.16577. URL: <https://arxiv.org/abs/2507.16577>.
- [22] Loïc Cabannes et al. “Sparse Delta Memory: Scaling the State of Linear RNNs through Sparsity”. In: *arXiv preprint arXiv:2607.07386* (2026). arXiv: 2607.07386. URL: <https://arxiv.org/abs/2607.07386>.
- [23] Han Guo et al. “Log-Linear Attention”. In: *arXiv preprint arXiv:2506.04761* (2025). arXiv: 2506.04761. URL: <https://arxiv.org/abs/2506.04761>.
- [24] Xin Wang et al. “Dynamic Linear Attention”. In: *arXiv preprint arXiv:2606.10650* (2026). arXiv: 2606.10650. URL: <https://arxiv.org/abs/2606.10650>.
- [25] Albert Gu, Karan Goel, and Christopher Ré. “Efficiently Modeling Long Sequences with Structured State Spaces”. In: *International Conference on Learning Representations*. 2022. arXiv: 2111.00396. URL: <https://openreview.net/forum?id=uYLFoz1v1AC>.
- [26] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *First Conference on Language Modeling*. 2024. arXiv: 2312.00752. URL: <https://openreview.net/forum?id=tEYskw1VY2>.
- [27] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 10041–10071. arXiv: 2405.21060. URL: <https://proceedings.mlr.press/v235/dao24a.html>.
- [28] Opher Lieber et al. “Jamba: A Hybrid Transformer-Mamba Language Model”. In: *arXiv preprint arXiv:2403.19887* (2024). arXiv: 2403.19887. URL: <https://arxiv.org/abs/2403.19887>.
- [29] Xin Dong et al. “Hymba: A Hybrid-head Architecture for Small Language Models”. In: *International Conference on Learning Representations*. 2025. arXiv: 2411.13676. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/f32def07618040e540e0a6182e290562-Abstract-Conference.html.
- [30] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. “Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention”. In: *arXiv preprint arXiv:2404.07143* (2024). arXiv: 2404.07143. URL: <https://arxiv.org/abs/2404.07143>.
- [31] Jusen Du et al. “Native Hybrid Attention for Efficient Sequence Modeling”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, 2026, pp. 3826–3842. DOI: 10.18653/v1/2026.acl-long.176. arXiv: 2510.07019. URL: <https://aclanthology.org/2026.acl-long.176/>.
- [32] Tianyang Lin et al. “A Survey of Transformers”. In: *arXiv preprint arXiv:2106.04554* (2021). arXiv: 2106.04554. URL: <https://arxiv.org/abs/2106.04554>.
- [33] Yi Tay et al. “Efficient Transformers: A Survey”. In: *arXiv preprint arXiv:2009.06732* (2020). arXiv: 2009.06732. URL: <https://arxiv.org/abs/2009.06732>.
- [34] Yunpeng Huang et al. “Advancing Transformer Architecture in Long-Context Large Language Models: A Comprehensive Survey”. In: *arXiv preprint arXiv:2311.12351* (2023). arXiv: 2311.12351. URL: <https://arxiv.org/abs/2311.12351>.
- [35] Xindi Wang et al. “Beyond the Limits: A Survey of Techniques to Extend the Context Length in Large Language Models”. In: *arXiv preprint arXiv:2402.02244* (2024). arXiv: 2402.02244. URL: <https://arxiv.org/abs/2402.02244>.
- [36] Jiaheng Liu et al. “A Comprehensive Survey on Long Context Language Modeling”. In: *arXiv preprint arXiv:2503.17407* (2025). arXiv: 2503.17407. URL: <https://arxiv.org/abs/2503.17407>.
- [37] Yutao Sun et al. “Efficient Attention Mechanisms for Large Language Models”. In: *Patterns* 7.9 (2026), p. 101594. DOI: 10.1016/j.patter.2026.101594. URL: <https://doi.org/10.1016/j.patter.2026.101594>.
- [38] Badri Narayana Patro and Vijay Srinivas Agneeswaran. “Mamba-360: Survey of State Space Models as Transformer Alternative for Long Sequence Modelling: Methods, Applications, and Challenges”. In: *arXiv preprint arXiv:2404.16112* (2024). arXiv: 2404.16112. URL: <https://arxiv.org/abs/2404.16112>.

- [39] Haoyang Li et al. “A Survey on Large Language Model Acceleration Based on KV Cache Management”. In: *Transactions on Machine Learning Research* (2025). arXiv: 2412.19442. URL: <https://arxiv.org/abs/2412.19442>.
- [40] Sining Zhou et al. “Memory for Large Language Models”. In: *arXiv preprint arXiv:2607.25380* (2026). arXiv: 2607.25380 [cs.CL]. URL: <https://arxiv.org/abs/2607.25380>.
- [41] Mahdi Karami, Razvan Pascanu, and Vahab Mirrokni. *Lattice: Learning to Compress the Cache in the Attention*. Google Research publication page. 2025. URL: <https://research.google/pubs/lattice-learning-to-compress-the-cache-in-the-attention/> (visited on 09/29/2026).
- [42] Mahdi Karami et al. “Trellis: Learning to Compress Key-Value Memory in Attention Models”. In: *arXiv preprint arXiv:2512.23852* (2025). arXiv: 2512.23852. URL: <https://arxiv.org/abs/2512.23852>.
- [43] Ali Behrouz et al. “Memory Caching: RNNs with Growing Memory”. In: *arXiv preprint arXiv:2602.24281* (2026). arXiv: 2602.24281. URL: <https://arxiv.org/abs/2602.24281>.
- [44] Zihan Qiu et al. “Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free”. In: *arXiv preprint arXiv:2505.06708* (2025). arXiv: 2505.06708. URL: <https://arxiv.org/abs/2505.06708>.
- [45] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. “Recurrent Memory Transformer”. In: *arXiv preprint arXiv:2207.06881* (2022). arXiv: 2207.06881. URL: <https://arxiv.org/abs/2207.06881>.
- [46] Fanxu Meng et al. “TransMLA: Multi-Head Latent Attention Is All You Need”. In: *arXiv preprint arXiv:2502.07864* (2025). arXiv: 2502.07864. URL: <https://arxiv.org/abs/2502.07864>.
- [47] Zihang Dai et al. “Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context”. In: *arXiv preprint arXiv:1901.02860* (2019). arXiv: 1901.02860. URL: <https://arxiv.org/abs/1901.02860>.
- [48] Jack W. Rae et al. “Compressive Transformers for Long-Range Sequence Modelling”. In: *arXiv preprint arXiv:1911.05507* (2019). arXiv: 1911.05507. URL: <https://arxiv.org/abs/1911.05507>.
- [49] Dongseong Hwang et al. “TransformerFAM: Feedback attention is working memory”. In: *arXiv preprint arXiv:2404.09173* (2024). arXiv: 2404.09173. URL: <https://arxiv.org/abs/2404.09173>.
- [50] DeepSeek-AI et al. “DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence”. In: *arXiv preprint arXiv:2606.19348* (2026). arXiv: 2606.19348. URL: <https://arxiv.org/abs/2606.19348>.
- [51] Noam Shazeer et al. “Talking-Heads Attention”. In: *arXiv preprint arXiv:2003.02436* (2020). arXiv: 2003.02436. URL: <https://arxiv.org/abs/2003.02436>.
- [52] Da Xiao et al. “Improving Transformers with Dynamically Composable Multi-Head Attention”. In: *arXiv preprint arXiv:2405.08553* (2024). arXiv: 2405.08553. URL: <https://arxiv.org/abs/2405.08553>.
- [53] Tianzhu Ye et al. “Differential Transformer”. In: *arXiv preprint arXiv:2410.05258* (2024). arXiv: 2410.05258. URL: <https://arxiv.org/abs/2410.05258>.
- [54] Zhixuan Lin et al. “Forgetting Transformer: Softmax Attention with a Forget Gate”. In: *arXiv preprint arXiv:2503.02130* (2025). arXiv: 2503.02130. URL: <https://arxiv.org/abs/2503.02130>.
- [55] Peng Jin et al. “MoH: Multi-Head Attention as Mixture-of-Head Attention”. In: *arXiv preprint arXiv:2410.11842* (2024). arXiv: 2410.11842. URL: <https://arxiv.org/abs/2410.11842>.
- [56] Aaron Grattafiori et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024). arXiv: 2407.21783. URL: <https://arxiv.org/abs/2407.21783>.
- [57] Meta AI. *The Llama 4 Herd: Scout and Maverick*. Official model card, release blog, and reference implementation. Apr. 2025. URL: <https://github.com/meta-llama/llama-models/tree/main/models/llama4> (visited on 09/29/2026).
- [58] An Yang et al. “Qwen2 Technical Report”. In: *arXiv preprint arXiv:2407.10671* (2024). arXiv: 2407.10671. URL: <https://arxiv.org/abs/2407.10671>.
- [59] An Yang et al. “Qwen3 Technical Report”. In: *arXiv preprint arXiv:2505.09388* (2025). arXiv: 2505.09388. URL: <https://arxiv.org/abs/2505.09388>.
- [60] Qwen Team. *Qwen3.5 Model Configuration and Model Card*. Official Hugging Face model release. Feb. 2026. URL: <https://huggingface.co/Qwen/Qwen3.5-72B> (visited on 09/29/2026).
- [61] Qwen Team. *Qwen3.6-35B-A3B Model Card*. Hugging Face model card. Apr. 2026. URL: <https://huggingface.co/Qwen/Qwen3.6-35B-A3B> (visited on 09/29/2026).
- [62] Qwen Team. *Qwen3.6-72B Model Card*. Hugging Face model card. Apr. 2026. URL: <https://huggingface.co/Qwen/Qwen3.6-72B> (visited on 09/29/2026).
- [63] DeepSeek-AI et al. “DeepSeek-V3 Technical Report”. In: *arXiv preprint arXiv:2412.19437* (2024). arXiv: 2412.19437. URL: <https://arxiv.org/abs/2412.19437>.

- [64] DeepSeek-AI et al. “DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models”. In: *arXiv preprint arXiv:2512.02556* (2025). arXiv: 2512.02556. URL: <https://arxiv.org/abs/2512.02556>.
- [65] Moonshot AI. *Kimi K2*. Official model repository. 2025. URL: <https://github.com/MoonshotAI/Kimi-K2> (visited on 09/29/2026).
- [66] Kimi Team et al. “Kimi K3: Open Frontier Intelligence”. In: *arXiv preprint arXiv:2607.24653* (2026). arXiv: 2607.24653. URL: <https://arxiv.org/abs/2607.24653>.
- [67] GLM-5-Team et al. “GLM-5: From Vibe Coding to Agentic Engineering”. In: *arXiv preprint arXiv:2602.15763* (2026). arXiv: 2602.15763. URL: <https://arxiv.org/abs/2602.15763>.
- [68] Z.ai. *GLM-5.2*. Official open-source model and model card. June 2026. URL: <https://huggingface.co/zai-org/GLM-5.2> (visited on 09/29/2026).
- [69] Z.ai. *GLM-5.3*. Official model repository. Aug. 2026. URL: <https://huggingface.co/zai-org/GLM-5.3> (visited on 09/29/2026).
- [70] Rewon Child et al. “Generating Long Sequences with Sparse Transformers”. In: *arXiv preprint arXiv:1904.10509* (2019). arXiv: 1904.10509. URL: <https://arxiv.org/abs/1904.10509>.
- [71] Jiayu Ding et al. “LongNet: Scaling Transformers to 1,000,000,000 Tokens”. In: *arXiv preprint arXiv:2307.02486* (2023). arXiv: 2307.02486. URL: <https://arxiv.org/abs/2307.02486>.
- [72] Lida Chen et al. “PowerAttention: Exponentially Scaling of Receptive Fields for Effective Sparse Attention”. In: *arXiv preprint arXiv:2503.03588* (2025). arXiv: 2503.03588. URL: <https://arxiv.org/abs/2503.03588>.
- [73] Guangxuan Xiao et al. “Efficient Streaming Language Models with Attention Sinks”. In: *arXiv preprint arXiv:2309.17453* (2023). arXiv: 2309.17453. URL: <https://arxiv.org/abs/2309.17453>.
- [74] Chi Han et al. “LM-Infinite: Zero-Shot Extreme Length Generalization for Large Language Models”. In: *arXiv preprint arXiv:2308.16137* (2023). arXiv: 2308.16137. URL: <https://arxiv.org/abs/2308.16137>.
- [75] Huiqiang Jiang et al. “MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention”. In: *arXiv preprint arXiv:2407.02490* (2024). arXiv: 2407.02490. URL: <https://arxiv.org/abs/2407.02490>.
- [76] Nikita Kitaev, ukasz Kaiser, and Anselm Levskaya. “Reformer: The Efficient Transformer”. In: *International Conference on Learning Representations*. 2020. arXiv: 2001.04451. URL: <https://arxiv.org/abs/2001.04451>.
- [77] Aurko Roy et al. “Efficient Content-Based Sparse Attention with Routing Transformers”. In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 53–68. DOI: 10.1162/tac1_a_00353. arXiv: 2003.05997. URL: <https://arxiv.org/abs/2003.05997>.
- [78] Jiaming Tang et al. “Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference”. In: *arXiv preprint arXiv:2406.10774* (2024). arXiv: 2406.10774. URL: <https://arxiv.org/abs/2406.10774>.
- [79] Ruyi Xu et al. “XAttention: Block Sparse Attention with Antidiagonal Scoring”. In: *arXiv preprint arXiv:2503.16428* (2025). arXiv: 2503.16428. URL: <https://arxiv.org/abs/2503.16428>.
- [80] Enzhe Lu et al. “MoBA: Mixture of Block Attention for Long-Context LLMs”. In: *arXiv preprint arXiv:2502.13189* (2025). arXiv: 2502.13189. URL: <https://arxiv.org/abs/2502.13189>.
- [81] Guangxuan Xiao et al. “Optimizing Mixture of Block Attention”. In: *arXiv preprint arXiv:2511.11571* (2025). arXiv: 2511.11571. URL: <https://arxiv.org/abs/2511.11571>.
- [82] Weilin Zhao et al. “InflLLM-V2: Dense-Sparse Switchable Attention for Seamless Short-to-Long Adaptation”. In: *arXiv preprint arXiv:2509.24663* (2025). arXiv: 2509.24663. URL: <https://arxiv.org/abs/2509.24663>.
- [83] Yuxiang Huang et al. “DashAttention: Differentiable and Adaptive Sparse Hierarchical Attention”. In: *arXiv preprint arXiv:2605.18753* (2026). arXiv: 2605.18753. URL: <https://arxiv.org/abs/2605.18753>.
- [84] Alexander Tian et al. “COBS: Cumulant Order Block Sparse Attention”. In: *arXiv preprint arXiv:2607.09052* (2026). arXiv: 2607.09052. URL: <https://arxiv.org/abs/2607.09052>.
- [85] Amirkeivan Mohtashami and Martin Jaggi. “Landmark Attention: Random-Access Infinite Context Length for Transformers”. In: *arXiv preprint arXiv:2305.16300* (2023). arXiv: 2305.16300. URL: <https://arxiv.org/abs/2305.16300>.
- [86] Yuzhen Mao, Michael Y. Li, and Emily B. Fox. “Simplified Sparse Attention via Gist Tokens”. In: *arXiv preprint arXiv:2604.20920* (2026). arXiv: 2604.20920. URL: <https://arxiv.org/abs/2604.20920>.
- [87] Yash Akhauri et al. “TokenButler: Token Importance is Predictable”. In: *arXiv preprint arXiv:2503.07518* (2025). arXiv: 2503.07518. URL: <https://arxiv.org/abs/2503.07518>.

- [88] Zhiwei Li et al. “SAS: Simple Attention Sparsification via End-to-End Optimization of Context Ranking”. In: *arXiv preprint arXiv:2609.13141* (2026). arXiv: 2609.13141. URL: <https://arxiv.org/abs/2609.13141>.
- [89] Aditya Desai et al. “HashAttention: Semantic Sparsity for Faster Inference”. In: *arXiv preprint arXiv:2412.14468* (2024). arXiv: 2412.14468. URL: <https://arxiv.org/abs/2412.14468>.
- [90] Ping Gong et al. “HATA: Trainable and Hardware-Efficient Hash-Aware Top-k Attention for Scalable Large Model Inference”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. 2025. arXiv: 2506.02572. URL: <https://arxiv.org/abs/2506.02572>.
- [91] Yizhao Gao et al. “SeerAttention: Learning Intrinsic Sparse Attention in Your LLMs”. In: *arXiv preprint arXiv:2410.13276* (2024). arXiv: 2410.13276. URL: <https://arxiv.org/abs/2410.13276>.
- [92] Yizhao Gao et al. “SeerAttention-R: Sparse Attention Adaptation for Long Reasoning”. In: *arXiv preprint arXiv:2506.08889* (2025). arXiv: 2506.08889. URL: <https://arxiv.org/abs/2506.08889>.
- [93] Huzama Ahmad and Se-Young Yun. “SpotAttention: Plug-In Block-Sparse Routing for Pretrained Long-Context Transformers”. In: *arXiv preprint arXiv:2606.22874* (2026). arXiv: 2606.22874. URL: <https://arxiv.org/abs/2606.22874>.
- [94] Xunhao Lai et al. “MiniMax Sparse Attention”. In: *arXiv preprint arXiv:2606.13392* (2026). arXiv: 2606.13392. URL: <https://arxiv.org/abs/2606.13392>.
- [95] MiniMax Team. *MiniMax-M3*. Official model release accompanying MiniMax Sparse Attention. 2026. URL: <https://huggingface.co/MiniMaxAI/MiniMax-M3> (visited on 09/29/2026).
- [96] Qwen Team. *Qwen3.8-Flash-Next Technical Report and Model Card*. Official technical report and model release. Introduces Qwen Sparse Attention (QSA). Aug. 2026. URL: <https://github.com/QwenLM/Qwen3.8-Flash-Next> (visited on 09/29/2026).
- [97] Wenshuai Yao et al. “Recall Before You Rank: Similarity-Guided Top- K Reuse for Efficient Long-Context Attention”. In: *arXiv preprint arXiv:2607.27692* (2026). arXiv: 2607.27692. URL: <https://arxiv.org/abs/2607.27692>.
- [98] Lijie Yang et al. “TidalDecode: Fast and Accurate LLM Decoding with Position Persistent Sparse Attention”. In: *arXiv preprint arXiv:2410.05076* (2024). arXiv: 2410.05076. URL: <https://arxiv.org/abs/2410.05076>.
- [99] Dhruv Deshmukh et al. “Kascade: A Practical Sparse Attention Method for Long-Context LLM Inference”. In: *arXiv preprint arXiv:2512.16391* (2025). arXiv: 2512.16391. URL: <https://arxiv.org/abs/2512.16391>.
- [100] Yutao Sun et al. “You Only Index Once: Cross-Layer Sparse Attention with Shared Routing”. In: *arXiv preprint arXiv:2606.06467* (2026). arXiv: 2606.06467. URL: <https://arxiv.org/abs/2606.06467>.
- [101] DeepSeek-AI. *DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression*. 2026. arXiv: 2609.19969 [cs.CL]. URL: <https://arxiv.org/abs/2609.19969>.
- [102] Yizhao Gao et al. “HySparse: A Hybrid Sparse Attention Architecture with Oracle Token Selection and KV Cache Sharing”. In: *arXiv preprint arXiv:2602.03560* (2026). arXiv: 2602.03560. URL: <https://arxiv.org/abs/2602.03560>.
- [103] Jianyu Wei et al. “HySparse2: Hybrid Sparse Attention with Two-Level KV Sharing”. In: *arXiv preprint arXiv:2609.26368* (2026). arXiv: 2609.26368. URL: <https://arxiv.org/abs/2609.26368>.
- [104] Meituan LongCat Team. *LongCat-2.0*. Official model repository and technical release. LongCat Sparse Attention language-model release. June 2026. URL: <https://huggingface.co/meituan-longcat/LongCat-2.0> (visited on 09/22/2026).
- [105] Songlin Yang et al. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length”. In: *arXiv preprint arXiv:2406.06484* (2024). arXiv: 2406.06484. URL: <https://arxiv.org/abs/2406.06484>.
- [106] Julien Siems et al. “DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products”. In: *arXiv preprint arXiv:2502.10297* (2025). arXiv: 2502.10297. URL: <https://arxiv.org/abs/2502.10297>.
- [107] MiniMax et al. “MiniMax-01: Scaling Foundation Models with Lightning Attention”. In: *arXiv preprint arXiv:2501.08313* (2025). arXiv: 2501.08313. URL: <https://arxiv.org/abs/2501.08313>.
- [108] Bo Peng et al. “Eagle and Finch: RWKV with Matrix-Valued States and Dynamic Recurrence”. In: *arXiv preprint arXiv:2404.05892* (2024). arXiv: 2404.05892. URL: <https://arxiv.org/abs/2404.05892>.
- [109] Bo Peng et al. “RWKV: Reinventing RNNs for the Transformer Era”. In: *arXiv preprint arXiv:2305.13048* (2023). arXiv: 2305.13048. URL: <https://arxiv.org/abs/2305.13048>.
- [110] Songlin Yang et al. “Gated Linear Attention Transformers with Hardware-Efficient Training”. In: *arXiv preprint arXiv:2312.06635* (2023). arXiv: 2312.06635. URL: <https://arxiv.org/abs/2312.06635>.

- [111] Zhen Qin et al. “HGRN2: Gated Linear RNNs with State Expansion”. In: *arXiv preprint arXiv:2404.07904* (2024). arXiv: 2404.07904. URL: <https://arxiv.org/abs/2404.07904>.
- [112] Bo Peng et al. “RWKV-7 "Goose" with Expressive Dynamic State Evolution”. In: *arXiv preprint arXiv:2503.14456* (2025). arXiv: 2503.14456. URL: <https://arxiv.org/abs/2503.14456>.
- [113] Kimi Team et al. “Kimi Linear: An Expressive, Efficient Attention Architecture”. In: *arXiv preprint arXiv:2510.26692* (2025). arXiv: 2510.26692. URL: <https://arxiv.org/abs/2510.26692>.
- [114] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. “Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention”. In: *arXiv preprint arXiv:2605.22791* (2026). arXiv: 2605.22791. URL: <https://arxiv.org/abs/2605.22791>.
- [115] Xiao Li et al. “Erase-then-Delta Attention: Decoupling Erase and Write Addresses in Delta-Rule Linear Attention”. In: *arXiv preprint arXiv:2606.26560* (2026). arXiv: 2606.26560. URL: <https://arxiv.org/abs/2606.26560>.
- [116] Yu Sun et al. “Learning to (Learn at Test Time): RNNs with Expressive Hidden States”. In: *arXiv preprint arXiv:2407.04620* (2024). arXiv: 2407.04620. URL: <https://arxiv.org/abs/2407.04620>.
- [117] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. “Test-time regression: a unifying framework for designing sequence models with associative memory”. In: *Journal of Machine Learning Research* 27.169 (2026), pp. 1–41. URL: <https://www.jmlr.org/papers/v27/25-0903.html>.
- [118] Kewei Zhang et al. “MHLA: Restoring Expressivity of Linear Attention via Token-Level Multi-Head”. In: *arXiv preprint arXiv:2601.07832* (2026). DOI: 10.48550/arXiv.2601.07832. arXiv: 2601.07832. URL: <https://arxiv.org/abs/2601.07832>.
- [119] Mingwei Xu et al. “Softmax Linear Attention: Reclaiming Global Competition”. In: *arXiv preprint arXiv:2602.01744* (2026). DOI: 10.48550/arXiv.2602.01744. arXiv: 2602.01744. URL: <https://arxiv.org/abs/2602.01744>.
- [120] Tommaso Cerruti et al. “Linear Attention Architectures: Mechanisms, Trade-offs, and Cross-Layer Routing”. In: *arXiv preprint arXiv:2607.07953* (2026). DOI: 10.48550/arXiv.2607.07953. arXiv: 2607.07953. URL: <https://arxiv.org/abs/2607.07953>.
- [121] Aakash Lahoti et al. “Mamba-3: Improved Sequence Modeling using State Space Principles”. In: *arXiv preprint arXiv:2603.15569* (2026). arXiv: 2603.15569. URL: <https://arxiv.org/abs/2603.15569>.
- [122] Yanbo Li et al. “MIMOMamba: From Scalar Duality to Matrix-Valued Attention”. In: *International Conference on Machine Learning*. 2026. URL: <https://openreview.net/forum?id=UmQ07sj13y>.
- [123] Albert Gu et al. “HiPPO: Recurrent Memory with Optimal Polynomial Projections”. In: *Advances in Neural Information Processing Systems* 33 (2020). arXiv: 2008.07669. URL: <https://arxiv.org/abs/2008.07669>.
- [124] Albert Gu et al. “Combining Recurrent, Convolutional, and Continuous-time Models with Linear State-Space Layers”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021. arXiv: 2110.13985.
- [125] Albert Gu et al. “On the Parameterization and Initialization of Diagonal State Space Models”. In: *arXiv preprint arXiv:2206.11893* (2022). arXiv: 2206.11893. URL: <https://arxiv.org/abs/2206.11893>.
- [126] Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. “Simplified State Space Layers for Sequence Modeling”. In: *arXiv preprint arXiv:2208.04933* (2022). arXiv: 2208.04933. URL: <https://arxiv.org/abs/2208.04933>.
- [127] Daniel Y. Fu et al. “Hungry Hungry Hippos: Towards Language Modeling with State Space Models”. In: *arXiv preprint arXiv:2212.14052* (2022). arXiv: 2212.14052. URL: <https://arxiv.org/abs/2212.14052>.
- [128] Simran Arora et al. “Zoology: Measuring and Improving Recall in Efficient Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2024. arXiv: 2312.04927. URL: <https://arxiv.org/html/2312.04927v1>.
- [129] Yehjin Shin, Seojin Kim, and Noseong Park. “Graph Signal Processing Meets Mamba2: Adaptive Filter Bank via Delta Modulation”. In: *International Conference on Learning Representations*. 2026. URL: <https://openreview.net/forum?id=w0XhHcXfKv>.
- [130] Thai-Khanh Nguyen et al. “MuonSSM: Orthogonalizing State Space Models for Sequence Modeling”. In: *International Conference on Machine Learning*. 2026. arXiv: 2606.30461. URL: <https://arxiv.org/abs/2606.30461>.
- [131] William Merrill, Jackson Petty, and Ashish Sabharwal. “The Illusion of State in State-Space Models”. In: *International Conference on Machine Learning (ICML)*. 2024. arXiv: 2404.08819. URL: <https://arxiv.org/html/2404.08819v3>.

- [132] Yash Akhauri, Safeen Huda, and Mohamed S. Abdelfattah. “Attamba: Attending To Multi-Token States”. In: *arXiv preprint arXiv:2411.17685* (2024). arXiv: 2411.17685. URL: <https://arxiv.org/abs/2411.17685>.
- [133] Yixiao Qian et al. “DART: Decoded Attention over Recurrent States for Efficient Long-Context Sequence Modeling”. In: *arXiv preprint arXiv:2608.02032* (2026). arXiv: 2608.02032. URL: <https://arxiv.org/abs/2608.02032>.
- [134] Liliang Ren et al. “Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling”. In: *arXiv preprint arXiv:2406.07522* (2024). arXiv: 2406.07522. URL: <https://arxiv.org/abs/2406.07522>.
- [135] Soham De et al. “Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models”. In: *arXiv preprint arXiv:2402.19427* (2024). arXiv: 2402.19427. URL: <https://arxiv.org/abs/2402.19427>.
- [136] Paolo Glorioso et al. “Zamba: A Compact 7B SSM Hybrid Model”. In: *arXiv preprint arXiv:2405.16712* (2024). arXiv: 2405.16712. URL: <https://arxiv.org/abs/2405.16712>.
- [137] Paolo Glorioso et al. “The Zamba2 Suite: Technical Report”. In: *arXiv preprint arXiv:2411.15242* (2024). arXiv: 2411.15242. URL: <https://arxiv.org/abs/2411.15242>.
- [138] Xuan Zhang et al. “LightTransfer: Your Long-Context LLM is Secretly a Hybrid Model with Effortless Adaptation”. In: *arXiv preprint arXiv:2410.13846* (2024). arXiv: 2410.13846. URL: <https://arxiv.org/abs/2410.13846>.
- [139] Aditya Chattopadhyay et al. “Priming: Hybrid State Space Models From Pre-trained Transformers”. In: *arXiv preprint arXiv:2605.08301* (2026). arXiv: 2605.08301. URL: <https://arxiv.org/abs/2605.08301>.
- [140] Yuxian Gu et al. “Jet-Nemotron: Efficient Language Model with Post Neural Architecture Search”. In: *Advances in Neural Information Processing Systems*. 2025. arXiv: 2508.15884. URL: <https://arxiv.org/abs/2508.15884>.
- [141] Yingfa Chen et al. “Hybrid Linear Attention Done Right: Efficient Distillation and Effective Architectures for Extremely Long Contexts”. In: *arXiv preprint arXiv:2601.22156* (2026). arXiv: 2601.22156. URL: <https://arxiv.org/abs/2601.22156>.
- [142] Yanhong Li et al. “Distilling to Hybrid Attention Models via KL-Guided Layer Selection”. In: *arXiv preprint arXiv:2512.20569* (2025). Accepted as an ICLR 2026 poster. arXiv: 2512.20569. URL: <https://arxiv.org/abs/2512.20569>.
- [143] Jingwei Zuo et al. “Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance”. In: *arXiv preprint arXiv:2507.22448* (2025). arXiv: 2507.22448. URL: <https://arxiv.org/abs/2507.22448>.
- [144] Guangxuan Xiao et al. “DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads”. In: *arXiv preprint arXiv:2410.10819* (2024). arXiv: 2410.10819. URL: <https://arxiv.org/abs/2410.10819>.
- [145] Zhentao Tan et al. “HydraHead: From Head-Level Functional Heterogeneity to Specialized Attention Hybridization”. In: *arXiv preprint arXiv:2606.20097* (2026). arXiv: 2606.20097. URL: <https://arxiv.org/abs/2606.20097>.
- [146] Zecheng Tang et al. “Elastic Attention: Test-time Adaptive Sparsity Ratios for Efficient Transformers”. In: *arXiv preprint arXiv:2601.17367* (2026). arXiv: 2601.17367. URL: <https://arxiv.org/abs/2601.17367>.
- [147] Runlin Shi, Bojian Yin, and Guoqi Li. “Modern Transformers Are Implicit Hybrids: From Functional Differentiation to Principled Hybrid Architecture Design”. In: *arXiv preprint arXiv:2609.02986* (2026). arXiv: 2609.02986. URL: <https://arxiv.org/abs/2609.02986>.
- [148] DeLesley Hutchins et al. “Block-Recurrent Transformers”. In: *Advances in Neural Information Processing Systems*. Vol. 35. Curran Associates, Inc., 2022. arXiv: 2203.07852. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/d6e0bbb9fc3f4c10950052ec2359355c-Abstract-Conference.html.
- [149] Yuhuai Wu et al. “Memorizing Transformers”. In: *arXiv preprint arXiv:2203.08913* (2022). arXiv: 2203.08913. URL: <https://arxiv.org/abs/2203.08913>.
- [150] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. “Titans: Learning to Memorize at Test Time”. In: *arXiv preprint arXiv:2501.00663* (2024). arXiv: 2501.00663. URL: <https://arxiv.org/abs/2501.00663>.
- [151] Michael Zhang et al. “LoLCATs: On Low-Rank Linearizing of Large Language Models”. In: *arXiv preprint arXiv:2410.10254* (2024). arXiv: 2410.10254. URL: <https://arxiv.org/abs/2410.10254>.

- [152] Luke McDermott, Robert W. Heath Jr., and Rahul Parhi. “LoLA: Low-Rank Linear Attention With Sparse Caching”. In: *arXiv preprint arXiv:2505.23666* (2025). arXiv: 2505.23666. URL: <https://arxiv.org/abs/2505.23666>.
- [153] Weikang Meng et al. “STILL: Selecting Tokens for Intra-Layer Hybrid Attention to Linearize LLMs”. In: *arXiv preprint arXiv:2602.02180* (2026). arXiv: 2602.02180. URL: <https://arxiv.org/abs/2602.02180>.
- [154] Difan Deng et al. “Neural Attention Search Linear: Towards Adaptive Token-Level Hybrid Attention Models”. In: *arXiv preprint arXiv:2602.03681* (2026). arXiv: 2602.03681. URL: <https://arxiv.org/abs/2602.03681>.
- [155] Artificial Analysis. *Artificial Analysis Intelligence Index v4.3.2*. Composite model-capability benchmark comprising ten independently conducted evaluations. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index> (visited on 09/21/2026).
- [156] Artificial Analysis. *Intelligence Benchmarking Methodology*. Methodology for the Artificial Analysis intelligence evaluations and model comparisons. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/methodology/intelligence-benchmarking> (visited on 09/21/2026).
- [157] Artificial Analysis. *Artificial Analysis Intelligence Index: Selected Open-Weight Model Comparison*. Dynamic leaderboard comparison view filtered to the eleven open-weight model endpoints reported in the manuscript; accessed September 22, 2026. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/?models=mimo-v2-6-pro%2Cglm-5-3%2Ckimi-k3%2Cglm-5-3-flash%2Cqwen3-8-2-4t-a95b%2Cqwen3-8-flash-next%2Cdeepseek-v4-1-flash%2Cdeepseek-v4-pro%2Cqwen3-8-27b%2Ck2-horizon-375b-a23b%2Cminimax-m3&model-filters=open-source> (visited on 09/22/2026).
- [158] Mistral AI. *Mistral Small 3.1 24B*. Official Hugging Face model card and released configuration. Mar. 2025. URL: <https://huggingface.co/mistralai/Mistral-Small-3.1-24B-Base-2503> (visited on 09/21/2026).
- [159] Aili Chen et al. “The MiniMax-M2 Series: Mini Activations Unleashing Max Real-World Intelligence”. In: *arXiv preprint arXiv:2605.26494* (2026). arXiv: 2605.26494. URL: <https://arxiv.org/abs/2605.26494>.
- [160] OpenBMB. *MiniCPM5-2B Model Card and Configuration*. Hugging Face model release. 2026. URL: <https://huggingface.co/openbmb/MiniCPM5-2B> (visited on 09/29/2026).
- [161] IFM Team. *K2-Horizon-375B-A23B*. Official Hugging Face model card and released configuration. Sept. 2026. URL: <https://huggingface.co/IFM/K2-Horizon-375B-A23B> (visited on 09/21/2026).
- [162] Meituan LongCat Team et al. “LongCat-Flash Technical Report”. In: *arXiv preprint arXiv:2509.01322* (Sept. 2025). arXiv: 2509.01322 [cs.CL]. URL: <https://arxiv.org/abs/2509.01322>.
- [163] Hong Liu et al. “Scaling Embeddings Outperforms Scaling Experts in Language Models”. In: *arXiv preprint arXiv:2601.21204* (Jan. 2026). arXiv: 2601.21204 [cs.CL]. URL: <https://arxiv.org/abs/2601.21204>.
- [164] Qwen Team. *Qwen3-Next-80B-A3B Model Card*. Hugging Face model card. 2025. URL: <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct> (visited on 09/29/2026).
- [165] MiniMax et al. “MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention”. In: *arXiv preprint arXiv:2506.13585* (2025). arXiv: 2506.13585. URL: <https://arxiv.org/abs/2506.13585>.
- [166] NVIDIA et al. “NVIDIA Nemotron 3: Efficient and Open Intelligence”. In: *arXiv preprint arXiv:2512.20856* (2025). arXiv: 2512.20856. URL: <https://arxiv.org/abs/2512.20856>.
- [167] Jinze Bai et al. “Qwen Technical Report”. In: *arXiv preprint arXiv:2309.16609* (2023). arXiv: 2309.16609. URL: <https://arxiv.org/abs/2309.16609>.
- [168] LLM-Core Xiaomi. *MiMo-V2.6: Scaling Reinforcement Learning Towards Self-Improvement*. Tech. rep. Xiaomi, Sept. 2026. URL: https://huggingface.co/XiaomiMiMo/MiMo-V2.6-Pro-RL/blob/main/MiMo_V2_6_technical_report.pdf (visited on 09/29/2026).
- [169] Qwen Team. *Qwen3.8-27B Model Card*. Hugging Face model card. Aug. 2026. URL: <https://huggingface.co/Qwen/Qwen3.8-27B> (visited on 09/29/2026).
- [170] Xin Cheng et al. “Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models”. In: *arXiv preprint arXiv:2601.07372* (2026). arXiv: 2601.07372 [cs.CL]. URL: <https://arxiv.org/abs/2601.07372>.

- [171] David Raposo et al. “Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models”. In: *arXiv preprint arXiv:2404.02258* (2024). arXiv: 2404.02258 [cs.LG]. URL: <https://arxiv.org/abs/2404.02258>.
- [172] Z.ai. *GLM-5.3-Flash*. Official open-source model, configuration, and model card. Aug. 2026. URL: <https://huggingface.co/zai-org/GLM-5.3-Flash> (visited on 09/29/2026).
- [173] Kimi Team et al. “Attention Residuals”. In: *arXiv preprint arXiv:2603.15031* (2026). arXiv: 2603.15031. URL: <https://arxiv.org/abs/2603.15031>.
- [174] Gemma Team et al. “Gemma 4 Technical Report”. In: *arXiv preprint arXiv:2607.02770* (2026). arXiv: 2607.02770. URL: <https://arxiv.org/abs/2607.02770>.
- [175] Mistral AI. *Mistral Medium 3.5 128B*. Official Hugging Face model card and released configuration. Apr. 2026. URL: <https://huggingface.co/mistralai/Mistral-Medium-3.5-128B> (visited on 09/21/2026).
- [176] MiniCPM Team et al. “MiniCPM-SALA: Hybridizing Sparse and Linear Attention for Efficient Long-Context Modeling”. In: *arXiv preprint arXiv:2602.11761* (2026). arXiv: 2602.11761. URL: <https://arxiv.org/abs/2602.11761>.
- [177] Xiaomi LLM-Core Team et al. “MiMo-V2-Flash Technical Report”. In: *arXiv preprint arXiv:2601.02780* (2026). arXiv: 2601.02780. URL: <https://arxiv.org/abs/2601.02780>.
- [178] OpenBMB. *MiniCPM4.1-8B Model Card and Configuration*. Hugging Face model release. 2025. URL: <https://huggingface.co/openbmb/MiniCPM4.1-8B> (visited on 09/29/2026).
- [179] NVIDIA et al. “NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model”. In: *arXiv preprint arXiv:2508.14444* (2025). arXiv: 2508.14444. URL: <https://arxiv.org/abs/2508.14444>.
- [180] MiniCPM Team et al. “MiniCPM4: Ultra-Efficient LLMs on End Devices”. In: *arXiv preprint arXiv:2506.07900* (2025). arXiv: 2506.07900. URL: <https://arxiv.org/abs/2506.07900>.
- [181] LLM-Core Xiaomi et al. “MiMo: Unlocking the Reasoning Potential of Language Model – From Pretraining to Posttraining”. In: *arXiv preprint arXiv:2505.07608* (2025). arXiv: 2505.07608. URL: <https://arxiv.org/abs/2505.07608>.
- [182] NVIDIA et al. “Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models”. In: *arXiv preprint arXiv:2504.03624* (2025). arXiv: 2504.03624. URL: <https://arxiv.org/abs/2504.03624>.
- [183] Gemma Team et al. “Gemma 3 Technical Report”. In: *arXiv preprint arXiv:2503.19786* (2025). arXiv: 2503.19786. URL: <https://arxiv.org/abs/2503.19786>.
- [184] Kimi Team et al. “Kimi k1.5: Scaling Reinforcement Learning with LLMs”. In: *arXiv preprint arXiv:2501.12599* (2025). arXiv: 2501.12599. URL: <https://arxiv.org/abs/2501.12599>.
- [185] OpenBMB. *MiniCPM3-4B Model Card and Configuration*. Hugging Face model release. 2024. URL: <https://huggingface.co/openbmb/MiniCPM3-4B> (visited on 09/29/2026).
- [186] Gemma Team et al. “Gemma 2: Improving Open Language Models at a Practical Size”. In: *arXiv preprint arXiv:2408.00118* (2024). arXiv: 2408.00118. URL: <https://arxiv.org/abs/2408.00118>.
- [187] Team GLM et al. “ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools”. In: *arXiv preprint arXiv:2406.12793* (2024). arXiv: 2406.12793. URL: <https://arxiv.org/abs/2406.12793>.
- [188] Mistral AI. *Mixtral-8x22B-v0.1*. Official Hugging Face model card and released configuration. Apr. 2024. URL: <https://huggingface.co/mistralai/Mixtral-8x22B-v0.1> (visited on 09/21/2026).
- [189] Gemma Team et al. “Gemma: Open Models Based on Gemini Research and Technology”. In: *arXiv preprint arXiv:2403.08295* (2024). arXiv: 2403.08295. URL: <https://arxiv.org/abs/2403.08295>.
- [190] Mistral AI. *Mixtral-8x7B-v0.1*. Official Hugging Face model card and released configuration. Dec. 2023. URL: <https://huggingface.co/mistralai/Mixtral-8x7B-v0.1> (visited on 09/21/2026).
- [191] Mistral AI. *Mistral-7B-v0.1*. Official Hugging Face model card and released configuration. Sept. 2023. URL: <https://huggingface.co/mistralai/Mistral-7B-v0.1> (visited on 09/21/2026).
- [192] Hugo Touvron et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint arXiv:2307.09288* (2023). arXiv: 2307.09288. URL: <https://arxiv.org/abs/2307.09288>.
- [193] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023). arXiv: 2302.13971. URL: <https://arxiv.org/abs/2302.13971>.
- [194] Aohan Zeng et al. “GLM-130B: An Open Bilingual Pre-trained Model”. In: *arXiv preprint arXiv:2210.02414* (2022). arXiv: 2210.02414. URL: <https://arxiv.org/abs/2210.02414>.